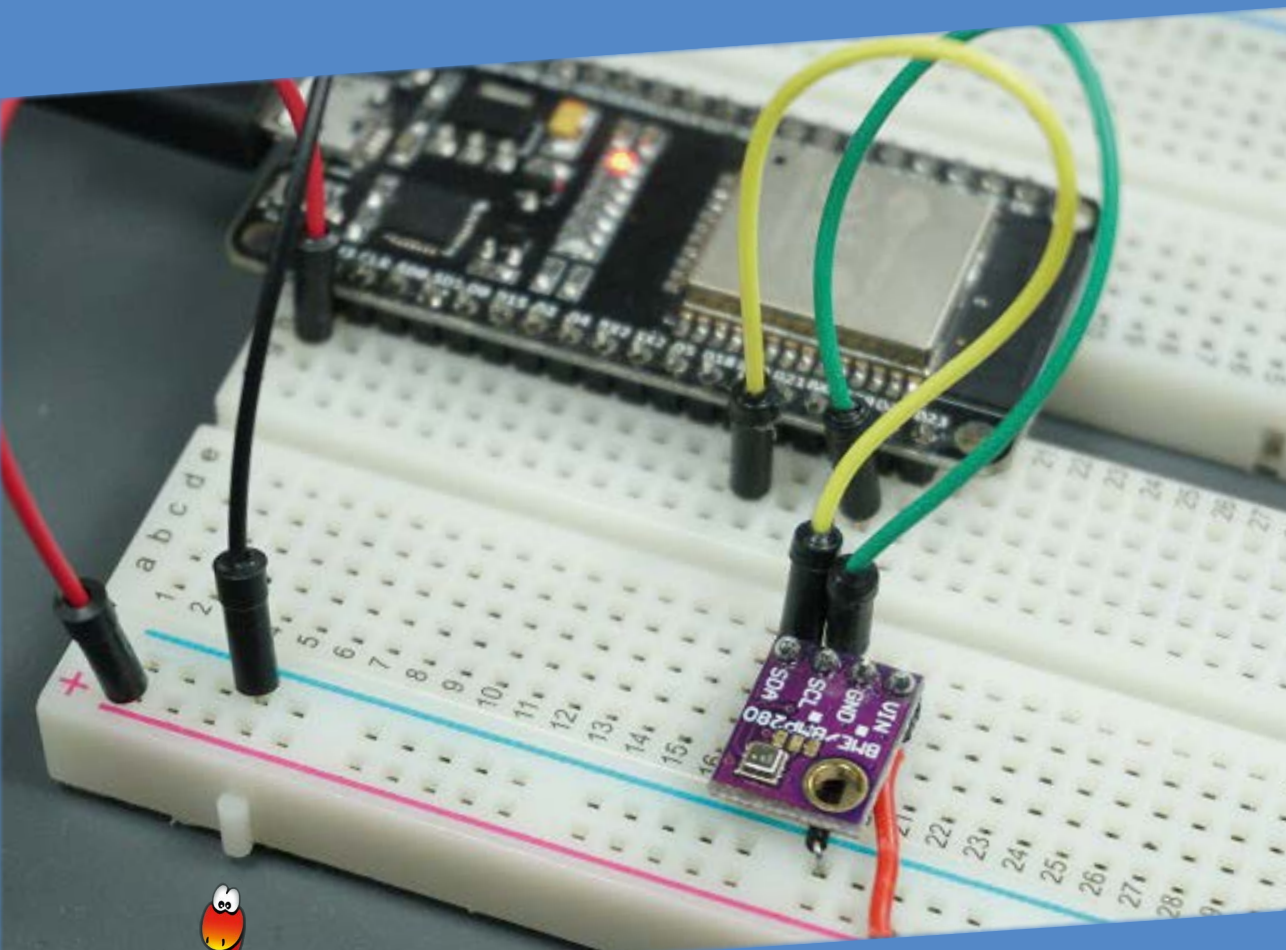


MicroPython для микроконтроллеров

Проекты с Thonny-IDE, uPyCraft-IDE и ESP32



Гюнтер Спаннер

MicroPython для микроконтроллеров

Проекты с Thonny-IDE, uPyCraft-IDE и ESP32



Гюнтер Спаннер

Глава 1 • Введение	11
1.1 Python, C, или Arduino?	12
1.2 1.2 Требования	12
Глава 2 • Разнообразие плат ESP	14
2.1 Ввод в эксплуатацию и функциональная проверка	16
2.2 ESP32 при питании от батареи	17
Глава 3 • Среды программирования и разработки	19
3.1 Установка uPyCraft IDE	19
3.2 MicroPython для ESP32	22
3.3 "Hello World" для контроллера	23
3.4 Для профессионалов: Работа с esptool	27
3.5 Thonny — Python-IDE для начинающих	31
3.6 Работа с Тонни	34
3.7 Работа с файлами	36
3.8 Советы по устранению неполадок для Thonny IDE	36
Глава 4 • Первые шаги в программировании	39
4.1 Никогда без комментариев	41
4.2 Оператор Print()	42
4.3 Отступы и блоки	44
4.4 Управляемое оборудование: цифровые входы и выходы	45
4.5 Управление временем и сон	48
4.6 Важные значения: переменные и константы	50
4.7 Количество и типы переменных	50
4.8 Преобразование числовых типов	52
4.9 Маленькие большие данные: массивы	52
4.10 Операторы	53
4.11 С форматом, пожалуйста: привлекательный текст и вывод данных	55
4.12 Символы в цепочках: строки	58
Глава 5 • Контроллер в практическом использовании	60
5.1 LED мигалка как имитатор системы охранной сигнализации	60

5.2 Полезно в экстренной ситуации: автоматический сигнал SOS.	61
Глава 6 • Структуры программ	63
6.1 Условия и циклы	63
6.2 Ходовые огни и освещение аэропорта	64
6.3 Электронная радуга: используется RGB-светодиод	66
6.4 SOS в компактном стиле	67
6.5 Метод проб и ошибок: попробуйте и за исключением	68
Глава 7 • Генерация аналоговых сигналов	70
7.1. Широтно-импульсная модуляция	70
7.2 Для романтических вечеров: симулятор сердцебиения	73
7.3 Световой будильник для расслабленного пробуждения	74
7.4 Mood-Light с многоцветным светодиодом	75
7.5 Чистота и плавность: аналоговые значения от ЦАП.	76
7.6 Вывод зависимых от времени напряжений.	77
7.7 Для интересных кривых: Генератор произвольных функций	78
Глава 8 • Прерывания и таймеры.	81
8.1 Требуется прерывание: прерывания	81
8.2 Автоматический ночной свет.	82
8.3 Мастера времени: Таймеры.	84
8.4 Многофункциональный проблесковый маячок	86
Глава 9 • Использование датчиков	90
9.1 Сбор данных измерений и датчиков	90
9.2 Точная регистрация напряжения: самодельный вольтметр	92
9.3 Коррекция линейности.	95
9.4 Линеаризация путем ограничения диапазона значений	96
9.5 Линеаризация входа АЦП с помощью компенсационного полинома	97
9.6 Измерение напряжения	98
9.7 Перекрестные помехи: побочные эффекты в сенсорной технике.	101
9.8 Прикосновение разрешено: емкостные датчики касания	102
9.9 Охлажденный или перегретый: датчики температуры обеспечивают четкость .	105
9.10 Цифровая запись температуры для безошибочной передачи данных	108
9.11 9.11 Однопроводной датчик DS18×20	108

9.12 Мощность данных: мультисенсорная матрица с термодатчиком DS18x20	110
9.13 В анфас: оптические датчики	112
9.14 Для кино- и фотопрофессионалов: электронный люксметр	113
9.15 Электронные летучие мыши: измерение расстояния с помощью ультразвука .	115
9.16 Больше никаких вмятин и царапин: датчик расстояния для гаражей	119
9.17 Оптимальный микроклимат в помещении для флоры и фауны	121
9.18 "Верь мне...": сравнение датчиков.	125
9.19 Измерение атмосферного давления и высоты.	126
9.20 Обнаружение магнитных полей датчиком Холла.	129
9.21 Датчики тревоги контролируют дверь и ворота.	130
Глава 10 • Технология отображения и экраны малого размера.	132
10.1 Графическое представление	135
10.2 OLED-дисплей в качестве графопостроителя	138
10.3 Пожалуйста, укажите точное время: цифровые часы с OLED-дисплеем	140
10.4 Не только для спортсменов: секундомер	144
10.5 Просто коснитесь: секундомер с сенсорными клавишами	145
10.6 Отличный климат с датчиком BME280!	148
Глава 11 • Светодиодные матрицы и большие дисплеи	150
11.1 Светодиодная матрица в действии	152
11.2 Запуск скриптов и анимационной графики.	153
Глава 12 • Физические вычисления: сервоприводы вносят движение в игру.	155
12.1 Сервотестер	155
12.2 Сервотермометр с мегадисплеем	158
Глава 13 • RFID и беспроводная передача данных	161
13.1 Чтение карт и чипов	162
13.2 Бесконтактный и безопасный: замок RFID	164
Глава 14 • MicroPython и Интернет вещей (IoT)	167
14.1 Для современных детективов: сетевой сканер	168
14.2 Подключено, но нет кабелей: WLAN	169
14.3 Переключение и управление с помощью веб-сервера	172
14.4 Веб-сервер WLAN в действии	176
14.5 Считывание данных датчика через WLAN.	177

4.6 Запись параметров окружающей среды: Термометр/гигрометр WLAN	179
Глава 15 • Просто и хорошо: протокол MQTT	183
15.1 MQTT через ThingSpeak	185
Глава 16 • Отправка данных в Интернет через ThingSpeak	190
16.1 Дождь или гроза? Виртуальная метеостанция доступна по всему миру	190
16.2 Графическое представление данных в ThingSpeak	194
16.3 Данные для смартфона с приложением ThingView.	195
16.4 От нежелательных посетителей: Оптическое наблюдение за помещением . . .	196
Глава 17 • Методы микромощности и спящие режимы	199
17.1 Энергосбережение защищает окружающую среду	199
17.2 Отключение ненужных потребителей	200
17.3 Метеостанция с батарейным или солнечным питанием	201
Глава 18 • Шинные системы для эффективной связи	202
18.1 Основы и применение шины I ² C	202
18.2 Шина SPI	207
18.3 Члены семейства SPI	210
18.4 Управление картами SD и μ SD через SPI	210
Глава 19 • Создание схем с помощью компонентов и макетов	212
19.1 Макетные платы.	213
19.2 Перемычки и соединительные кабели	214
19.3 Резисторы	215
19.4 Светодиоды (LED).	216
19.5 Конденсаторы и электролитические конденсаторы.	217

Глава 1 Введение

Внедрение чипа ESP32 от Espressif Systems знаменует собой новое поколение микроконтроллеров (МК), которые предлагают отличную производительность, функциональность Wi-Fi и Bluetooth по непревзойденной цене. Эти функции покорили производителей. Самые разнообразные приложения и проекты в области Интернета вещей (IoT) и домашней автоматике могут быть легко и экономично реализованы. ESP32 так же легко программируется, как и классические платы Arduino. Однако для сравнения ESP32 также предлагает, среди прочего:

- Большая флэш-память и память SRAM
- Значительно более высокая скорость процессора
- Встроенный Wi-Fi/WLAN
- Функциональность Bluetooth
- Больше пинов GPIO
- Широкая функциональность интерфейса
- Аналого-цифровой преобразователь с более высоким разрешением
- Цифро-аналоговый преобразователь
- Функции безопасности и шифрования

По этим причинам его можно считать наиболее многообещающим преемником Arduino. В этом контексте часто используется термин «убийца Arduino».

Эта книга знакомит с программированием современных однокристальных систем (Systems on Chip — SoC). Помимо технической подготовки, основное внимание уделяется языку программирования Python, особенно его варианту «MicroPython». Основные взаимосвязи между электроникой и электротехникой будут рассматриваться только в той мере, в какой это необходимо для проектирования схем и экспериментов.

В любом случае, «аппаратное обеспечение» для начала может быть очень простым. Во-первых, требуется только плата контроллера и несколько светодиодов, а также резисторы. ПК или ноутбук, необходимый для программирования чипа, должен быть в каждом доме. Соответствующую среду программирования можно бесплатно загрузить из Интернета. Однако при работе со средой программирования MicroPython быстро возникают первые проблемы.

В последние годы Python пережил огромный подъем. Различные одноплатные системы, такие как Raspberry Pi, особенно способствовали его популярности. Но Python также нашел широкое применение в других областях, таких как искусственный интеллект или машинное обучение. Следовательно, использование Python или варианта MicroPython для приложений в SoC также является логичным выбором.

Однако мы не ограничимся простым введением в язык программирования. Во многих случаях полученные навыки программирования применяются на практике вместе с электронной схемой. Все полностью описанные проекты подходят для использования в лабораториях или в повседневной жизни. Таким образом, в дополнение к образовательному эффекту удовольствие от сборки полных и полезных устройств также находится на переднем плане. С помощью лабораторных съемных плат можно без особых усилий реализовать схемы всех типов. Таким образом, тестирование и испытание приложений становится образовательным удовольствием.

Благодаря различным приложениям, таким как метеостанции, цифровые вольтметры и функциональные генераторы, представленные проекты также идеально подходят для стажировок или учебных курсов по естественным наукам или на уроках науки и техники.

1.1 Python, C или Arduino?

Для начинающих среда программирования Arduino — одно из самых простых мест для программирования ESP32. За этим интерфейсом стоит версия C для Arduino, а также C++. Эти два языка программирования годами были популярны для разработки встраиваемых систем. Версия C для Arduino еще больше упростила начало работы. Кроме того, для этой цели создано одно из крупнейших технологических сообществ в мире. С появлением новых библиотек, программных исправлений и поддержки плат проблемы обычно решались быстро. Однако ограничение, заключающееся в том, что Arduino-C работает только в предназначенной для него среде, немаловажно. Специально для разработки более масштабных проектов отсутствуют полезные и важные функции. Таким образом, Arduino-C оставался в основном ограниченным хобби и проектами для начинающих.

MicroPython относительно новый. Сообщество пользователей растет, и поддерживается все больше и больше платформ. MicroPython — это упрощенная версия Python, одного из самых популярных языков программирования в мире. Поэтому конкретные проблемы можно решать не только в сообществах MicroPython. На самом деле общие форумы по Python все больше способствуют решению проблем MicroPython.

В дополнение к поддержке сообщества, MicroPython также обладает некоторыми особенностями, которые ставят его намного выше класса Arduino. Одной из таких функций является так называемая функция REPL. REPL расшифровывается как «цикл чтения-оценки-печати». Это позволяет быстро выполнять программы и разделы кода. Компилировать или загружать не нужно. Таким образом, части кода могут быть протестированы быстро и эффективно во время разработки.

MicroPython содержит очень компактную реализацию интерпретатора Python. Для этого требуется всего 256 КБ флэш-памяти и 16 КБ ОЗУ. Тем не менее интерпретатор разработан для максимальной совместимости со стандартным Python. Синтаксис и языковой диапазон в значительной степени соответствуют Python версии 3.4, поэтому опытные программисты на Python должны быть в состоянии сразу найти обходной путь. Кроме того, определено несколько языковых элементов, выходящих за рамки стандарта, что снижает требования к памяти и увеличивает скорость выполнения.

1.2 Требования

Для успешной работы с этой книгой необходимо выполнить следующие требования:

- Базовые знания любого языка программирования, такого как C, Java и т.п.;
- Предполагаются базовые знания в области электроники, особенно в области «ток — напряжение — сопротивление».

Для получения специальных знаний в области электроники, пожалуйста, обратитесь к обширной технической литературе, особенно от Elektor, через их книги, журналы и комплекты. Аппаратная структура была намеренно сделана простой, поскольку основное внимание должно быть уделено программированию с помощью MicroPython. Тем не менее, требуются различные компоненты и детали. Пояснения можно найти в отдельных главах, в которых компоненты используются в первую очередь. Кроме того, в последних разделах книги объясняются некоторые основные компоненты, такие как резисторы или светоизлучающие диоды. Вы можете обратиться к ним, если есть какие-либо неясности относительно отдельных компонентов.

Дополнительные требования

- ПК или ноутбук с USB,
- ОС Windows 10,
- доступ в Интернет.

Также полезно использовать активный USB-концентратор между компьютером и контроллером, т.к. гарантирует определенную защиту ПК. Хаб должен иметь собственное питание 5В через отдельный блок питания. Тогда ПК или ноутбук лучше всего защищен от короткого замыкания за хабом, так как очень маловероятно, что короткое замыкание «пронесет» активный хаб в USB-порт компьютера.

Глава 2 • Разнообразие плат ESP

ESP32 — это современный и чрезвычайно мощный микроконтроллер. Помимо высокой тактовой частоты и обширных внутренних функциональных блоков, чип имеет встроенные Wi-Fi и Bluetooth. Контроллер был разработан китайской компанией Espressif Systems, базирующейся в Шанхае, и пользуется все большей популярностью. Характеристики производительности ESP намного превосходят известные платы Arduino с точки зрения цены и производительности.

Поскольку ESP32 доступен только в виде SMD-чипа, для использования контроллера в непрофессиональной среде требуется так называемая плата прорыва (BoB) или плата разработки. На рынке доступно множество различных версий. Самые известные версии:

Board	Pins	Buttons	LiPo Charger
ESP32-PICO-KIT	34	EN and BOOT	no
JOY-IT NodeMCU	30	EN and BOOT	no
ESP32 DEV KIT DOIT	30/36	EN and BOOT	no
Adafruit Huzzah32	28	RESET	yes
ESP32 Thing	40	EN and BOOT	yes
LOLIN32	40	EN and BOOT	no
Node-ESP-Board	38	EN and BOOT	no

На следующем изображении показаны только три разные версии:

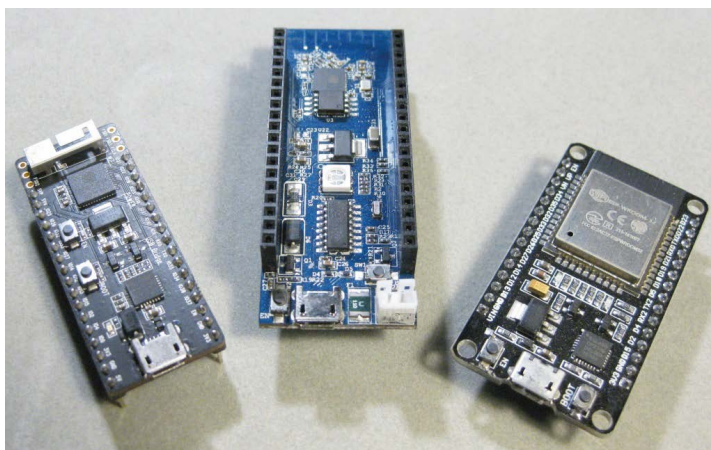


Рис.2.1: Платы ESP32 (PICO-KIT, Node-ESP и NodeMCU).

Часто помимо контроллера на платы монтируются другие компоненты, такие как кнопки, зарядное устройство для литий-ионных аккумуляторов или различные светодиоды.

Это означает, что первоначальные тесты и эксперименты можно проводить без внешней схемы.

В следующем разделе собраны наиболее важные данные о ESP32. Обзор должен создавать только первое впечатление. Более глубокое понимание отдельных характеристик и функций будет представлено в соответствующих разделах книги.

Процессор:	двухъядерный микропроцессор Tensilica LX6 160/240 МГц
Память:	520 КБ SRAM / 16 МБ флэш-памяти
Источник питания:	от 2,2 В до 3,6 В
Потребляемый ток:	Стандарт: прибл. 50 - 70 мА Режим Wi-Fi: прибл. 80 - 170 мА Режим глубокого сна: прибл. 2,5 мкА
Темпер. окр. среды	-40 °C to +125 °C
Входы/выходы (GPIO):	32 порта общего назначения с ШИМ Логика функций и таймера
Wi-Fi:	802.11 b/g/n/e/i
Пропуск. способ. сети:	135 Мбит/с (по протоколу UDP)
Чувствит. приемника:	-98 dBm
Bluetooth:	V4.2 BR/EDR и BLE
Функционал Bluetooth:	Classic и Bluetooth Low Energy (со встроенной антенной)
Датчики:	Датчик Холла 10-кратный емкостный сенсорный датчик
Интерфейсы:	Интерфейсы: 3x UART с потоком 3x интерфейса SPI Контроллер CAN-шины 2.0 2 интерфейса I2S и 2xI2C 18 аналоговых входов с 12-битными АЦП 2 аналоговых выхода с 10-битными ЦАП Инфракрасный (ИК) (TX/RX) ШИМ мотора LED-PWM до 16 каналов Интерфейс для внешней флэш-памяти SPI до 16 МБ Аппаратное обеспечение SD-карты
Безопасность:	Wi-Fi: WPA, WPA/WPA2 и безопасная загрузка WAPI Флэш-шифрование Криптографическое аппаратное ускорение Криптография на эллиптических кривых Генератор случайных чисел (RNG)

ESP32-PICO-KIT обычно используется для примеров приложений в этой книге. В качестве альтернативы можно использовать ESP32 DEV KIT или другую плату. Используемый вариант указывается явно в каждом случае. В принципе, однако, различные платы в значительной степени совместимы. Отличаются они в основном размерами и порядком расположения пинов. Рисунок 2.2. показан PICO-KIT с его функциональными узлами и соединениями.

Рис. 2.2: Плата ESP32 PICO-KIT.

Платы лучше всего подходят для экспериментов и разработки. Электронные компоненты, такие как светодиоды, датчики температуры или даже исполнительные механизмы, такие как сервоприводы модели R/C, могут быть подключены через разъемы портов. Плата Pico Kit имеет, среди прочего, следующие функции:

- Контроллер ESP с двумя 32-битными ядрами
- Быстрый интерфейс Wi-Fi и WLAN (до 150 Мбит/с)
- Функции АЦП и ЦАП
- Сенсорный блок
- Хост-контроллер для SD/SDIO/MMC
- Контроллер SDIO/SPI
- Блок EMAC и PWM для управления светодиодами и двигателями
- Интерфейсы UART, SPI, I²C и I²S
- Инфракрасный пульт дистанционного управления
- Интерфейс GPIO
- Bluetooth/Bluetooth LE (4.2)
- USB-to-Serial-Chip для доступа через интерфейс USB

2.1 Ввод в эксплуатацию и функциональная проверка

Как только плата будет доступна, ее следует подвергнуть первоначальному функциональному тестированию. Для этого USB-кабель подключается к ПК или ноутбуку и разъему micro-USB на плате.

На большинстве плат светодиод загорается при подключении к USB-порту с питанием. Если, вопреки ожиданиям, этот так называемый светодиод включения питания не загорается, следует немедленно отключить соединение USB. Таким образом, вы можете предотвратить серьезное повреждение из-за возможного короткого замыкания. Полезные советы по дальнейшему устранению неполадок даны в соответствующей главе в конце книги.

Платы ESP всегда должны эксплуатироваться на беспаячной плате (см. рис. 2.3).

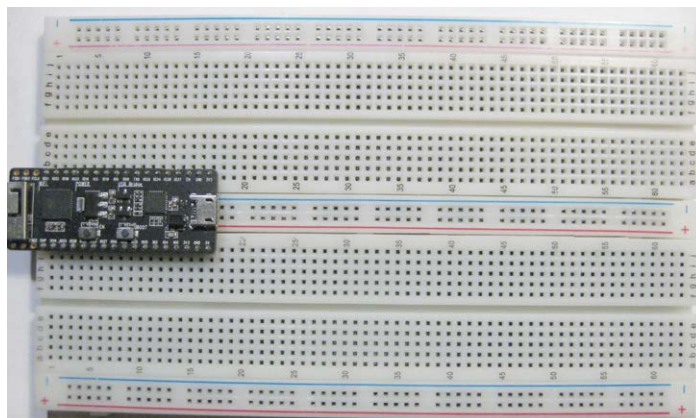


Рис.2.3: Плата ESP, подключенная к макетной плате.

2.2 ESP32 с питанием от батареи

В большинстве случаев плата ESP питается через разъем Micro-USB. Поскольку при создании программы контроллер все равно подключается к ПК или ноутбуку, дополнительное питание не требуется.

Если прямой обмен данными с ПК больше не нужен, плата также может питаться от источника питания USB, которая должна обеспечивать выходной ток не менее 1000 мА (1 А; 1 ампер), чтобы избежать нежелательных падений напряжения. Кроме того, имеется определенный запас мощности для работы некоторых светодиодов, дисплеев или датчиков.

Даже без подключения USB плата может отправлять и получать данные через Wi-Fi и Bluetooth. Для достижения полной независимости модуль должен питаться только от (перезаряжаемых) батарей.

Некоторые платы имеют для этой цели разъем для литий-ионной (Li-Ion) батареи (см. рис. 2.4). Подходящие элементы можно подключать напрямую через стандартный штекер. Затем внутренняя регулировка напряжения обеспечивает оптимальное питание контроллеров. Кроме того, подключенная батарея заряжается, как только плата подключается к действующему USB-разъему.

Для этого приложения подходят аккумуляторы емкостью около 1500 мАч и более. Аккумуляторы меньшего размера менее 300 мАч не следует использовать, так как встроенный контроллер заряда может привести к их перезарядке.

При типичном энергопотреблении ок. 50 мА, вариант на 1500 мАч может обеспечить время работы около 30 часов, то есть чуть более суток. При использовании функций сна контроллера может быть достигнуто даже значительно более длительное время работы.

Одноэлементные LiPo (литий-полимерные) или литий-ионные батареи обеспечивают достаточную мощность для ESP32. Однако их напряжение от 3,7 до 4,2 В, в зависимости от уровня заряда, слишком велико для ESP32. Поэтому оно регулируется с помощью внутреннего модуля.

Поскольку работа с литий-ионными аккумуляторами всегда сопряжена с определенной степенью опасности, нельзя упускать следующую информацию:

- Литий-ионные аккумуляторы чутко реагируют на неправильные зарядные токи или напряжения. При определенных обстоятельствах существует даже риск возгорания или взрыва.
- Каждый пользователь несет ответственность за собственные проекты.
- Ни Издатель, ни Автор не несут никакой ответственности.

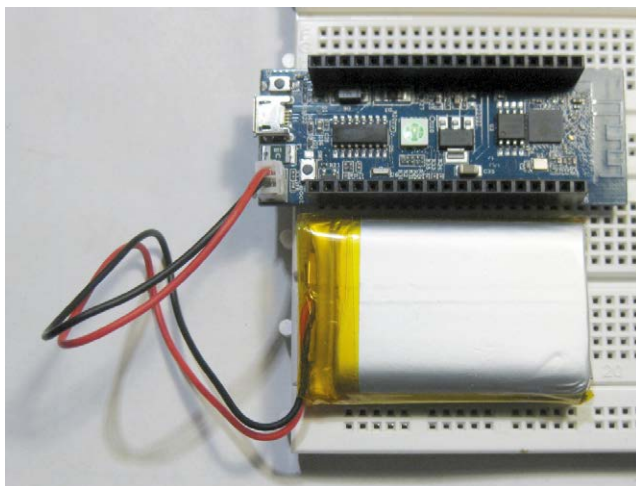


Рис.2.4: Плата *NodeESP* в режиме работы от батареи.

Глава 3 • Среды программирования и разработки

В отличие от ситуации, скажем, с системой Arduino, для работы с MicroPython доступно несколько интегрированных сред разработки (IDE). В принципе, со всеми IDE можно писать программы и загружать их на контроллер. Двумя наиболее распространенными средами программирования в настоящее время являются:

- µPyCraft
- Thonny

Оба имеют свои специфические преимущества и недостатки. Различия заключаются главным образом в различных процедурах разработки и управления программным кодом для проектов приложений.

Первый вариант под названием µPyCraft предлагает сравнительно простой интерфейс для разработки MicroPython на контроллере ESP32. Он работает с простыми графическими элементами и напоминает текстовые операционные системы. Работа с отдельными функциями проста для понимания, а работе с различными меню легко научиться.

Thonny, с другой стороны, имеет полностью графический интерфейс в стиле Windows. IDE очень популярна среди производителей, особенно потому, что она доступна в операционной системе Raspbian на Raspberry Pi. Поэтому многие пользователи Raspberry Pi уже хорошо знакомы с Тонни.

IDE обозначают наиболее важные операционные системы, такие как

- Windows PC
- Mac OS X
- Linux Ubuntu

Если при установке или использовании одной из систем возникают проблемы, можно использовать другую версию в качестве альтернативной системы программирования. Выбор версии зависит, конечно, от личных склонностей и привычек пользователя.

3.1 Установка IDE µPyCraft

Перед установкой µPyCraft IDE на используемом вами компьютере должна быть установлена последняя версия Python 3.7.X. Если его нет, то установку можно произвести по следующей инструкции:

1. Загрузите установочный файл со страницы загрузки Python по адресу:

www.python.org/downloads

2. После операции загрузки файл с именем python-3.7.X.exe должен находиться на вашем компьютере. Двойной щелчок по файлу запускает установку.
3. Выберите «Add Python 3.7 to PATH» и нажмите кнопку «Install Now».
4. Процесс установки завершается через несколько секунд, и отображается сообщение «Setup was successful». После этого окно можно закрыть.

Теперь uPyCraft IDE для Windows можно скачать с

<https://github.com/DFRobot/uPyCraft>

в виде файла с именем uPyCraft_V1.x.exe. После нажатия на этот файл .exe откроется uPyCraft-IDE:

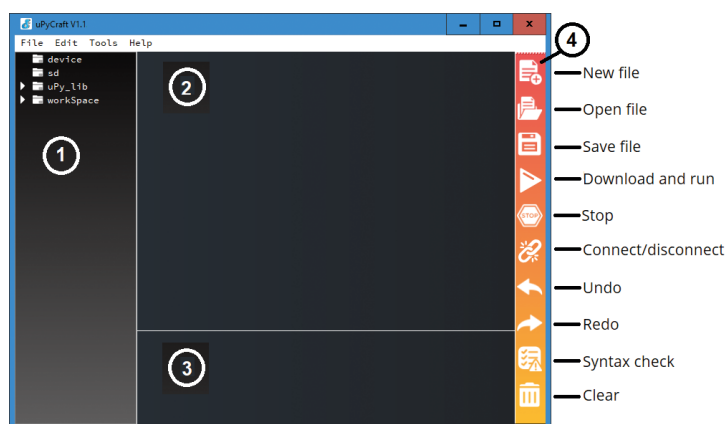


Рис.3.1: uPyCraft-IDE

После установки IDE на компьютер можно загрузить микропрограмму ESP32 на чип. Текущую версию микропрограммы MicroPython для ESP32 можно найти по адресу

<http://micropython.org/download#esp32>

Там вы прокручиваете до раздела «Модули ESP32». После перехода по ссылке «Generic ESP32module» вы попадете на страницу загрузки файла ESP32-BIN. Это будет выглядеть следующим образом:

esp32-idf3-20191220-v1.12.bin

Теперь вы можете запустить uPyCraft-IDE. Под

Tools -> Serial

выберите порт ESP32-COM, здесь COM5:

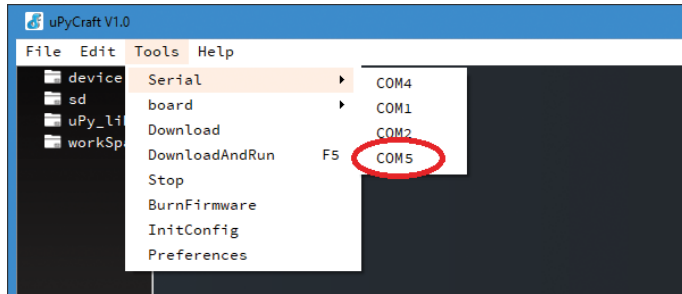


Рис.3.2: Выбор порта.

Если плата ESP32 подключена к компьютеру, но порт ESP32 не отображается в uPyCraft IDE, возможно, отсутствует соответствующий драйвер USB. В этом случае драйвер необходимо переустановить. Соответствующий драйвер можно найти в разделе

<https://www.silabs.com/products/development-tools/software/usb-to-uart-bridge-vcp-drivers>

После этого вы можете следовать

Tools -> Board

Далее необходимо выбрать опцию «esp32»:

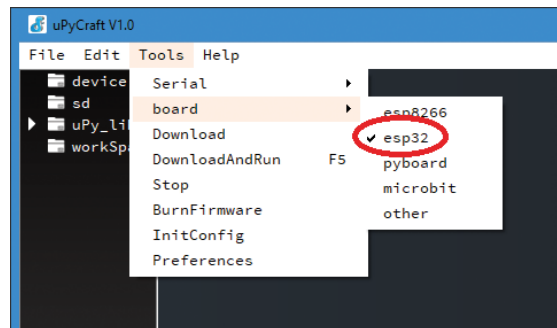


Рис.3.3: Выбор типа платы

Теперь интерпретатор MicroPython можно записать в ESP32 с помощью

Extras -> Burn Firmware

Подходящие варианты:

- плата: esp32
- burn_addr: 0x1000
- erase_flash: да
- com: COMX (here COM5, see above)

В разделе «USERS» выберите загруженный файл ESP32-BIN, как показано на рис. 3.4.

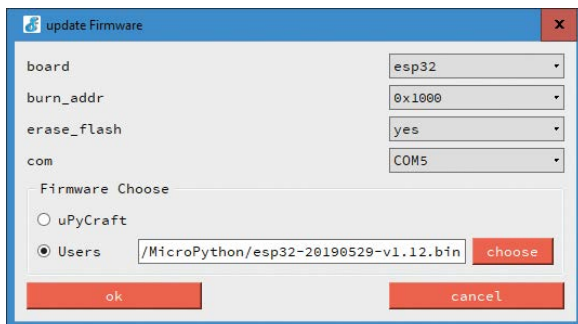


Рис.3.4: Параметры прошивки

Если все настройки выбраны правильно, то на некоторых вариантах платы необходимо нажать кнопку "BOOT/FLASH" на плате ESP32. Как только начнется процесс «EraseFlash», ключ можно отпустить. Через несколько секунд прошивка должна быть загружена на плату ESP32. Однако во многих случаях загрузка начнется без нажатия кнопок.

Если экран «EraseFlash» не запускается или отображается сообщение об ошибке, повторите шаги, описанные выше. Также снова нажмите клавишу «BOOT/FLASH», чтобы ESP32 перешел в режим прошивки.

3.2 MicroPython для ESP32

За некоторыми исключениями, все возможности и функции Python также доступны в MicroPython. Самое большое отличие состоит в том, что микро-версия была разработана для использования в однокристальных системах, поэтому классические подпрограммы, необходимые только для ПК, отсутствуют.

По этой причине MicroPython не содержит полной стандартной библиотеки, а только части, относящиеся к микроконтроллерам. Таким образом, доступны все модули, необходимые для доступа к используемому устройству. Таким образом, с соответствующими библиотеками вы можете легко получить доступ к пинам GPIO. Специально для ESP32 также доступны модули для поддержки сетевых подключений (Wi-Fi) и Bluetooth. В частности, поддерживаются следующие платы:

- ESP32
- ESP8266
- PyBoard
- Teensy 3.X
- WiPy - Pycom

Хотя еще не все функции контроллера ESP полностью доступны в MicroPython, библиотеки содержат наиболее важные команды и подпрограммы. Поэтому многие проекты и приложения могут быть реализованы без проблем. Кроме того, реализация недостающих функций идет быстрыми темпами, так что даже этот небольшой косметический недостаток будет быстро устранен.

После установки микропрограммы MicroPython на ESP32 вы также можете легко вернуться, например, в Arduino IDE. Для этого просто загрузите новый код с IDE на контроллер. Специальная процедура удаления не требуется. Тем не менее, если вы хотите снова использовать MicroPython после этого, микропрограмму MicroPython необходимо снова перепрограммировать.

3.3 "Hello World" для контроллера

В отличие от контроллеров AVR, таких как те, что используются в системе Arduino, ESP32 может поддерживать полную файловую систему. Первые поколения контроллеров программировались либо на ассемблере, либо на C. Поэтому программный код создавался и компилировался в среде разработки. Затем в контроллер передавался только готовый «машинный код». Таким образом, память целевой системы всегда содержала ровно одну программу.

Напротив, при программировании на MicroPython несколько программ могут храниться на чипе ESP32. Затем они могут быть обработаны непосредственно интерпретатором, который также доступен в системе. Файловой системой можно управлять напрямую с помощью uPyCraft-IDE. Поэтому рекомендуется более подробно ознакомиться с IDE, прежде чем загружать первую прикладную программу в ESP. Среда разработки содержит, как и многие другие инструменты программирования, следующие компоненты (см. также рис. 3.1):

1. Folders and files - Папки и файлы
2. Editor - редактор
3. MicroPython Shell / Terminal - Оболочка/терминал MicroPython
4. Tools - Инструменты

В левом подокне («Folders and files») файлы, хранящиеся в данный момент на плате ESP, видны в папке устройства («device»). Как только плата будет подключена к uPyCraft-IDE через последовательное соединение, все сохраненные файлы будут загружены при открытии папки устройства. Сразу после установки интерпретатора Python здесь виден только файл «boot.py». Для выполнения кода приложения также необходимо создать файл main.py. Вы можете создать файл main.py, используя:

file → new

Он создает новый файл ("untitled - без названия"). С помощью значка дискеты в окне «Tools» этот файл можно сохранить локально под именем «main.py» в микросхеме ESP.

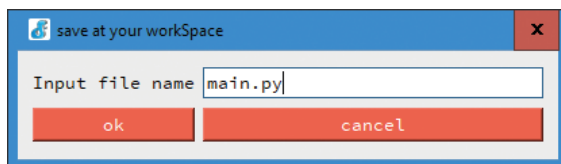


Рис.3.5: Создание нового файла «main.py».

Следующие два файла теперь находятся в папке устройства:

- boot.py: выполняется каждый раз при перезагрузке платы
- main.py: основной скрипт для кода приложения.

Папка SD следует за папкой устройства. Эта папка предназначена для доступа к файлам, хранящимся на SD-карте. Некоторые платы ESP-32 имеют слот для SD-карты. Если здесь вставлена карта microSD, файлы на карте появятся в папке «sd».

Папка uPy_lib следует ниже. Здесь показаны файлы встроенной библиотеки IDE. Здесь вы можете найти различные файлы сразу после установки интерпретатора MicroPython. Они предоставляются как стандартные библиотеки.

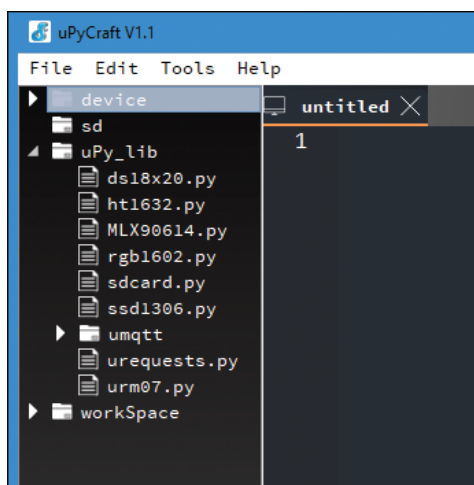


Рис.3.6: Стандартные библиотеки в папке uPy_lib.

Последняя папка содержит так называемый «workSpace». Это каталог для сохранения файлов приложений. Отображаемые здесь файлы хранятся на компьютере, подключенном через интерфейс. Там должны храниться все активные файлы.

Поэтому при первом использовании uPyCraft рекомендуется создать подходящий рабочий каталог с именем «workSpace», а затем последовательно использовать его для работы с контроллером.

В области редактора (2) создается код для прикладных программ .py. Редактор открывает новую вкладку для каждого файла.

Раздел под областью редактора — это «MicroPython Shell/Terminal». (3) Все введенные здесь команды немедленно выполняются платой ESP. Кроме того, терминал также отображает информацию о состоянии запущенной программы. Здесь отображаются любые синтаксические ошибки в текущей программе или сообщения об ошибках при загрузке и т. д.

С помощью символов в области «Tools» в правой части главного окна (4) задачи можно выполнять быстро и напрямую. Кнопки имеют следующие функции:

- New File: создает новый файл в редакторе.
- Open File: открывает файл на компьютере.
- Save File: открывает файл на компьютере.
- Download and run: загрузите код на плату ESP и запустите его.
- Stop: Завершает выполнение кода. Это соответствует вводу CTRL + C .
- Подключить или отключить последовательный интерфейс.
Последовательный порт можно выбрать в меню Tools -> Serial.
- Undo: отменяет последнее изменение в редакторе кода.
- Redo: повторить последнее изменение в редакторе кода.
- Syntax check: проверяет синтаксис текущего кода.
- Clear: удаляет сообщения оболочки / окна терминала.

Чтобы ознакомиться с написанием программы и выполнением кода на ESP32, ниже будет разработана и выполнена короткая программа на Python, которая заставит мигать светодиод. Первым шагом является установление связи с платой ESP:

1. Выберите текущую плату через Tools -> Board
2. В Tools -> Port выберите COM-порт, к которому подключена ESP-Board.
3. Затем кнопка Connect устанавливает последовательную связь с модулем ESP32.
4. После успешного подключения к плате в окне оболочки отображается ">>>".

Теперь можно ввести команду печати, чтобы проверить, правильно ли работает связь:

```
>>> print('test')
ответ
Test
>>>
```

появляется в окне терминала. Когда сообщение отображается, все в порядке. В противном случае убедитесь, что последовательная связь с платой установлена и микропрограмма MicroPython была успешно записана на плату.

Теперь можно создать скрипт мигания светодиода. Для этого необходимы следующие шаги:

1. В окно редактора созданного выше файла main.py вводится следующая программа:

```
from machine import Pin
from time import sleep
led = Pin(2, Pin.OUT)
while True:
    led.value(not led.value())
    sleep(1)
```
2. Нажав на кнопку «Stop», скрипт, который может быть еще запущен, может быть остановлен.
3. По нажатию на кнопку "Download And Run - Скачать и запустить" скрипт записывается в контроллер
4. В окне оболочки теперь должно отображаться сообщение «download ok».

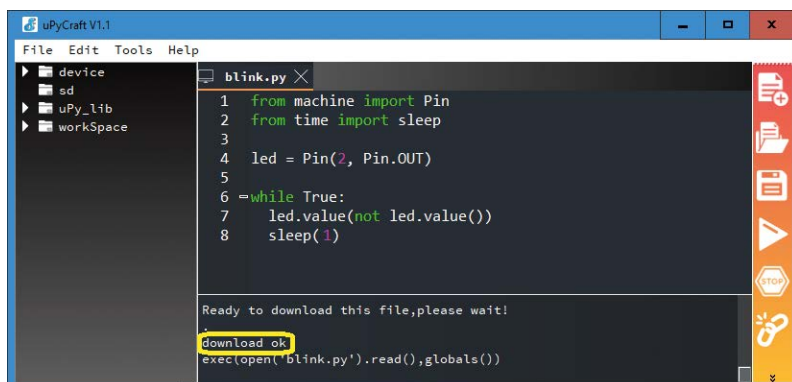


Рис.3.7: Успешная загрузка первой программы.

Теперь встроенный светодиод платы ESP32 должен мигать каждую секунду. Это означает, что первая программа Python была успешно передана в контроллер и немедленно выполнена.

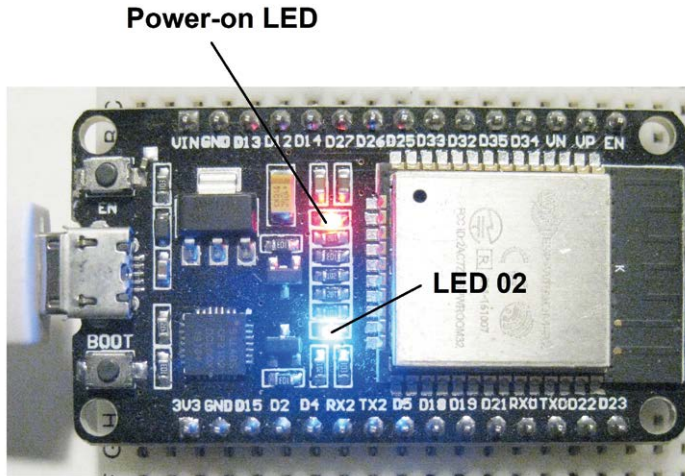


Рис. 3.8: Внутренний индикатор порта 02 в действии.

Некоторые платы не имеют встроенного светодиода. В этом случае к контроллеру должен быть подключен светодиод и добавочный резистор (см., например, рис. 4.6 и рис. 4.7). Дополнительные сведения и информацию о программе прошивки и подключении внешних светодиодов можно найти в Разделе 4.4.

3.4 Для профессионалов: Работа с esptool

Если возникают проблемы с µPyCraft-IDE или другие причины не позволяют вам использовать его, вы также можете загрузить прошивку MicroPython на контроллер с помощью утилиты под названием esp-tool.

Затем можно даже передавать программы на контроллер через терминал, такой как PUTTY или TeraTerm. Однако этот метод уже требует некоторых глубоких знаний интерфейса командной строки или диспетчера устройств. Поэтому эта процедура не рекомендуется для начинающих. Тем не менее, этот метод будет описан здесь, так как он также обеспечивает дальнейшее понимание обработки программирования контроллера. Кроме того, представленные здесь методы также могут быть легко перенесены в системы Linux или UNIX.

Для работы с esptool в системе должен быть установлен Python 3.7.X или более новая версия Python. Затем текущую версию esptool можно загрузить на компьютер через окно терминала с помощью `pip install esptool`.

```
pip install esptool
```

В некоторых установках Python эта инструкция может привести к появлению сообщения об ошибке. В этом случае следующие инструкции могут привести к месту назначения:

```
pip3 install esptool
python -m pip install esptool
pip2 install esptool
```

Теперь файл esptool должен быть установлен в стандартную директорию для исполняемых файлов. Для доступа к контроллеру необходимо знать его серийный номер порта. Это можно найти в диспетчере устройств:

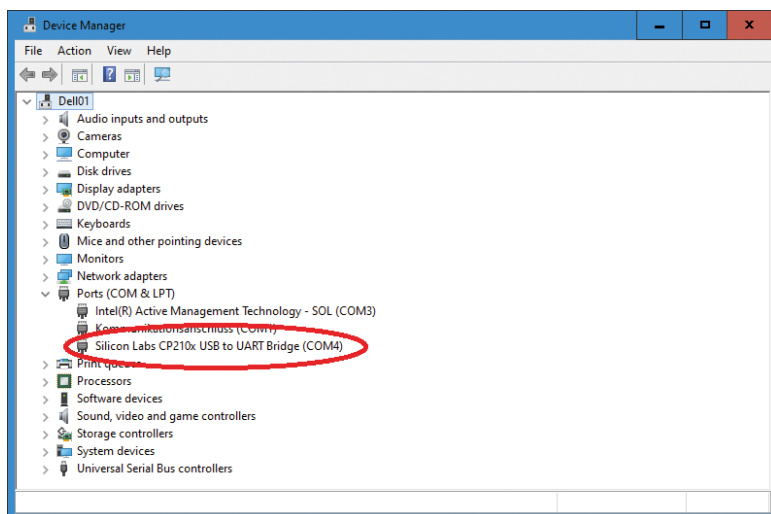


Рис.3.9: COM-порт ESP32 в диспетчере устройств.

Например, на рисунке номер порта COM3. Это позволяет выполнить следующую команду в окне терминала:

```
esptool --port COM3 flash_id
```

Она предоставляет информацию о системе ESP32, подключенной к этому порту:

Рис. 3.10: Информация о ESP32.

Строки

```
...  
Detecting chip type... ESP32  
Chip is ESP32-PICO-D4 (revision 1)  
Features: WiFi, BT, Dual Core, Embedded Flash
```

```
Кварц 40MHz
MAC: d8:a0:1d:40:54:14
...
Device: 4016
Detected flash size: 4MB
```

Таким образом, вы получите важную информацию о типе микросхемы, плате, доступных интерфейсах, частоте кварца и доступной флэш-памяти.

Теперь вы можете воспользоваться инструкцией

```
esptool --port COM3 erase_flash
```

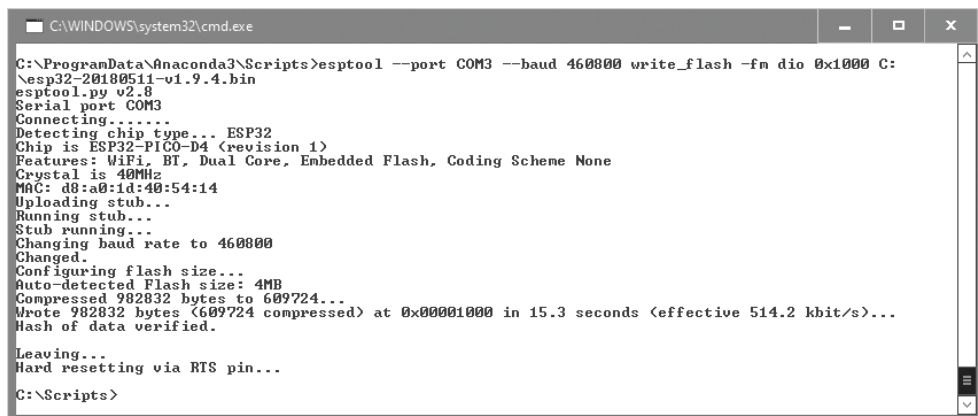
стереть флэш-память чипа.

Рис.3.11: Успешное стирание флэш-памяти.

Это освобождает путь для загрузки прошивки MicroPython. Загрузка прошивки из Интернета уже была описана в предыдущих главах. Команда для загрузки

```
esptool --port COM3 --baud 460800 write_flash -fm dio 0x1000
C:\Users\Documents\workSpace\esp32-xxxxxxx-vx.x.x.bin
```

Разумеется, здесь снова должен быть указан правильный путь, где находится файл esp32-20xxxxxx-vx.x.x.bin. Через несколько секунд загрузка должна быть завершена:



```
C:\WINDOWS\system32\cmd.exe
C:\ProgramData\Anaconda3\Scripts>esptool --port COM3 --baud 460800 write_flash -fn dio 0x1000 C:\
\esp32-20180511-v1.9.4.bin
esptool.py v2.8
Serial port COM3
Connecting.....
Detecting chip type... ESP32
Chip is ESP32-PICO-D4 (revision 1)
Features: WiFi, BT, Dual Core, Embedded Flash, Coding Scheme None
Crystal is 40MHz
MAC: d8:a0:1d:40:54:14
Uploading stub...
Running stub...
Stub running...
Changing baud rate to 460800
Changed.
Configuring flash size...
Auto-detected Flash size: 4MB
Compressed 982832 bytes to 609724...
Wrote 982832 bytes (609724 compressed) at 0x00001000 in 15.3 seconds (effective 514.2 kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
C:\Scripts>
```

Рис.3.12: Загрузка прошивки завершена.

Теперь вы можете связаться с ESP через PUTTY или TeraTerm. Скорость передачи должна быть установлена на 115200:

Рис.3.13: MicroPython отправляет отчеты Tera Terminal.

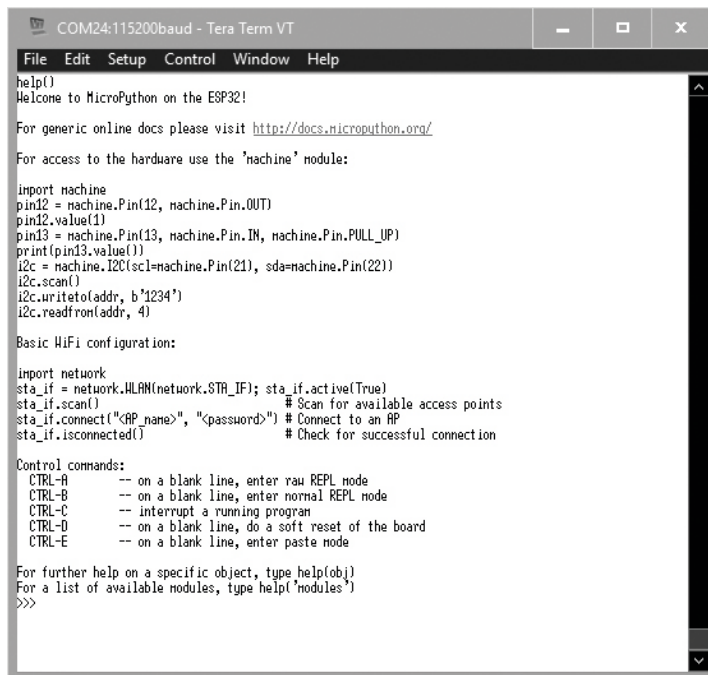
Первая программа Python уже может быть загружена через терминал, например:

```
from machine import Pin
import time
p23 = Pin(23, Pin.OUT)
while True:
    p23.value(1)
    time.sleep(1)
    p23.value(0)
    time.sleep(.1)
```

Рис.3.14: Программирование через TeraTerminal.

После передачи программы светодиод, подключенный к порту 23, должен мигать с частотой 1 секунду.

Консоль предлагает обычную справочную систему, которая при необходимости может предоставить полезную информацию:



```
COM24:115200baud - Tera Term VT
File Edit Setup Control Window Help
help()
Welcome to MicroPython on the ESP32!

For generic online docs please visit http://docs.micropython.org/

For access to the hardware use the 'machine' module:

import machine
pin12 = machine.Pin(12, machine.Pin.OUT)
pin12.value(1)
pin13 = machine.Pin(13, machine.Pin.IN, machine.Pin.PULL_UP)
print(pin13.value())
i2c = machine.I2C(scl=machine.Pin(21), sda=machine.Pin(22))
i2c.scan()
i2c.writeto(addr, b"1234")
i2c.readfrom(addr, 4)

Basic WiFi configuration:

import network
sta_if = network.WLAN(network.STA_IF); sta_if.active(True)
sta_if.scan() # Scan for available access points
sta_if.connect("<AP_name>", "<password>") # Connect to an AP
sta_if.isconnected() # Check for successful connection

Control commands:
CTRL-A -- on a blank line, enter raw REPL mode
CTRL-B -- on a blank line, enter normal REPL mode
CTRL-C -- interrupt a running program
CTRL-D -- on a blank line, do a soft reset of the board
CTRL-E -- on a blank line, enter paste mode

For further help on a specific object, type help(obj)
For a list of available modules, type help('modules')
>>>
```

Рис.3.15: Функция справки в терминале.

3.5 Thonny — Python-IDE для начинающих

Тонни — хорошая альтернатива uPyCraft. Хотя даже эта среда программирования еще не полностью свободна от ошибок, в целом с ней можно хорошо работать. Кроме того, система постоянно обновляется и совершенствуется, так что этот недостаток также будет устранен в обозримом будущем. Thonny также доступен для всех распространенных операционных систем. Под Raspbian для Raspberry Pi он даже установлен по умолчанию. Установка относительно проста, поэтому в процессе установки вряд ли возникнут проблемы. Обязательным условием для работы с Thonny является то, что прошивка MicroPython уже загружена на ESP32. Начинающим рекомендуется использовать uPyCraft-IDE. Эксперты также могут использовать esptool (см. последнюю главу), чтобы полностью обойти uPyCraft-IDE.

Примеры в этой книге были созданы с помощью Thonny 3.2. В принципе, будущие версии должны быть совместимы с ним. Однако в случае непредвиденных проблем следует вернуться к версии, упомянутой выше. Соответствующий пакет можно загрузить с

<https://thonny.org>

Когда загрузка будет завершена, вы можете запустить установочный файл. Теперь вам нужно только следовать за помощником, пока процесс установки не будет завершен. После этого можно открыть Thonny-IDE.

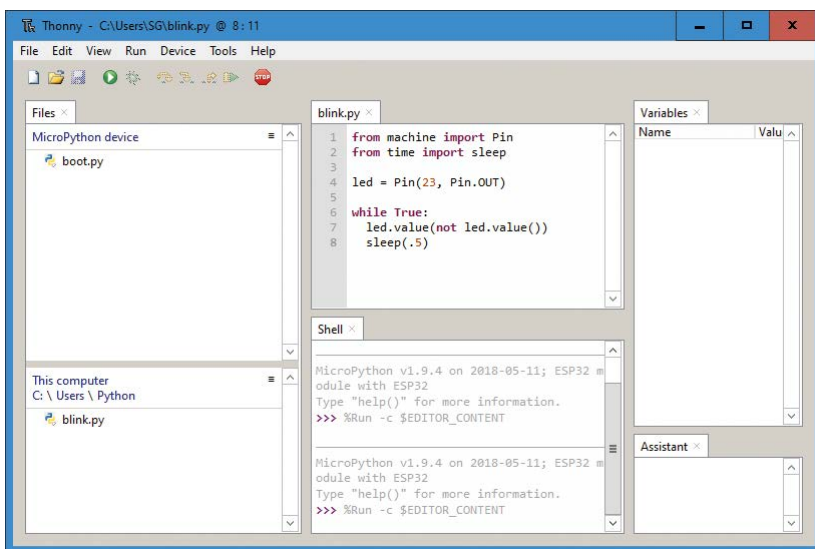


Рис.3.16: Thonny-IDE после запуска.

Теперь плату ESP32 можно подключить к компьютеру. Чтобы протестировать установку, Thonny должен быть настроен для интерпретатора MicroPython. Кроме того, должна быть выбрана используемая плата. Для этого необходимы следующие шаги:

1. Run → Interpreter открывает следующее окно:
- 2.

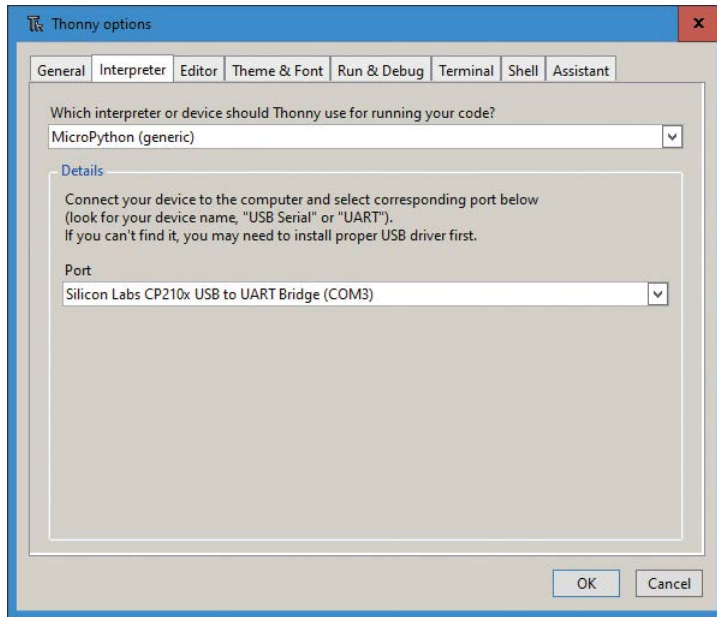


Рис.3.17: Окно параметров в Thonny.

2. В первом окне выбора выберите «MicroPython» (generic).
3. COM-интерфейс ESP32 должен быть указан в разделе Port.

Теперь Thonny-IDE должна быть подключена к плате, и в окне оболочки должно появиться приглашение ">>>". В качестве альтернативы можно выбрать опцию «Try automatic recognition - Попробовать автоматическое распознавание». Однако она работает не со всеми платами. Наконец, в оболочку вводится команда `help()`. Она вернет приветственное сообщение и некоторую информацию:

```
Welcome to MicroPython on the ESP32!
For generic online docs please visit http://docs.micropython.org/
For access to the hardware use the ,machine' module:
```

```
import machine
pin12 = machine.Pin(12, machine.Pin.OUT)
pin12.value(1)
pin13 = machine.Pin(13, machine.Pin.IN, machine.Pin.PULL_UP)
print(pin13.value())
i2c = machine.I2C(scl=machine.Pin(21), sda=machine.Pin(22))
i2c.scan()
i2c.writeto(addr, b'1234')
i2c.readfrom(addr, 4)
```

Basic WiFi configuration:

```
import network
```

```
sta_if = network.WLAN(network.STA_IF); sta_if.active(True)
sta_if.scan() # Поиск доступных точек доступа
sta_if.connect("<AP_name>", "<password>") # Подключиться к AP
sta_if.isconnected() # Проверяем успешное подключение
```

Команды управления:

```
CTRL-A    -- в пустой строке введите необработанный режим REPL
CTRL-B    -- в пустой строке войти в обычный режим REPL
CTRL-C    -- прервать запущенную программу
CTRL-D    -- на пустой строке сделать программный сброс платы
CTRL-E    -- на пустой строке войти в режим вставки
```

Для получения дополнительной справки по конкретному объекту введите `help(obj)`.
Чтобы получить список доступных модулей, введите `help('modules')`

Таким образом, установка и ввод в эксплуатацию Thonny успешно завершены. Аппаратные функции теперь можно активировать с помощью инструкций оболочки. Прежде всего, можно импортировать машинный модуль:

```
>>> из машины импортировать пин
```

Затем светодиод, подключенный к порту 23, можно включить через

```
>>> led = Pin(23, Pin.OUT).value(1)
```

В качестве альтернативы можно использовать стандартный светодиод, доступный на большинстве плат.

```
>>> led = Pin(23, Pin.OUT).value(0)
```

соответствующий светодиод снова гаснет.

3.6 Работа с Тонни

В Thonny-IDE есть несколько разных разделов, среди которых редактор и оболочка или терминал MicroPython:

- В области редактора создается и редактируется код. Можно открыть несколько файлов, для каждого файла будет доступна новая вкладка.
- В оболочке MicroPython вводятся команды, которые должны немедленно выполняться платой ESP. Терминал также предоставляет информацию о статусе выполняемой программы, указывает на ошибки, связанные с загрузкой, синтаксические ошибки, печать сообщений и т. д.

Также доступны другие полезные вкладки. Их можно настроить в меню View. Вкладка «Variables - Переменные», в частности, часто может быть использована с большим преимуществом. Она показывает все переменные программы и их текущие значения.

Для ознакомления с написанием программ и выполнением кода на ESP32 снова используется уже известный скрипт, который заставляет мигать встроенный светодиод платы ESP32 или внешний светодиод.

Сначала на плате создается файл `main.py`:

1. Когда Тонни запускается в первый раз, редактор показывает файл без заголовка. Этот файл сохраняется как `main.py`. Для этого файл сохраняется через

`file → save as`

переименован в `main.py` и сохранен на плате ("Micro Python Device").

2. Теперь доступна вкладка с именем «`main.py`».
3. Здесь вводится следующий код:

```
from machine import Pin
import time
p23 = Pin(23, Pin.OUT)
while True:
    p23.value(1)
    time.sleep(1)
    p23.value(0)
    time.sleep(.1)
```

Код доступен из папки с кодами, и его можно использовать первоначально путем копирования и вставки. Позже будет объяснено, как файлы могут быть скопированы непосредственно с ПК на контроллер.

С помощью зеленой стрелки или

`run → run current script`

или нажатием функциональной клавиши F5 код передается на контроллер. В оболочку выводится следующая информация:

```
MicroPython v1.9.4 on 2018-05-11; ESP32 module with ESP32
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT
```

При перезапуске ESP сначала выполняется `boot.py`, а затем `main.py`. Если `boot.py` отсутствует, он запускается сразу с `main`. Таким образом, программа должна стать активной сразу после загрузки, а светодиод на выбранном порту (№ 23 в примере выше) должен мигать.

Для некоторых вариантов платы также может потребоваться нажатие клавиши ESP EN/RESET.

3.7 Работа с файлами

Чтобы загрузить файл с уникальным именем в ESP с помощью Thonny IDE, необходимо выполнить следующие шаги:

- Создание нового файла
- Сохраните файл на компьютере под именем, например «blink.py».

Файл можно открыть как новую вкладку в подокне «Этот компьютер». Затем его можно открыть через меню

file → save as

на котором можно сохранить ESP под своим именем ("blink.py"). В запросе необходимо выбрать второй вариант "MicroPython device" (см. рисунок).

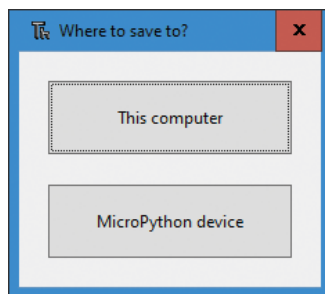


Рис.3.18: Запрос при сохранении

Теперь файл загружен на плату и появится в подокне «Files». Теперь его можно запустить с помощью зеленой клавиши со стрелкой или F5.

Соответственно, также возможно загрузить файлы с чипа на компьютер, выбрав «Этот компьютер». Другие команды для удаления или переименования файлов и т.д. также находятся в меню "file".

3.8 Советы по устранению неполадок с Thonny IDE

В текущем разделе будут обсуждаться некоторые сообщения об ошибках Thonny-IDE. Соответствующие проблемы обычно относительно легко решаются:

- Во многих случаях перезапуск ESP с помощью встроенного ключа EN/RST уже успешен.
- В Thonny-IDE проблемы со связью между ПК и чипом часто можно решить, нажав кнопку «Stop / Restart Backend" button - Стоп/Перезапустить серверную часть» (или CTRL-F2).

В противном случае могут помочь следующие советы:

Ошибка 1: Нет связи с платой.

В этом случае отображаются сообщения об ошибках:

```
===== ПЕРЕЗАПУСК =====
Unable to connect to COM4 - Невозможно подключиться к COM4
Error: could not open port 'COM4': FileNotFoundError(2, 'The system cannot
find the file specified.', None, 2) - Система не может найти указанный файл
```

или:

```
===== ПЕРЕЗАПУСК =====
Could not connect to REPL - Не удалось подключиться к REPL
Убедитесь, что ваше устройство имеет подходящую прошивку и не находится в
режиме загрузчика!
Disconnecting - Отключение.
```

или:

```
===== ПЕРЕЗАПУСК =====
Потеряно соединение с устройством (EOF).
```

В этом случае часто бывает полезно прервать USB-подключение к модулю, а затем восстановить его. Вы также должны проверить, установлен ли правильный последовательный порт в разделе

Run -> Select Interpreter

Эта ошибка также может указывать на то, что последовательный порт уже используется другой программой, например, последовательным терминалом или Arduino IDE. В этом случае убедитесь, что все программы, которые могут последовательно обмениваться данными с картой ESP, закрыты. Затем необходимо перезапустить Thonny IDE.

Error 2: Thonny IDE не отвечает или выдает внутреннюю ошибку.

После закрытия и повторного открытия активного окна вы сможете продолжить нормальную работу. При повторяющихся сбоях необходимо перезапустить всю Thonny-IDE.

Error 3: Thonny IDE больше не реагирует на клавишу «Stop / Restart Backend».

После нажатия кнопки «Stop / Restart Backend» следует подождать несколько секунд. ESP нужно время, чтобы перезапустить и восстановить последовательную связь с Тонни. Если кнопка «Стоп» нажата несколько раз или очень быстро одно за другим, у модуля ESP не будет достаточно времени для правильного перезапуска. Это может привести к сбою Thonny IDE.

Error 4: Проблема при перезапуске карты ESP, запуске нового скрипта или открытии последовательного порта.

Если появляется следующее сообщение об ошибке:

```
Brownout detector was triggered
```

или если есть продолжающиеся перезагрузки, или если информация:

```
ets Jun 8 2016 00:22:57

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
config: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0018,len:4
load:0x3fff001c,len:4732
load:0x40078000,len:7496
load:0x40080400,len:5512
```

это может указывать на аппаратную проблему. Часто это вызвано одной из следующих проблем:

- USB-кабель плохого качества
- USB-кабели слишком длинные
- Плата ESP имеет дефект, например, плохая пайка
- Неисправное USB-подключение к компьютеру
- USB-порт компьютера не обеспечивает достаточного питания.

В этих случаях поможет использование качественного и максимально короткого USB-кабеля. Также может помочь переход на другой USB-разъем на ПК. Для ноутбуков следует использовать активный USB-концентратор с собственным внешним блоком питания. Таким образом, вы больше не зависите от производительности USB-блока питания ноутбука.

Если проблемы сохраняются или появляются другие странные сообщения об ошибках, рекомендуется обновить плату ESP последней версией микропрограммы MicroPython. Это, по крайней мере, предотвратит уже исправленные ошибки, которые могут усложнить работу.

Error 5: нет связи между ПК и ESP32.

Может случиться так, что контроллер слишком загружен для установления USB-соединения. В этом случае нажатие кнопки «Stop/ Restart Backend» несколько раз может привести к желаемому успеху. Однако это повторное нажатие должно происходить через определенные промежутки времени (см. выше).

При запуске скрипта, использующего Wi-Fi, переключающегося в спящий режим или выполняющего несколько задач параллельно, рекомендуется попытаться установить связь три или четыре раза. Если это по-прежнему невозможно, следует повторно прошить ESP с помощью текущей прошивки MicroPython.

Глава 4 • Первые шаги в программировании

Python уже несколько лет является одним из наиболее часто используемых языков программирования. Одна из причин этого заключается в том, что он был очень просто спроектирован и поэтому легок в освоении. Разработка MicroPython делает программирование систем микроконтроллеров сравнительно простым и понятным. Это делает язык программирования также очень подходящим для новичков в мире встраиваемых систем.

Разработчики MicroPython поставили перед собой цель максимально упростить программирование цифровой электроники, таким образом, отхватив максимально возможный круг пользователей. Программы Python можно найти в сфере хобби, а также в образовательных или научных целях. Но и профессиональные разработчики все чаще работают с Python. В ИТ-индустрии многие лидеры рынка, такие как Google или Amazon, уже давно используют Python для разработки своего программного обеспечения.

Кроме того, бесплатные модули и библиотеки, такие как Matplotlib, NumPy, SciKit или SciPy, предоставляют широкие возможности. Они варьируются от научного анализа данных до машинного обучения и искусственного интеллекта.

MicroPython был разработан как облегченная версия Python3. Поскольку язык должен быть интерпретирован, он обычно медленнее, чем скомпилированные системы. MicroPython был разработан для максимально эффективной работы на небольших встроенных системах. Следовательно, его также можно запускать на микроконтроллерах, которые работают намного медленнее и имеют гораздо меньше памяти, чем обычные персональные компьютеры.

Недостатком классического программирования на Python является сложность реализации низкоуровневых элементов управления. По этой причине классические варианты Python используются скорее в качестве второстепенных в аппаратном программировании. Этот недостаток в значительной степени устранен в MicroPython. Основываясь на стандарте, версия Micro также сильно основана на Python 3 по своему синтаксису. Кроме того, существуют виртуальные машины и связанные с ними библиотеки.

Если сравнить два самых популярных языка программирования в среде микроконтроллеров, то окажется, что Python чаще предпочитают C/C++. В рейтингах самых популярных языков программирования Python все чаще занимает первое место. С другой стороны, конкурент C/C++ все чаще отводится на более низкие позиции. Причина такого развития в основном основана на следующих преимуществах. Python:

- Python очень удобен для начинающих благодаря простой языковой структуре.
- Различные интернет-форумы поддерживают программиста учебными пособиями и примерами кода.
- Имеются обширные библиотеки.

Новички обычно быстро находят решения своих проблем на форумах. В других языках эта форма взаимной поддержки выражена не так ярко.

В C программирование осуществляется с помощью управляющих регистров, указателей и других структур и инструкций, которые часто трудно понять. Прошивка для целевого контроллера должна быть запрограммирована, скомпилирована и, наконец, передана в контроллер с помощью программатора. MicroPython объединяет все эти шаги. Простым щелчком мыши пользователи могут управлять низкоуровневым оборудованием, таким как светодиоды, дисплеи или двигатели. Получение аналоговых значений напряжения или работа с SD-картами становится детской игрой с соответствующими библиотеками. Интегрированная очистка памяти и процесс динамического выделения обеспечивают эффективное управление памятью в Python. Это означает, что вам вряд ли придется прибегать к указателям или подобным конструкциям, которые новичкам обычно трудно понять.

Часто загадочные C-символы, такие как `x++`, `<<`, `>>` и т. д., а также сложное объявление переменных представляют собой препятствие для новичка. Python известен своей простотой и отличной читабельностью кода.

Поскольку MicroPython разрабатывался как «облегченная версия» для МК, поддерживаются не все библиотеки и функции стандартного Python. Тем не менее, вы можете легко переключиться на микроверсию, если вы уже знакомы с Python. Лишь несколько синтаксических структур или инструкций недоступны или не применимы в MicroPython.

Питон интерпретируется. Это означает, что исходный программный код обрабатывается непосредственно целевым процессором. Поэтому компиляция не требуется. Таким образом, Python предлагает возможность выполнять программу, когда-то написанную, в самых разных системах. Для этого необходимо установить только соответствующий интерпретатор. Одним из самых больших преимуществ кода Python является его полная совместимость. Программы Python могут выполняться на классических компьютерах под управлением Windows, MacOS или Linux, а также на небольших одноплатных системах, таких как Raspberry Pi или аналогичных микросистемах. В частности, использование на «RPi» (нем. «Raspi») также способствовало росту популярности Python.

С появлением новых мощных контроллеров, таких как ESP32, стало возможным эффективно и удобно использовать Python и в этой области. Прочно интегрированные проекты гарантируют, что программы могут быть легко разработаны как в малых, так и в больших масштабах. Таким образом, Python превосходно масштабируется. Возможная инкапсуляция данных и программного кода в четкие, удобные для использования модули, т. е. объекты, делает Python объектно-ориентированным языком программирования. C++ обычно используется в настоящее время, особенно в аппаратно-ориентированном программировании. Классические варианты Python до сих пор плохо подходили для этой цели. С MicroPython этот пробел закрыт.

C++ включает клиентские приложения, а также мощные серверные приложения, драйверы устройств и встроенные компоненты драйверов. Область применения варьируется от системного программного обеспечения до прикладного программирования. Поскольку Python — относительно новый язык программирования по сравнению с C, он еще не нашел универсального применения во всех областях информационных технологий. Однако видно, что Python набирает силу практически во всех областях.

Главным недостатком Python, безусловно, является его сравнительно низкая скорость обработки. Здесь компилируемые языки, такие как C, могут ясно показать свои преимущества. Быстрые циклы управления или системы реального времени, управление транспортными средствами и запросы безопасности могут быть реализованы намного проще и безопаснее на C.

Однако, поскольку эти области применения вряд ли играют роль для непрофессиональных пользователей, недостаток скорости едва ли значителен.

Python также приобрел особое значение в очень актуальной области искусственного интеллекта (ИИ). Благодаря обширным библиотекам, таким как NumPi, SciPi и т. д., и таким дистрибутивам, как Anaconda, Python на сегодняшний день стал самым популярным языком программирования. Поэтому все двери открыты для опытного пользователя Python. От программирования контроллера, связанного с аппаратным обеспечением, до приложений ИИ — с Python нет ограничений для интуиции и творчества.

В этой главе будут собраны основы MicroPython. У вас должна быть функциональная среда программирования, потому что команды и инструкции всегда иллюстрируются практическими примерами. Затем их можно сразу же протестировать непосредственно на целевом оборудовании, то есть на ESP32. Это не остается чисто теоретическим курсом программирования, а полученные знания можно сразу применять на практике.

4.1 Никогда без: Комментарии

Пояснительные комментарии важны в любом языке программирования. Это единственный способ узнать месяцы или годы спустя, что делает определенный раздел программы, без необходимости снова и снова вникать в старые детали.

Нет необходимости комментировать каждую строку программы по отдельности. Опытные программисты должны понимать отдельные инструкции без комментариев. Только в случае специальных конструкций или необычных или инновационных строк кода рекомендуется однострочный комментарий. С другой стороны, для подпрограмм или целых логических частей программы не должно отсутствовать краткое объяснение того, как они работают.

Простые комментарии будут начинаться со знака #. Они начинаются с # и заканчиваются концом строки:

```
>>> print("hello ESP32") # this is a comment
hello ESP32
```

Многострочные комментарии также могут быть помечены тройным двойным кавычком ("""). Затем та же строка символов завершает комментарий:

```
"""
первая строка комментария
вторая строка комментария
"""
```

На практике это может выглядеть так:

```
'''
This is a multi-line comment.
Prints "hello world".
'''
```

```
print("hello world")
```

Кроме того, функция комментирования может быть использована в Тонни. Это позволяет пометить несколько строк как комментарии одновременно знаком # (решётка).



Рис.4.1: Функция комментариев в Thonny.

Функция комментариев также очень удобна для комментирования определенных частей программы. Если, например, при тестировании более объемного кода некоторые разделы не будут выполняться в пробном режиме, их можно пометить знаком комментария. Линии тогда больше не наблюдаются интерпретатором. Это делает ненужным длительное удаление и последующую повторную вставку разделов программы.

Комментарии помогают новичкам лучше понять структуру программы. С технической точки зрения интерпретатор игнорирует все комментарии, т.е. они не влияют на ход программы.

4.2 Оператор Print()

Информация может быть выведена на терминал с помощью инструкции `print()`. Команду можно выполнить прямо в терминале:

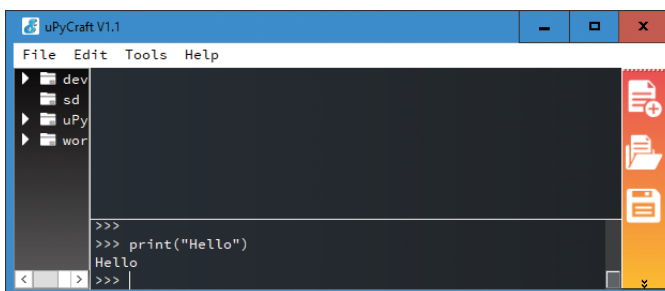


Рис.4.2: Команда печати в консоли.

С другой стороны, он используется в программах для вывода текстовой информации:

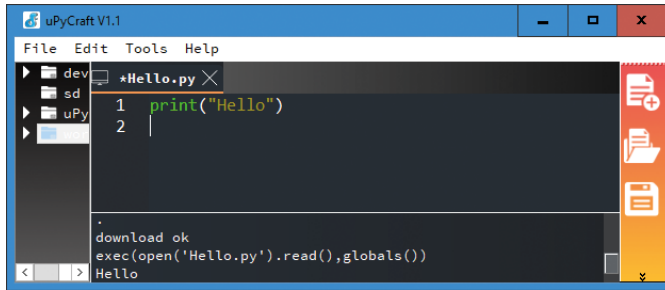


Рис.4.3: Команда печати как программная инструкция.

`print()` может содержать несколько строк, разделенных `","`:

```
>>> print("hello", "world!")
hello world!
```

Оператор `print()` выполняет разрыв строки по умолчанию. С

```
end = ""
```

это можно подавить:

```
print("hello", end=" ")
print("world")
```

и приводит к следующему результату:

```
hello world
```

Следующие две иллюстрации снова иллюстрируют версии инструкций печати в Thonny-IDE:

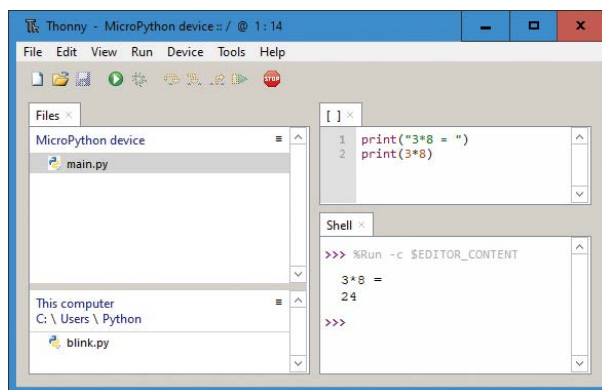


Рис.4.4: Команда печати как программная инструкция в Thonny.

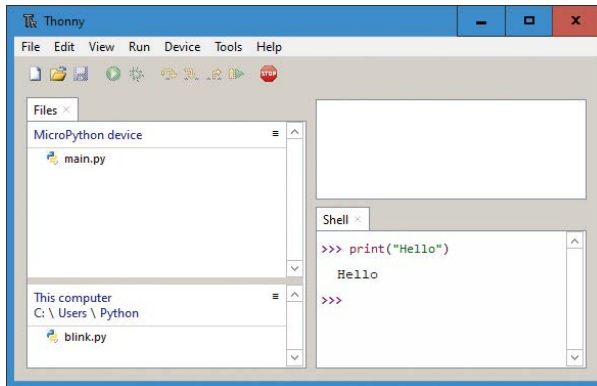


Рис.4.5: Команда печати в консоли Thonny.

4.3 Отступы и блоки

MicroPython различает разные блоки по отступу. Нет необходимости использовать фигурные скобки ("{}") или подобные. Это одно из основных отличий от большинства других языков, таких как C, Pascal, Basic и т. д. Преимущество этого метода в том, что он практически вынужден придерживаться определенной степени структуры программы.

```
if True:
    # block 01
    print ("True")
else:
    # block 02
    print ("False")
```

Количество пробелов для отступов может быть разным, но в одном и том же блоке всегда должно поддерживаться одинаковое количество пробелов для отступов.

В противном случае отображается сообщение об ошибке:

```
if True:
    print ("Answer")
    print ("True")
else:
    print ("Answer")
    print ("False") # Другой отступ приведет к ошибке во время выполнения.
```

Это приводит к следующему сообщению:

```
>>> %Run -c $EDITOR_CONTENT
Traceback (most recent call last): - (последний вызов
  File "<stdin>", line 6
IndentationError: unexpected indent - неожиданный отступ
```

В условиях и циклах блоки формируются одинаково.

4.4 Управляемое оборудование: цифровые входы и выходы

В отличие от программирования на ПК или ноутбуке, MicroPython обычно фокусируется на прямом доступе к аппаратным функциям. Особое значение имеет управление отдельными пинами ввода/вывода (пины ввода/вывода или GPIO для ввода/вывода общего назначения).

Чтобы проверить инструкции на практике, все, что вам нужно сделать, это подключить светодиод с последовательным резистором к ESP32. На следующем рисунке показана соответствующая схема подключения:

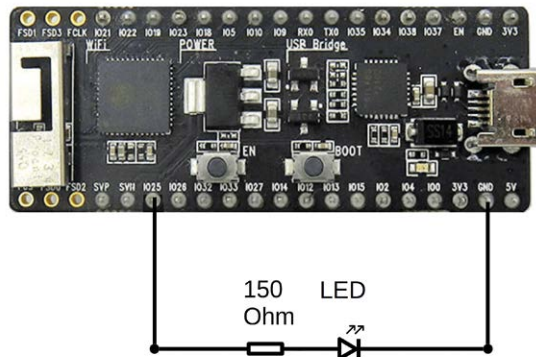


Рис.4.6: Схема подключения светодиода к порту 25.

Схема на макетной плате выглядит так:

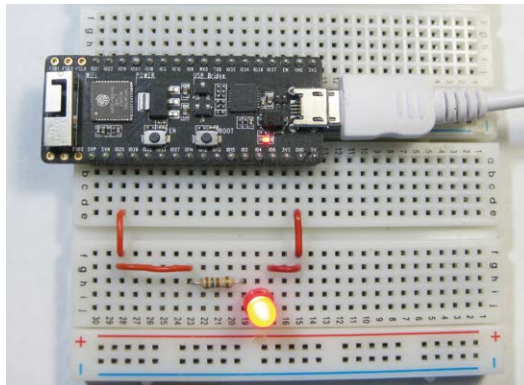


Рис.4.7: Светодиод подключен к порту 25.

Что касается программного обеспечения, цифровой вывод должен быть сначала инициализирован. Для этого создается переменная, соответствующая пину. Это не просто номер пина, а полноценный объект, который знает все аспекты пина. Чтобы использовать пины, класс "Pin" должен быть импортирован из модуля "machine".

```
From machine import Pin
```

После этого уже можно выполнить инициализацию ножки светодиода:

```
led = Pin(25, Pin.OUT)
```

Цифра 25 — это номер GPIO, которая обычно находится на плате напротив соответствующего контакта. Инструкция определяет этот контакт как выход.

Рис.4.8: Номера контактов здесь (здесь 25 и 26) на плате ESP.

Теперь ножку LED можно использовать:

```
led.value(1)
```

Он устанавливает порт ввода-вывода в «1», т. е. теперь он несет напряжение 3,3 В. Если все подключено правильно, светодиод на порту 25 загорится после выполнения команды через консоль. Сброс пина в «0» (0 вольт) выполнит следующую инструкцию:

```
led.value(0)
```

Вместе с инструкцией сна (см. также следующую главу) из модуля time теперь вы можете запрограммировать светодиодную мигалку ([blink_simple.py](#)):

```
# blink_simple.py

from machine import Pin
from time import sleep
led = Pin(25, Pin.OUT)
while True:
    led.value(1)
    sleep(1)
    led.value(0)
    sleep(1)
```

Из-за инструкции ожидания

```
sleep(1)
```

теперь светодиод мигает с интервалом в одну секунду, т. е. с периодом в две секунды или с частотой 0,5 герц.

Каждый контакт ввода-вывода также может использоваться как цифровой вход. Соответствующее инициирование осуществляется через:

```
pin_in = Pin(25, Pin.IN)
```

Это позволяет определить уровень, подаваемый на контакт, т. е. 0 для 0 В и 1 для 3,3 В. Открытый вход, т. е. контакт без какого-либо соединения, не дает надежного результата. Сигналы помех, такие как помехи 50 Гц, сигналы WLAN или радиочастоты, могут случайным образом изменять уровень. Для предотвращения этого необходимы так называемые подтягивающие или ограничивающие резисторы. С ESP32 они могут быть включены даже внутри. С использованием

```
pin_in = Pin(2, Pin.IN, Pin.PULL_UP)
```

гарантирует, что на неподключенный вход поступает сигнал высокого уровня. Соответственно,

```
button_pin = Pin(2, Pin.IN, Pin.PULL_DOWN)
```

несет низкий сигнал на открытом входе. С помощью следующей программы можно протестировать функции:

```
# read_pin_25.py
from machine import Pin
from time import sleep

pin_in = Pin(25, Pin.IN, Pin.PULL_DOWN)

while(True):
    print(pin_in())
    sleep(1)
```

На рисунке схема выглядит так:

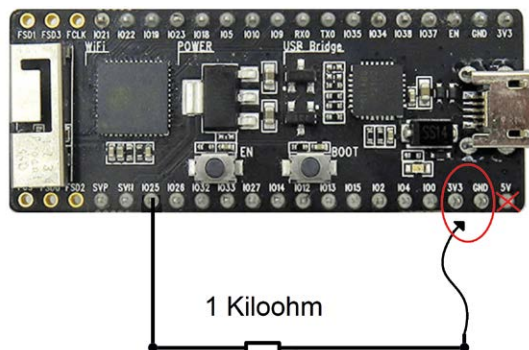


Рис.4.9: Регистрация уровней напряжения.

При настройке оборудования можно использовать отрезок провода для проверки схемы, которая подключена к GND или 3,3 В:

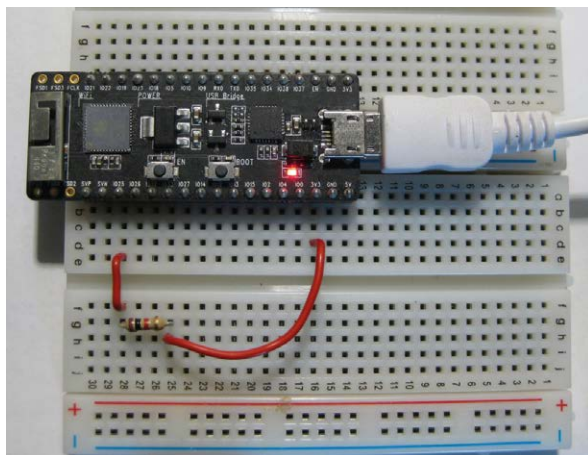


Рис. 4.10: Аппаратная структура для считывания уровней напряжения

В зависимости от того, подключена ли перемычка к GND (0 В) или 3V3, на клемме появляется значение 0 или 1. Если вход разомкнут, на терминале отображается 0 (для ограничения) или 1 (для подтягивания).

ВНИМАНИЕ: Ни в коем случае нельзя подключать вход к 5 В. Это может привести к выходу из строя микросхемы. Выводы ESP32 могут управляться только сигналами с максимальным напряжением 3,3 В!

Резистор 1 кОм используется для защиты входа. Это снижает вероятность непреднамеренного применения чрезмерного напряжения, вызывающего повреждение. Тем не менее, вы всегда должны избегать подачи напряжения выше 3,3 В на вывод ESP32.

4.5 Управление временем и сон

В примере с мигающим светодиодом инструкция задержки уже использовалась. Она содержится в модуле «time» и управляется

```
import time
```

С оператором

```
time.sleep(seconds)
```

можно установить фиксированное время задержки в секундах. Кроме того, можно импортировать только саму команду сна:

```
from time import sleep
```

Тогда инструкцию можно сократить до

```
sleep(seconds)
```

Хотя команду можно использовать и для долей секунды, для очень коротких задержек рекомендуется использовать

```
time.sleep_ms(milliseconds)
```

потому что она имеет лучшую точность для малых времен.

Недостаток этих функций в том, что они тормозят. Это означает, что контроллер не может выполнять какие-либо другие задачи в течение периода ожидания, поскольку он занят подсчетом тактов процессора. Альтернативой является использование прерываний или других методов программирования. Подробности приведены в последующих главах.

Для запросов времени доступны следующие две подпрограммы:

```
time.ticks_ms()
time.ticks_us()
```

Они показывают текущее время работы системы в милли- или микросекундах. Классическим приложением является измерение времени выполнения программы. С помощью следующего кода можно показать, например, что математические операции требуют определенного времени:

```
# runtime.py

import time
import math

while(True):
    start = time.ticks_us()

    # x = math.exp(math.sin(22.5))

    stop = time.ticks_us()
    print(stop-start)
```

Если перед расчетом убрать знак комментария, отображаемая скорость обработки увеличится где-то от 100 050 мкс до 100 170 мкс. Таким образом, время вычисления формулы составляет около 120 мкс.

4.6 Важные значения: переменные и константы

В Python особенно легко создавать переменные. Нет необходимости указывать тип данных переменной во время присваивания. Это главное отличие от других языков. Там переменные всегда должны быть явно инициализированы с определенным типом (например, `int a = ...`)

Переменные также можно использовать в консоли:

```
>>> a = 17
>>> print(a)
17
```

К присвоению имен переменных применяются следующие правила:

- Имя переменной должно содержать только цифры, буквы и символы подчеркивания.
- Первым символом имени переменной должна быть буква или знак подчеркивания.
- Имя переменной чувствительно к регистру.

Переменным могут быть присвоены значения разных типов. Типы в MicroPython включают числа, строки, списки, словари, кортежи и т. д. С помощью `type()` можно проверить тип данных переменных и констант, например.

```
>>> a = 17
>>> print(type(a))
<class 'int'>

>>> a, b, c, d = 17, 1.5, True, 5+7j
>>> print(type(a), type(b), type(c), type(d))
<class 'int'> <class 'float'> <class 'bool'> <class 'complex'>
```

Цифры вроде 10, 100 или строки вроде «Hello World!» являются константами. MicroPython предлагает ключевое слово «`const`», обозначающее неизменяемое значение

```
from micropython import const

a = const(33)
print(a)
```

4.7 Количество и типы переменных

MicroPython поддерживает следующие типы чисел

- Integer (int); Целое (int)
- Floating point (float); С плавающей запятой (float);
- Boolean (bool); and Булево значение (bool);
- Complex.

Числовой объект создается сразу после указания значения. С помощью `del()` вы можете снова удалить объекты. В консоли это выглядит так:

```
>>> a = 17
>>> a
17
>>> del a
>>> a
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'a' is not defined
>>> |
```

Рис.4.11: Удаление переменных.

Также возможно присваивать значения нескольким переменным одновременно, т.е. разделяя их только запятыми, например:

```
a, b = 1, 2
```

Результат деления (/) всегда возвращает значение с плавающей запятой, например:

```
12/3 = 4.0
```

Целочисленный формат эквивалентен числам в математике, включая знаки:

```
1, 100, -8080, 0 , .....
```

Число с плавающей запятой соответствует действительным числам. Также можно использовать экспоненциальное представление:

```
>> a = 1.234e3
>>> a
> 1234.0
```

Важное различие между целым числом и числом с плавающей запятой заключается в том, что целые значения обязательно хранятся. С другой стороны, числа с плавающей запятой округляются.

Переменные типа `Bool` могут принимать только два значения "True" и "False". Здесь также делается различие между прописными и строчными буквами.

Переменная типа «complex» состоит из действительной и мнимой частей в виде $a+bj$ или комплекса(a, b). И действительная часть a , и мнимое значение b могут быть числами с плавающей запятой.

Это также позволяет выполнять вычисления с комплексными числами:

```
>>> a = 1j
>>> b = 1j
>>> a*b
>>> (-1+0j)
```

Также можно использовать двоичные (0b...) и шестнадцатеричные (0x...) числа:

```
>>> a = 0b1010
>>> a
>>> 10

>>> a = 0xff
>>> a
>>> 255
```

4.8 Преобразование типов чисел

Числа и переменные также могут быть преобразованы из одного типа в другой. Для этого используются следующие инструкции:

<code>int (x):</code>	Преобразует x в целое число
<code>float (x):</code>	Преобразует x в число с плавающей запятой
<code>complex (x):</code>	Преобразует x в комплексное число

В первом случае десятичная часть не округляется, а отрезается.

```
>>> a = 2.0
>>> print(int(a)) # Преобразовать число с float в int
2

>>> x, y = 4.3, 7
>>> c = complex(x, y)
>>> print(c)
(4.3+7j)
```

4.9 Маленькие большие данные: массивы

Массивы представляют собой набор элементов одного типа. Они доступны в большинстве языков программирования, таких как Java или C/C++.

Преимущество массивов в том, что они могут хранить большие объемы данных в структурированном виде. Если, например, необходимо записать серию измерений, отдельные результаты можно сохранить в виде массива. Затем вы можете легко получить доступ к отдельным значениям. Для работы с массивами должен быть загружен соответствующий модуль, например:

```
import array as arr
```

Тогда уже можно создавать массив:

```
a = arr.array('d', [1.1, 3.5, 4.5])
```

Для числового типа, используемого в массиве, можно указать либо «d» для чисел с плавающей запятой, либо «i» для целых чисел.

В следующей программе сначала создается массив. Затем значения выводятся по одному в консоль:

```
# array.py

import array as arr
a = arr.array('d', [1.1, 3.5, 4.5])

for n in range(len(a)):
    print(a[n])
```

Использование массива может привести к очень эффективным программам. Пример этого можно найти в главе 9 при одновременной оценке нескольких датчиков температуры.

4.10 Операторы

Здесь представлены арифметические операторы MicroPython:

+	Сложение	$c = a + b$
-	Вычитание	$c = a - b$
*	Умножение	$c = a * b$
/	Деление	$c = a / b$

Есть также несколько более специализированных операторов:

//	Целочисленное деление	$11 // 3 ==> 3$
%	по модулю (остальная часть делен.)	$11 \% 3 ==> 2$
**	Показатель степени	$2 ** 3 ==> 8$

Логические операторы "not", "and" и "or" также доступны в MicroPython:

a	not a
True	False
False	True

a	b	a and b	a or b
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

Битовые операторы представлены в Python следующими инструкциями:

&	Побитовое AND
	Побитовое OR
^	Побитовое XOR
~	Побитовое NOT

Также можно использовать операторы сдвига:

<< Move bits	Переместить биты влево
>> Move bits	Переместить биты вправо

Следующие примеры иллюстрируют применение:

```
>>> a = 0b1010
>>> b = 0b1100
>>> bin(a&b)
,0b1000'

>>> a = 0b1010
>>> bin(a<<1)
,0b10100'
```

Операторы сравнения, опять же, имеют свой обычный математический смысл:

==	равно
!=	Не равно
>	больше, чем
<	меньше, чем
<=	меньше или равно
>=	больше или равно

Например

```
>>> a = 5
>>> b = 7
>>> a < b True
>>> a > b False
```

4.11 С форматом, пожалуйста: привлекательный текст и вывод данных

Вывод текста и данных прост, если их не нужно форматировать. Однако часто вывод строк чисел или текстов фиксированной длины должен быть выровнен по правому или левому краю. Также часто требуется вывод измеренных значений с ведущими нулями или определенным числом знаков после запятой. MicroPython предлагает множество удобных решений. Наиболее важные из них описаны в текущем разделе.

Самый простой способ вывода текста — это уже известный оператор печати:

```
print("This is a string")
```

Если текст или числа должны быть встроены в другой текст, лучше всего использовать заполнители. Они обозначаются парой фигурных скобок `{}`. Каждая строка может быть расширена этим оператором форматирования. Пара квадратных скобок `"{}"` пишется в тексте везде, где что-то должно быть вставлено. Затем в команде форматирования значения просто даются в требуемом порядке. В консоли это выглядит так:

```
>>> result = "4 * 7"
>>> print("The result of {} is {}".format(result, 4*7))
The result of 4 * 7 is 28
```

Также можно указать длину поля. Кроме того, данные и тексты могут быть отформатированы по левому или правому краю или по середине. Для этого используется двоеточие `:`. Следующая программа показывает пример приложения:

```
# format.py

text = "Standard {:10}". numbers"pprint(text.format(12345))

text = "Left-aligned {:>10} numbers"print(text.format(12345))

text = "Right-aligned {:<10} numbers"print(text.format(12345))

text = "centred {:^10}". numbers"print(text.format(12345))
```

Проблема выглядит следующим образом

```
Standard 12345 numbers
Left-aligned    12345      numbers
Right-aligned           12345 numbers
Centered          12345      numbers
```

Для табличных представлений часто желательно, чтобы тексты и рисунки были представлены сжато. Для этой цели должна быть определена фиксированная длина для вывода чисел. Заполнитель снова дополняется двоеточием, затем указывается желаемая длина и определяется тип данных. Числа с десятичными разрядами требуют большого количества вычислений и поэтому в MicroPython поддерживаются лишь в минимальной степени. Существуют ограничения по сравнению со стандартным Python в отношении возможностей форматирования и точности. Тем не менее, количество знаков после запятой и общая длина могут быть указаны для форматирования. Действует следующая инструкция:

```
{:Total length.decimal placesX}
```

X означает d (целое число) или f (плавающее число). Программа

```
# table_format.py
a = 3.5
output = "Result is {:8.4f} Volt"
print(output.format(a))

a = 123.5
output = "Result is {:8.4f} Volt"
print(output.format(a))

a = -3.5223
output = "Result is {:8.4f} Volt"
print(output.format(a))

a = 7/3
output = "Result is {:8.4f} Volt"
print(output.format(a))
```

приводит несколько примеров:

```
Результат: 3.5000 Volt
Результат: 123.5000 Volt
Результат: -3.5223 Volt
Результат: 2.3333 Volt
```

Количество разрядов всегда следует корректировать с требуемой точностью, иначе могут возникнуть непредвиденные ошибки округления. Также доступны следующие варианты:

```
По левому краю: {<8.4f}
По правому краю: {>8.4f}
По центру: {^8.4f}
Всегда выводить знак: {:+8.4f}
Ведущие нули и знаки: {:+08.4f}
Установите знак один под другим: {:+=8.4f}
```

Установите один знак под другим и ведущими нулями: `{:0=+8.4f}`

Значениям также можно давать имена. Затем значения можно вводить в любом порядке:

```
# labeled_format.py
text = "The result of {calc} is {res}"
print(text.format(res = 3*8, calc = "3 x 8"))
text = "The result of {calc} is {res:5.3f}"
print(text.format(res = 22 / 3, calc = "22 / 3"))
```

Это приводит к следующему результату:

Результат 3 x 8 равен 24.

Результат 22/3 7.333

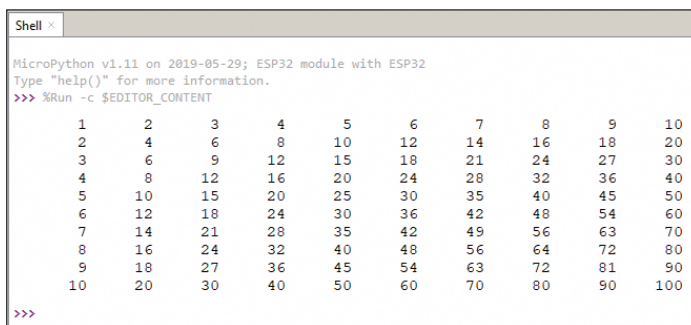
Вот тоже инструкция

```
print("No NewLine",end=" ")
```

используется для подавления чересстрочной развертки. Это позволяет очень легко выводить отформатированные таблицы. Например, программа

```
# Multiplication_table.py
for i in range(1,11,1):
    for j in range(1,11,1):
        print("%d"%(j*i),end=" ")
    print()
```

предоставляет следующую таблицу в классическом формате 1 x 1:



```
Shell <
MicroPython v1.11 on 2019-05-29; ESP32 module with ESP32
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT
  1    2    3    4    5    6    7    8    9   10
  2    4    6    8   10   12   14   16   18   20
  3    6    9   12   15   18   21   24   27   30
  4    8   12   16   20   24   28   32   36   40
  5   10   15   20   25   30   35   40   45   50
  6   12   18   24   30   36   42   48   54   60
  7   14   21   28   35   42   49   56   63   70
  8   16   24   32   40   48   56   64   72   80
  9   18   27   36   45   54   63   72   81   90
 10   20   30   40   50   60   70   80   90  100
>>>
```

Рис.4.12: Маленькая таблица умножения 1 на 1.

4.12 Символы в цепочках: строки

Строки являются одними из самых популярных типов в MicroPython. Их можно легко создать, поместив строки в кавычки. MicroPython поддерживает как одинарные, так и двойные кавычки. Создать строку так же просто, как присвоить значение переменной:

```
var1 = 'Hello!'  
var2 = "Python"
```

MicroPython не поддерживает типы символов. Этот вариант рассматривается как строка символов длины one и поэтому также считается подстрокой. Квадратные скобки используются для доступа к подстрокам. Это позволяет вырезать любые части строки:

```
var1 = ,Hello!'  
var2 = "Python"  
  
print (var1[0])  
print (var2[1:5])
```

возвращает результат

```
>>> H  
>>> ytho
```

Существующую строку можно «обновить», присвоив переменную другой строке, возможно, снова. Новое значение может быть связано с его предыдущим значением или с совершенно другой строкой. Например

```
var = 'Hello World!'  
print ("Updated String : ", var[:6] + 'Python')
```

дает результат

```
>>> Updated String : Hello Python
```

Важным моментом при работе со строками являются «escape characters - экранирующие символы». Следующая таблица содержит список этих управляющих или непечатаемых символов, которые могут отображаться с обратной косой чертой.

Character	Function	Effect
\b	Backspace	Шаг назад
\n	Новая линия	Разрыв строки
\r	Возврат каретки	Возврат каретки
\s	Space	Space
\t	Tab	Tabulator

Например, оператор

```
>>> print("Hello \n World")
```

дает результат

```
>>> Hello
      World
```

Глава 5 • Контроллер в практическом использовании

Теперь, когда вы знакомы с некоторыми важными основами программирования, пришло время для первых практических приложений. Помимо вывода на консоль, светодиоды, в частности, будут использоваться в качестве индикаторов состояния портов. Это уже позволяет реализовать некоторые интересные приложения.

В этих примерах можно использовать либо встроенные светодиоды отдельных плат, либо внешние светодиоды. Встроенные светодиоды обычно подключаются к порту 02. Особенно это касается платы NodeMCU. Плата Node-ESP даже имеет многоцветный светодиод трех цветов, подключенный к портам 00, 02 и 04. Комплект Pico не имеет встроенного светодиода. В этом случае вам придется переключиться на внешние светодиоды (см. рис. 4.6 или 4.7). Не забудьте добавочный резистор.

5.1 Мигающий LED как имитатор системы охранной сигнализации

Мигающий светодиод уже реализован в разделе 4.4. В отличие от постоянно горящих дисплеев мигающие огни привлекают внимание. Поэтому мигание часто привлекает внимание к исключению или опасной ситуации. Например, постоянно горящий светодиод на морозильной камере обычно указывает на то, что устройство работает нормально. Если светодиод начинает мигать, это обычно указывает на то, что температура внутри больше не соответствует заданному значению. То же самое относится и к электроприборам на батарейках. Горящий светодиод указывает на полный заряд батареи. Если светодиод начинает мигать, остается лишь небольшое количество энергии.

Для систем сигнализации мигающий или мигающий светодиод указывает на то, что устройство активировано. Таким образом, светодиод в режиме мигания может использоваться в качестве симулятора системы охранной сигнализации. Вариации таких устройств действительно используются на практике. Их можно, например, установить в автомобиле. Их невозможно отличить от настоящей, вооруженной сигнализации, и они удержат от взлома хоть каких-нибудь воров-авантюристов. Соответствующую программу опять же можно найти в прилагаемой папке с кодами.

```
# Alarmsimulator.py
from machine import Pin
from time import sleep
led = Pin(2, Pin.OUT)
while True:
    led.value(1)
    sleep(.1)
    led.value(0)
    sleep(5)
```

Единственное отличие от программы перепрошивки состоит в том, что значения в инструкциях `sleep()` были изменены на 0,1 или 5. Это превращает обычное мигание в короткое. Как только новая программа будет загружена, светодиод будет кратковременно мигать каждые пять секунд, и симулятор системы охранной сигнализации готов к использованию.

5.2 Полезно в экстренной ситуации: автоматический сигнал SOS

Еще одно интересное приложение — автоматический сигнал SOS. Это может даже спасти жизнь в нештатных ситуациях. Чтобы подать сигнал SOS, светодиод должен сначала мигнуть тремя короткими вспышками, затем тремя длинными и, наконец, снова тремя короткими. Эта последовательность сигналов должна постоянно повторяться.

Код имитатора охранной сигнализации можно легко изменить так, чтобы светодиод подавал нужный сигнал:

```
# SOS.py

from machine import Pin
from time import sleep

led = Pin(2, Pin.OUT)

while True:

    # "S"
    led.value(1)
    sleep(.1)
    led.value(0)
    sleep(.5)
    led.value(1)
    sleep(.1)
    led.value(0)
    sleep(.5)
    led.value(1)
    sleep(.1)
    led.value(0)
    sleep(1)

    # "O"
    led.value(1)
    sleep(.4)
    led.value(0)
    sleep(.5)
    led.value(1)
    sleep(.4)
    led.value(0)
    sleep(.5)
    led.value(1)
    sleep(.4)
    led.value(0)
    sleep(1)
```

```
# "S"
led.value(1)
sleep(.1)
led.value(0)
sleep(.5)
led.value(1)
sleep(.1)
led.value(0)
sleep(.5)
led.value(1)
sleep(.1)
led.value(0)
sleep(1)

sleep(2)
```

Это, конечно, не самый элегантный метод генерации сигнала SOS. Однако эта программа демонстрирует, что даже без обширных знаний языка программирования Python вы можете реализовать свои собственные полезные проекты. В разделе 6.4 показано, как такую программу можно реализовать гораздо эффективнее.

Тем не менее, программа выполняет свою задачу. Если используется особенно яркий светодиод, схема может оказаться весьма полезной в таких ситуациях, как бедствие на море или в горах.

Глава 6 • Структуры программ

Ни один язык программирования не может обойтись без управляющих структур. Для этой цели MicroPython предоставляет инструкции ветвления и цикла. Пример генерации сигнала SOS уже показал, что при программировании часто приходится повторять множество одинаковых или похожих последовательностей. Циклы предлагают гораздо более элегантный метод для этой цели, чем простое повторение инструкций в коде.

Если необходимо принять решение, необходимы инструкции по ветвлению. Это позволяет программе правильно реагировать на различные ситуации.

В отличие от других языков программирования, MicroPython обладает очень широкими функциональными возможностями, когда речь идет о циклах. Это будет объяснено в следующих разделах.

6.1 Условия и циклы

В ответвлении условие определяется в программе ключевыми словами «if - если» и «else - иначе». В зависимости от того, истинно это условие или нет, программа будет продолжаться в разных точках. Итак, программа возвращает

```
Temperature = 2

if temperature < 3:
    print("Risk of frost" )
else:
    print("No danger of frost")
```

это приведет к выводу

```
>>> Risk of frost
```

Если вместо этого установлено, что Temperature = 17, выход будет

```
>>> No danger of frost
```

Результат после ключевого слова «if» возвращает логическое выражение. Это может быть либо правдой, либо ложью. Если выражение истинно, операторы выполняются сразу после строки if. Эти операторы должны иметь отступ, чтобы было ясно, какие операторы принадлежат блоку if. Обратите внимание, что логические выражения в строке if должны заканчиваться двоеточием.

Операторы «else» выполняются только в том случае, если запрос if неверен.

Циклы используются для повторения инструкций. С помощью циклов блоки кода могут выполняться несколько раз. Выполнение продолжается до тех пор, пока не будет выполнено заданное условие. В MicroPython доступны два типа циклов:

-
- циклы `while`
 - для циклов

Если вы хотите вывести на консоль числа от 1 до 10, можно использовать следующий цикл `while`:

```
number=1
while number<=10:
    print(number)
    number=number+1
```

Повторяемый код снова указывается отступом. Он выполняется до тех пор, пока значение переменной «number» меньше или равно (`<=`) 10. В каждом цикле отображается текущее число, а затем значение числа увеличивается на 1.

Задачу также можно выполнить с помощью цикла `for`:

```
number = 1
for number in range(1, 10):
    print(number)
```

Цикл `for` выполняется до тех пор, пока значение переменной `counter` находится в диапазоне от 1 до 10. Функция `range()` присваивает переменной соответствующий диапазон значений. Цикл `for` продолжает работать до тех пор, пока переменная счетчика меньше заданного конечного значения. Поэтому число 9 выводится как наибольшее значение.

Таким образом, циклы `for` используются, когда кодовый блок должен быть повторен с определенным заранее заданным числом проходов. Циклы `while`, с другой стороны, извлекают часть кода до тех пор, пока не будет выполнено определенное условие.

Как и в случае с условными операторами, логические выражения в циклах `for` и `while` должны заканчиваться двоеточием.

6.2 Ходовые огни и освещение аэропорта

До сих пор плата ESP управляла только одним светодиодом. Зато с помощью шлейфов можно без проблем управлять несколькими светодиодами с интересными рисунками. В этом разделе изначально будут использоваться 5 светодиодов. Они должны загореться один за другим. Такое световое шоу можно увидеть, например, в конце автомагистралей или на дорожных стройках. Здесь несколько фотовспышек соединены таким образом, чтобы создавалось впечатление, что по сигнальной цепочке движется светящаяся точка. На взлетно-посадочных полосах аэропортов такие «бегущие огни» часто также служат сигналом приближающихся самолетов. Принципиальная схема этого приложения выглядит следующим образом:

Рис.6.1: Принципиальная схема ходовых огней (чейзера).

Схема может быть легко смонтирована на небольшой макетной плате:

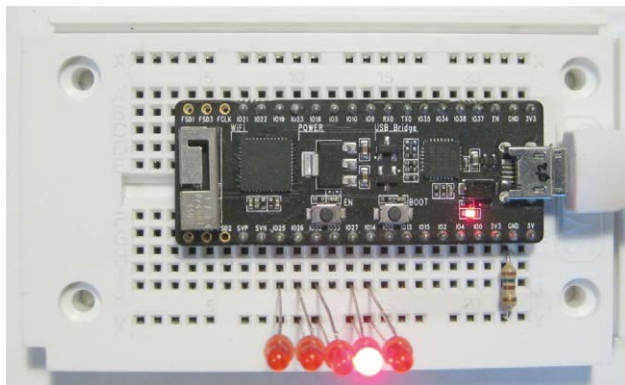


Рис.6.2: Метод изготовления ходовых огней или светодиодного чейзера.

Программа MicroPython для этого выглядит так:

```
# LED_chaser_list.py

from machine import Pin
from time import sleep

LED=[25,26,32,27,14]

del_time = 0.1

for n in range(5):
    LED[n]=Pin(LED[n],Pin.OUT)

while True:
    for n in range(5):
        LED[n].value(1)
```

```
sleep(del_time)
LED[n].value(0)
```

В первом цикле `for` все используемые порты настраиваются как выходные. Второй цикл `for` обеспечивает последовательное загорание светодиодов.

6.3 Электронная радуга: использование RGB-светодиода

Теперь, когда приложения с несколькими светодиодами больше не являются проблемой, также можно использовать так называемые многоцветные светодиоды. Во-первых, программу чейзера можно изменить:

```
# MulticolorLED.py

from machine import Pin
from time import sleep

LED = [25,26,32]

del_time = 0.5

for n in range(3):
    LED[n] = Pin(LED[n],Pin.OUT)

while True:
    for n in range(3):
        LED[n].value(1)
        sleep(del_time)
        LED[n].value(0)
```

После загрузки отдельные чипы RGB-светодиода, подключенные к портам 25, 26 и 32, мигают синим, зеленым и красным цветами один за другим. В более позднем приложении будут предприняты меры для обеспечения плавности переходов, что приведет к приятному внутреннему освещению.

На следующем рисунке показано, как подключить RGB-светодиод к ESP32 в соответствии с приведенной выше программой.

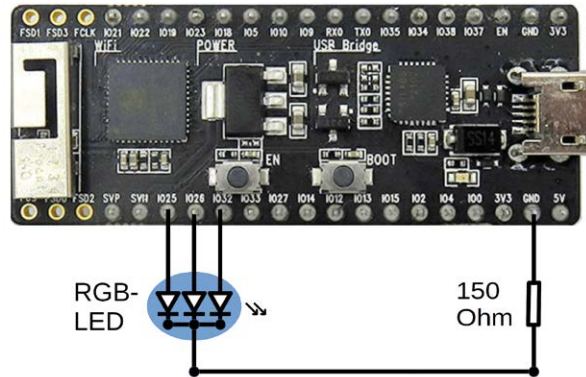


Рис.6.3: Принципиальная схема управления многоцветным LED.

6.4 SOS в компактном стиле

С уже знакомыми программными структурами приложение SOS может быть запрограммировано более элегантно и компактно:

```
# SOS_compact.py
from machine import Pin
from time import sleep
led=Pin(2,Pin.OUT)
while True:
    for n in range(3):
        led.value(1)
        sleep(.1)
        led.value(0)
        sleep(.5)
    sleep(1)
    for n in range(3):
        led.value(1)
        sleep(.4)
        led.value(0)
        sleep(.5)
    sleep(1)
    for n in range(3):
        led.value(1)
        sleep(.1)
        led.value(0)
        sleep(.5)
    sleep(2)
```

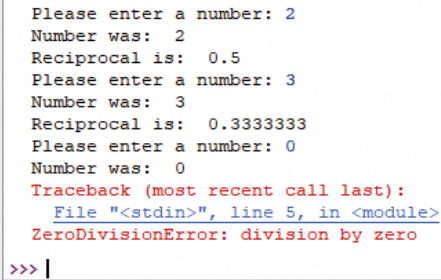
This is where the advantage of looping becomes clear. The instructions to be repeated need only be specified once. The loop ensures that they are repeated according to the requirements.

6.5 Метод проб и ошибок: попробовать и исключить

Во время выполнения программы могут возникать исключения или ошибки. Обработка исключений — это процедура, которая позволяет перехватывать такие ситуации. Затем код может выполняться дальше без прерывания программы. Некоторые более высокие языки программирования, такие как Java или Ruby, а также MicroPython, имеют встроенные механизмы для включения обработки исключений. В Python код, подверженный риску исключения, встроен в блок `try/except`. Следующая программа должна вычислять обратные величины:

```
# reciprocal.py
while True:
    n = int(input("Please enter a number: "))
    print("Number was: ", n)
    print("Reciprocal is: ", 1/n)
```

Пока вводятся значения, отличные от нуля, программа работает отлично. Однако, если введен ноль, программа прерывается с сообщением об ошибке:



```
Please enter a number: 2
Number was: 2
Reciprocal is: 0.5
Please enter a number: 3
Number was: 3
Reciprocal is: 0.3333333
Please enter a number: 0
Number was: 0
Traceback (most recent call last):
  File "<stdin>", line 5, in <module>
ZeroDivisionError: division by zero
>>> |
```

Рис.6.4: Прерывание программы с сообщением об ошибке.

Используя конструкцию `try/except`, можно перехватить ошибку:

```
# try_except.py
while True:
    n = int(input("Please enter a number: "))
    print("Number was: ", n)
    try:
        print("Reciprocal is: ", 1/n)
    except:
        print("error")
```

Если теперь введен ноль, сообщается об ошибке ("error"), но программа продолжает работать и доступна для новых записей:

```
>>> %Run -c $EDITOR_CONTENT
Please enter a number: 0
Number was: 0
error
Please enter a number: |
```

Рис.6.5: Ошибка перехвачена.

При использовании микроконтроллеров эта опция используется, помимо прочего, для запроса значений датчиков. Если датчик не дает разумного значения, это не приводит к сбою программы, а только к сообщению об ошибке. Когда датчик снова получает пригодные для использования значения, система может продолжать работать без перерыва.

Глава 7 • Генерация аналоговых сигналов

Пины классического порта ввода-вывода микроконтроллера могут принимать только цифровые значения напряжения. Контроллер ESP32 работает с 3,3 В и, следовательно, только два напряжения

LOW: 0 V

HIGH: 3.3 V

Реальный мир, напротив, не существует в цифровом виде. Резких изменений температуры не происходит. В течение дня яркость в помещении без искусственного освещения непрерывно увеличивается и уменьшается. Итак, в природе и в быту всегда существуют непрерывные, т. е. аналоговые, промежуточные значения. Во многих приложениях такое поведение также должно имитироваться техническими средствами. Например, гораздо приятнее проснуться от медленно нарастающей музыки, чем от внезапного сигнала будильника. Постепенно усиливающийся свет более приятен для глаз, чем свет включенной лампы..

Существуют различные методы генерации аналоговых напряжений. Двумя наиболее важными из них являются:

- широтно-импульсная модуляция (ШИМ)
- Цифро-аналоговый преобразователь (ЦАП)

В этой главе показано, как оба метода можно использовать в MicroPython. Далее объясняются различные приложения.

7.1 Широтно-импульсная модуляция

Если светодиоды должны не только включаться и выключаться, но и принимать скользкие значения яркости, можно использовать метод широтно-импульсной модуляции (ШИМ). С помощью этого метода можно генерировать квазианалоговый сигнал на чисто цифровом выходе. Это достигается очень быстрым переключением порта между LOW и HIGH, что управляется двумя параметрами:

- частота переключения
- рабочий цикл (скважность)

Рабочий цикл определяется продолжительностью высокого уровня по сравнению с продолжительностью всего периода сигнала. Максимальное напряжение сигнала достигается, когда порт постоянно включен (скважность = 100%). Если порт постоянно находится на НИЗКОМ уровне (рабочий цикл = 0%), выводится минимальный уровень сигнала 0 В. Следующий рисунок иллюстрирует кривую сигнала, которая имеет место в этом случае.

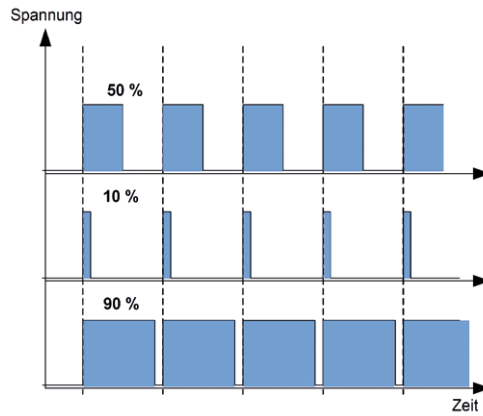


Рис.7.1: ШИМ-сигнал.

На экране осциллографа сигналы выглядят так:

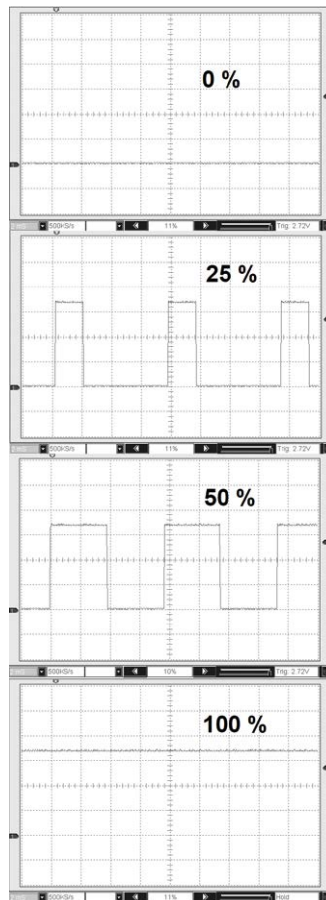


Рис.7.2: ШИМ-сигнал на экране осциллографа.

С аппаратной стороны светодиод снова подключается к порту 25 (см. рис.4.7). Чтобы активировать порт с поддержкой ШИМ, необходимо сначала импортировать модуль времени:

```
from time import sleep
```

Затем можно создать объект PWM с именем «LED»:

```
LED = machine.Pin(25)
pwmLED = machine.PWM(LED)
```

Этот объект требует два параметра:

- пин, к которому он подключен
- базовая частота сигнала ШИМ

Для базовой частоты можно использовать значение от 0 до 78125. Частоты 5000 Гц вполне достаточно для управления светодиодом:

```
pwmLED.freq(5000)
```

ШИМ работает с разрешением 10 бит. Таким образом, возможно $2^{10} = 1024$ значения. Таким образом, переменная PWM может принимать значение от 0 до 1023. Здесь 1023 соответствует скважности 100 % (полная яркость), а 0 соответствует скважности 0 % (светодиод не горит).

Метод `duty()` используется для установки коэффициента включения/выключения. Параметр

```
timedelay = 0.005
```

управляет скоростью бега. Если светодиод должен постепенно становиться ярче, а затем снова темнеть, появится следующий код:

```
# LED_fader.py
import machine
from time import sleep

LED = machine.Pin(25)

# создаем ШИМ-объект
pwmLED = machine.PWM(LED)
timedelay = 0.005

# установить частоту и рабочий цикл от 0 (все выключено) до 1023 (все включено)
# 512 -> 50% рабочий цикл
```

```

pwmLED.freq(5000)

while True:

    for dc in range (0,1023):
        pwmLED.duty(dc)
        pwmLED
        sleep(timedelay)

    for dc in range (1023, 0, -1):
        pwmLED.duty(dc)
        pwmLED
        sleep(timedelay)

```

7.2 Для романтических вечеров: симулятор сердцебиения

«Бьющееся сердце» иногда бывает на распродажах. Это красный светодиод с ШИМ-управлением в прозрачном пластиковом корпусе в форме сердца. Благодаря знаниям, полученным в предыдущих разделах, сигнал бьющегося сердца также можно легко имитировать с помощью MicroPython. Следующая программа имитирует типичное двойное биение человеческого сердца:

```

# HeartBeatSimulator.py
import machine
from time import sleep_us
LED = machine.Pin(25)
# создаем ШИМ-объект
pwmLED = machine.PWM(LED)
timedelay1 = 300
timedelay2 = 1000
pwmLED.freq(5000)
while True:
    # основной такт
    for dc in range (0,1023):
        pwmLED.duty(dc)
        pwmLED
        sleep_us(timedelay1)
    for dc in range (1023, 0, -1):
        pwmLED.duty(dc)
        pwmLED
        sleep_us(timedelay1)
    # low такт

```

```

for dc in range (0,255):
    pwmLED.duty(dc)
    pwmLED
    sleep_us(timedelay2)
for dc in range (255, 0, -1):
    pwmLED.duty(dc)
    pwmLED
    sleep_us(timedelay2)

pwmLED.duty(0)
sleep_us(500000)

```

Вместо команды `sleep()` здесь используется вариант `sleep_us()`. Это создает паузу в микросекундах, что приводит к гораздо лучшему временному разрешению.

7.3 Light alarm clock for a relaxed wake-up

Искусственный восход солнца также можно имитировать с помощью ШИМ-сигналов, если яркость светодиода увеличивается очень медленно. Полученный световой будильник может, например, изменять яркость от абсолютной темноты до максимального значения в течение 30 минут. Если это начать примерно за полчаса до фактического времени пробуждения, это приведет к очень расслабленному пробуждению. Подсознательно этот процесс воспринимается как виртуальный рассвет, и фаза глубокого сна мягко завершается. В этом контексте часто звучит понятие биологически оптимизированного пробуждения.

Для истинного аналогового напряжения колебание «прямоугольного сигнала», которое возникает во время ШИМ, все равно должно быть отфильтровано электронным способом. Однако такое усреднение сигнала происходит на глазах у наблюдателя при использовании светодиодов. Быстрое переключение светодиода становится размытым от определенной частоты и далее до непрерывного впечатления яркости. Тридцать минут соответствуют 1800 секундам. При полном разрешении ШИМ 1024 это дает значение $1800/1024 = 1,75$ секунды на шаг. Таким образом, следующая программа дает желаемый результат:

```

# Sunrise.py
import machine
from time import sleep

LED = machine.Pin(25)

# create the PWM object
pwmLED = machine.PWM(LED)

timedelay = 1.75
pwmLED.freq(5000)

while True:
    for dc in range (0, 1023):
        pwmLED.duty(dc)

```

```
pwmLED
sleep(timedelay)
```

7.4 Mood-Light с многоцветным светодиодом

ШИМ-управление также идеально подходит для управления так называемыми неподвижными светильниками. В разделе 6.3 была представлена цифровая радуга, в которой красный, зеленый и синий цвета загорались один за другим с помощью RGB-светодиода. Однако при непрерывных переходах это впечатление можно многократно улучшить. Устройства этого типа также доступны в различных версиях в виде «светильников настроения». С многоцветным светодиодом можно легко воспроизвести такое освещение. Здесь светодиод управляется так, что он непрерывно меняет цвет. При использовании ШИМ-управления цветовые переходы становятся не резкими, а плавными. Кроме того, появляются не только отдельные основные цвета, но и полный цветовой спектр RGB-светодиода.

Соответствующий скетч обеспечивает медленное увеличение и уменьшение всех аналоговых выходов один за другим. Аддитивное смешивание цветов приводит к широкому спектру цветовых нюансов, включая почти белые оттенки.

Светодиод RGB снова подключается к пинам 25, 26 и 32 (см. рис.6.3). Программа для этого выглядит так:

```
# MulticolorLED.py
import machine
from time import sleep

# create the PWM objects
LEDred = machine.Pin(25)
LEDgreen = machine.Pin(26)
LEDblue = machine.Pin(32)

# create the PWM objects
pwmLEDred = machine.PWM(LEDred)
pwmLEDgreen = machine.PWM(LEDgreen)
pwmLEDblue = machine.PWM(LEDblue)

timedelay = 0.001
pwmLEDred.freq(100)
pwmLEDgreen.freq(100)
pwmLEDblue.freq(100)

while True:
    for dc in range (0, 1023):
        pwmLEDblue.duty(dc)
        pwmLEDblue
        sleep(timedelay)
```

```
for dc in range (0, 1023):
    pwmLEDgreen.duty(dc)
    pwmLEDgreen
    sleep(timedelay)

for dc in range (0, 1023):
    pwmLEDred.duty(dc)
    pwmLEDred
    sleep(timedelay)

for dc in range (1023, 0, -1):
    pwmLEDgreen.duty(dc)
    pwmLEDgreen
    sleep(timedelay)

for dc in range (1023, 0, -1):
    pwmLEDblue.duty(dc)
    pwmLEDblue
    sleep(timedelay)

for dc in range (1023, 0, -1):
    pwmLEDred.duty(dc)
    pwmLEDred
    sleep(timedelay)
```

7.5 Чистота и плавность: аналоговые значения от ЦАП

Контроллеры ESP32 имеют два цифро-аналоговых преобразователя (ЦАП) с 8-битным разрешением каждый. В отличие от ШИМ, сигналы выводятся как реальные, стабильные значения, а не как действующие значения определенного отношения импульс-пауза.

Таким образом можно генерировать калибровочные сигналы, точные элементы управления или высококачественные звуковые сигналы. Это означает, что контроллеры ESP уже достигли областей применения простых цифровых сигнальных процессоров (DSP).

ЦАП ESP32 доступны на GPIO25 (канал 1) и GPIO26 (канал 2). Следующий код Python выводит фиксированное аналоговое напряжение на пине 25:

```
# DAC_test.py
# 8-битный диапазон ЦАП: 0-255 - выходное напряжение: 0-3,3 В
from machine import Pin, DAC
DACValue_0=125
dac0=DAC(Pin(25))
dac0.write(DACValue_0) # output 1.62V
```

Значение напряжения вычисляется из

$$U = 3.3 \text{ V} / 255 * \text{DAC_val}$$

Например, значение `DAC_val = 125` дает следующее:

$$U = 3.3 \text{ V} / 255 * 125 = 1.617 \text{ V}$$

С помощью точного цифрового вольтметра можно контролировать значение выходного напряжения:

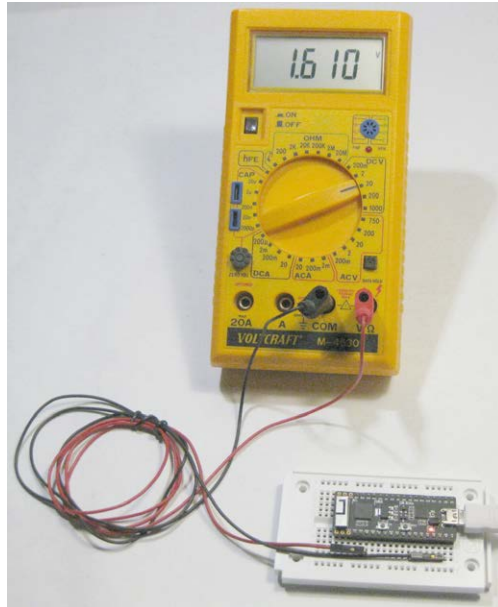


Рис.7.3: Аналоговый выход напряжения на цифровом мультиметре

При допустимой погрешности лучше 1% результат очень хороший.

7.6 Вывод зависимых от времени напряжений

В дополнение к значениям статического напряжения ЦАП ESP32 также могут генерировать кривые напряжения, зависящие от времени, с частотами до звукового диапазона. Для этого требуемые значения передаются на ЦАП в цикле `for`:

```
# sawtooth.py
#8-bit DAC range:0-255 output voltage:0-3.3V

from machine import Pin, DAC
from time import sleep_us

#define AnalogOut 25
dac=DAC(Pin(25))
```

```
while (True):
    for DACValue_0 in range(255):
        print(DACValue_0)
        dac0.write(DACValue_0)
        sleep_us(1)
```

В результате получается асимметричное напряжение с достаточно линейным ростом и возвратом практически к 0 вольт.

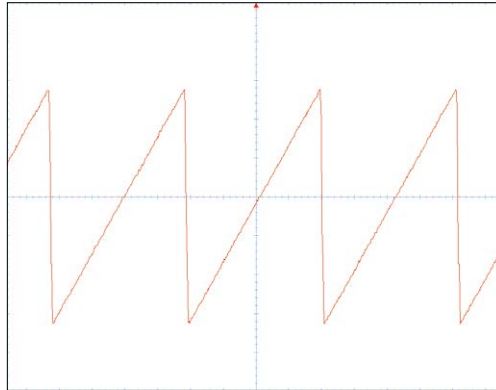


Рис.7.4: Пилообразное напряжение на осциллографе.

7.7 Для интересных кривых: генератор произвольных функций

Цифро-аналоговый преобразователь может без проблем генерировать еще более сложные кривые напряжения. Используя два цифро-аналоговых преобразователя в модуле ЦАП ESP32, можно одновременно генерировать прямоугольное напряжение и синусоидальную кривую напряжения. Для синусоидальной кривой напряжения таблица синусов хранится в массиве `sinus[]`. Эти значения массива затем передаются в ЦАП в цикле. С помощью следующего «если»-запроса

```
if buf[i] > amplitude:
    dac1.write(255)
else:
    dac1.write(0)
```

функция прямоугольника также может быть сгенерирована одновременно. Полная программа MicroPython для этого выглядит так:

```
# sinus_square_generator.py
from math import sin, pi
from machine import DAC
from machine import Pin
dac0 = DAC(Pin(25))
```

```

dac1 = DAC(Pin(26))

offset = 128
amplitude = 127
bufferlength = 100

# create a buffer containing a sine-wave
buf = bytearray(100)
for n in range(len(buf)):
    buf[n] = offset + int(amplitude * sin(2*pi*n/len(buf)))

while 1:
    for i in range(len(buf)):
        dac0.write(buf[i])
        if buf[i] > amplitude:
            dac1.write(255)
        else:
            dac1.write(0)

```

Кривые напряжения на пинах 25 и 26 снова можно посмотреть с помощью осциллографа:

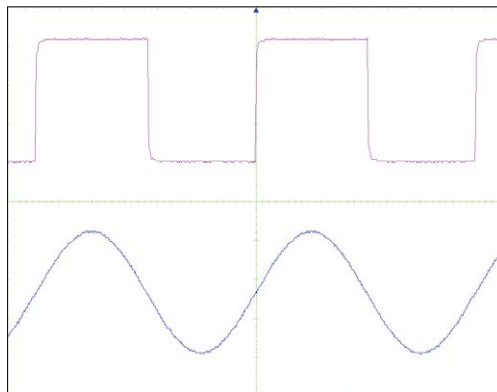


Рис.7.5: ESP32 как генератор синуса/прямоугольника.

ЦАП можно использовать для широкого спектра приложений, таких как аудиовыход, генерация звука или цифровые синтезаторы. Специальная конфигурация также могут быть сгенерирована таким образом:

```

# sinus_square_generator.py

from math import sin, pi
from machine import DAC
from machine import Pin

dac0 = DAC(Pin(25))

```

```
offset = 128
amplitude = 127
buffferlength = 100

# create a buffer containing a sine-wave
buf = bytearray(buffferlength)
for n in range(len(buf)):
    buf[n] = offset + int(amplitude * sin(2*pi*n/len(buf)))

# insert special values
buf[25] = offset
buf[75] = offset

while 1:
    for i in range(len(buf)):
        dac0.write(buf[i])
```

Программа выдает «возмущенный» синусоидальный сигнал, который кратковременно падает до 0 В при достижении пикового напряжения. Такой сигнал нужен, если вы хотите протестировать специальные усилители или фильтры. С помощью такого сигнала можно проверить, правильно ли схемы реагируют на эти формы сигналов.

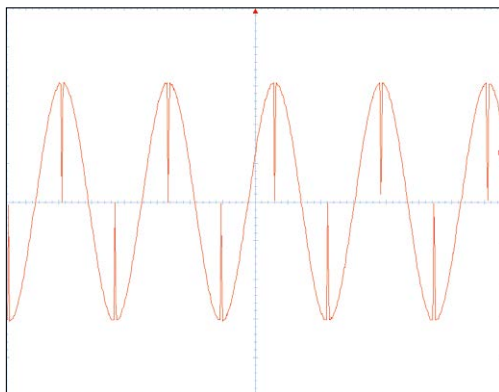


Рис. 7. 6: Искаженный синусоидальный сигнал.

Глава 8 • Прерывания и таймеры

В среде МК временные прерывания работающей программы называются прерываниями. Они используются для выполнения часто критического процесса. Типичными приложениями являются аварийные сообщения или защитное отключение компонентов и частей машины. Если, например, генератор контролируется электронным способом, на сигнал превышения температуры от датчика необходимо реагировать немедленно. Временные задержки из-за мгновенной высокой загрузки процессора могут иметь в этом случае катастрофические последствия и поэтому недопустимы.

Прерывания также необходимы для использования таймеров. Здесь аппаратные блоки в контроллере устанавливают определенные временные интервалы. Если они истекли, запускается прерывание. Это позволяет достичь очень точной синхронизации независимо от нагрузки на ядро основного процессора. Типичными приложениями являются часы и все виды контроля времени. Точность таймеров в конечном счете зависит только от точности кварца процессора. Поскольку точность в диапазоне 1/1000 может быть достигнута без проблем, таймеры используются во многих практических приложениях.

8.1 Требуется нарушение: прерывания

Прерывания используются, когда необходимо быстро реагировать на непредвиденные или нестандартные события. Если изменение состояния обнаружено на прерываемом выводе, отдельный аппаратный блок в контроллере запускает событие прерывания. Таким образом, нет необходимости постоянно контролировать соответствующий вывод с помощью процедуры запроса. После запуска прерывания может быть вызвана предопределенная функция, которая соответствующим образом реагирует на событие.

Когда происходит прерывание, процессор прекращает выполнение основной программы, чтобы выполнить процедуру обработки прерывания. Только когда это будет выполнено, он вернется в основную программу. Следующий рисунок иллюстрирует этот процесс:

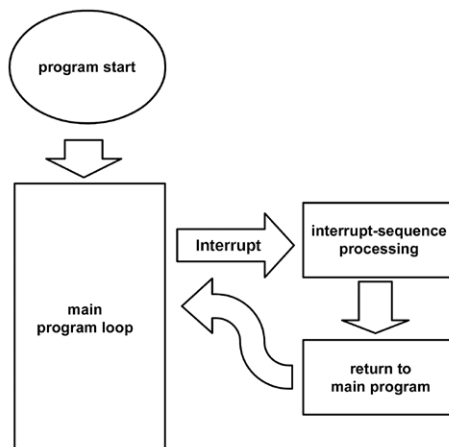


Рис.8.1: Прерывания.

Таким образом, процедуры прерывания выполняются квазипараллельно с основной программой. Но это не настоящая параллельная обработка. Процессор скорее обрабатывает основную программу и программу прерывания одну за другой.

Поэтому процедуры прерывания должны быть как можно короче, т. е. их обработка должна занимать как можно меньше времени. В подпрограмме обработки прерывания не должны выполняться трудоемкие вычисления или выполняться обширные программные циклы и т. д.

8.2 Автоматический ночной свет

В этом разделе объясняется, как настроить прерывания в MicroPython. Например, считывается кнопка. Эта кнопка предназначена для включения светодиода на определенный период времени. После этого светодиод должен автоматически погаснуть. Применением этой схемы может быть, например, автоматическое освещение ванной комнаты или лестничной клетки. Но такие схемы также используются для внутреннего освещения в транспортных средствах. Благодаря яркому белому светодиоду плата ESP также может использоваться в качестве автоматической прикроватной лампы. На рис.8.2 показана принципиальная схема.

Для получения дополнительной информации об аппаратных таймерах ESP32 обратитесь к следующей главе.

Рис.8.2: Схема автоматического ночного освещения.

Для использования прерываний машинный модуль сначала снова импортируется, а затем используется для доступа к аппаратным функциям:

```
from machine import Pin
```

Кроме того, методы сна необходимы, потому что позже в программе требуется период ожидания:

```
from time import sleep
```

Переменная используется для запроса того, была ли нажата клавиша. Эта переменная определена глобально, потому что она модифицируется в функции обработки прерывания. Достаточно, если она может принимать логические значения True или False. Инициализация выполняется с False:

```
key_pressed = False
```

В случае прерывания должна быть запущена следующая процедура:

```
def interrupt_handler(pin):
    global key_pressed
    key_pressed = True
```

Эта функция вызывается каждый раз при нажатии клавиши. Функция `interrupt_handle` имеет входной параметр (`pin`), в который передается объект класса `Pin`, если происходит прерывание. Параметр указывает, на каком выводе сработало прерывание. Если используется только один вывод прерывания, эта информация не нужна. Однако, если несколько прерываний могут запускать одну и ту же функцию обработки прерываний, может представлять интерес, какой GPIO инициировал прерывание.

В примере `theinterrupt_handler` изменяет переменную `key_pressed` на `True`. Поскольку время обработки функции прерывания должно быть как можно короче, следует по возможности избегать таких функций, как `sleep()` или `print()`. Следовательно, инструкции, которые должны выполняться при возникновении прерывания, должны находиться в основной программе.

Чтобы переменную можно было использовать как внутри функции, так и во всем коде, ее необходимо объявить «глобальной». В противном случае переключение светодиода не сработало бы, поскольку изменение переменной имело бы эффект только внутри функции, но не в основной программе.

Написав

```
led = Pin(25, Pin.OUT)
button = Pin(4, Pin.IN)
```

пины для светодиода и кнопки определены. Инструкция

```
button.irq(trigger=Pin.IRQ_RISING, handler=handle_interrupt)
```

вызывает срабатывание прерывания всякий раз, когда нажимается клавиша. Параметр «триггер» может принимать следующие значения:

- `IRQ_RISING`: Trigger on rising edge
- `IRQ_FALLING`: Trigger on falling edge

Если прерывание срабатывает, контроллер переходит к процедуре обработки прерывания, а для переменной `key_pressed` устанавливается значение `True`.

После возврата в основную программу отображается сообщение **"Interrupt detected! - Обнаружено прерывание!"**. Кроме того, сообщается, на каком выводе сработало прерывание. Затем светодиод включается на указанный период времени, а затем снова выключается. Наконец, `key_pressed` снова устанавливается в `False`, и контроллер доступен для следующего прерывания. Полная программа выглядит так:

```
# LED_interrupt.py

from machine import Pin
from time import sleep

timeOn=3
key_pressed=False

def handle_interrupt(pin):
    global key_pressed
    key_pressed=True

led = Pin(25, Pin.OUT)
button1 = Pin(4, Pin.IN)

button1.irq(trigger=Pin.IRQ_RISING, handler=handle_interrupt)

while True:
    if key_pressed:
        print('Interrupt detected')
        led.value(1)
        sleep(timeOn)
        led.value(0)
        key_pressed = False
```

В этом примере используется уже известная команда `sleep()`, чтобы светодиод оставался включенным в течение трех секунд. На данный момент, однако, приложение таймера было бы лучшим решением. Как и почему используются таймеры, показано в следующем разделе.

8.3 Мастера времени: таймеры

Таймеры можно использовать для различных задач. Среди наиболее распространенных приложений

- периодический вызов функции
- подсчет событий
- генерация сигналов ШИМ

Каждый таймер состоит из двух 16-битных каналов, которые можно связать в 32-битный таймер. Режим работы должен быть настроен для каждого таймера, но период или частота могут быть выбраны независимо для каждого канала. С помощью метода обратного вызова событие таймера может вызвать функцию python. Таким образом, следующая программа заставляет светодиод мигать:

```
# LED_timer.py

import machine

led = machine.Pin(25, machine.Pin.OUT)
timer = machine.Timer(0)
```

```
def handleInterrupt(timer):
    led.value(not led.value())

timer.init(period=1000, mode=machine.Timer.PERIODIC,
callback=handleInterrupt)
```

Отличие от классической программы с функцией `sleep()` в том, что контроллер не блокируется использованием функции таймера. Во время выполнения функции `sleep()` ядро процессора занято только подсчетом тактов. Ядро не может выполнять никаких других задач, помимо этого. Таким образом, функция `sleep()` является "обструктивной". С другой стороны, для таймеров доступны отдельные аппаратные блоки, не нагружающие ядро основного процессора.

О том, что ядро процессора остается свободным для других задач при использовании прерывания, свидетельствует и появление командной строки `>>>` в оболочке. Следовательно, вы можете продолжать работать здесь одновременно:

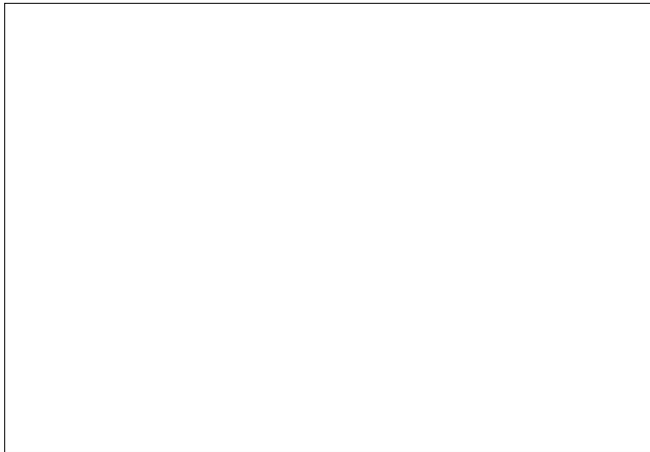


Рис.8.3: Таймер работы и активная оболочка.

Другие функции также могут выполняться в программе, пока таймер активен. Следующий код приводит к тому, что `led1` на контакте 25 мигает с периодом 125 мс, в то время как `led2` на порте 26 мигает каждую секунду:

```
# LED_timer_and_mainprogram.py

from machine import Pin, Timer
from time import sleep

led1 = Pin(25, Pin.OUT)
led2 = Pin(26, Pin.OUT)
timer = Timer(0)
```

```
def handleInterrupt(timer):
    led1.value(not led1.value())

timer.init(period=125, mode=Timer.PERIODIC, callback=handleInterrupt)

while True:
    led2.value(1)
    sleep(.1)
    led2.value(0)
    sleep(1))
```

Второй светодиод должен быть подключен к порту 26 с последовательным резистором.

Наконец, следующая программа показывает, что два светодиода с совершенно разными частотами можно переключать с помощью двух таймеров:

```
# LED_double_timer.py

import machine

led1 = machine.Pin(25, machine.Pin.OUT)
led2 = machine.Pin(26, machine.Pin.OUT)

timer1 = machine.Timer(0)
timer2 = machine.Timer(1)

def handleInterrupt1(timer1):
    led1.value(not led1.value())

def handleInterrupt2(timer2):
    led2.value(not led2.value())

timer1.init(period=125, mode=machine.Timer.PERIODIC,
callback=handleInterrupt1)

timer2.init(period=517, mode=machine.Timer.PERIODIC,
callback=handleInterrupt2)
```

Без прерываний и таймеров эта относительно простая задача была бы невыполнима.

8.4 Многофункциональный проблесковый маячок

Красные велосипедные фонари сегодня оснащены несколькими мощными светодиодами. Кроме того, в дополнение к непрерывному световому режиму часто можно установить различные схемы сигналов. Таким образом можно привлечь повышенное внимание. Мигающие задние фонари, постоянно установленные на велосипеде, незаконны, но ношение дополнительных фонарей на шлеме или рюкзаке разрешено при определенных обстоятельствах. Однако, прежде чем использовать «мигалку» на дороге, всегда следует ознакомиться с текущей правовой ситуацией.

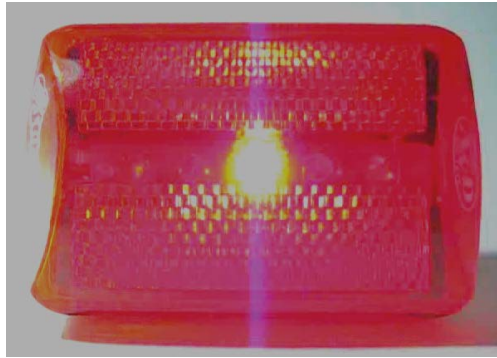


Рис.8.4: Светодиодный велосипедный задний фонарь.

С контроллером ESP можно легко воспроизвести многофункциональную мигающую лампочку. Запроектированный задний фонарь должен иметь следующие эксплуатационные характеристики:

- 5 LED
- Постоянный свет
- Мерцание
- Мигание
- Эффект погони

Переключение между отдельными рабочими состояниями является классическим применением прерывания. Без использования технологии прерывания эффективное управление переключением было бы невозможно.

Следующая программа выполняет эту работу:

```
# bike light.py

from machine import Pin
from time import sleep

LED = [25,26,32,27,14]
del_time = 0.1
mode=1

def handle_interrupt(button1):
    global mode
    mode+=1
    if(mode>4):
        mode=1
    print(mode)

button1 = Pin(4, Pin.IN)
button1.irq(trigger=Pin.IRQ_RISING, handler=handle_interrupt)
```

```

for n in range(len(LED)):
    LED[n] = Pin(LED[n],Pin.OUT)

while True:
    if (mode==1):      # все светодиоды горят
        for n in range(len(LED)):
            LED[n].value(1)
    if (mode==2):      # светодиоды мерцают
        for n in range(len(LED)):
            LED[n].value(1)
        sleep(del_time)
        for n in range(len(LED)):
            LED[n].value(0)
        sleep(del_time)
    if (mode==3):      # светодиоды мигают
        for n in range(len(LED)):
            LED[n].value(1)
        sleep(del_time)
        for n in range(len(LED)):
            LED[n].value(0)
        sleep(del_time*5)
    if (mode==4):      # LED вращаются (эффект погони)
        for n in range(len(LED)):
            LED[n].value(1)
            sleep(del_time)
            LED[n].value(0)
            sleep(del_time)

```

Поскольку, в отличие от простого чейзера из раздела 6.2, теперь одновременно активны несколько светодиодов, аппаратную схему необходимо несколько изменить:

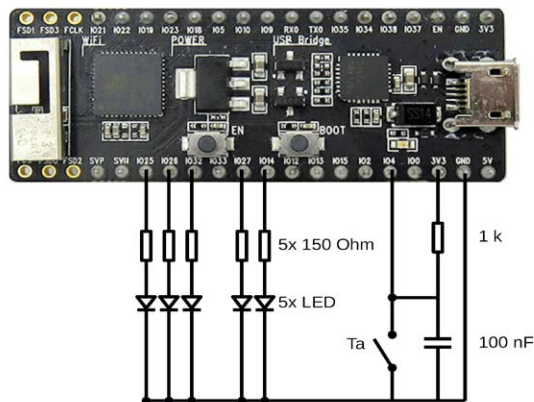


Рис.8.5: Принципиальная схема светодиодного заднего фонаря.

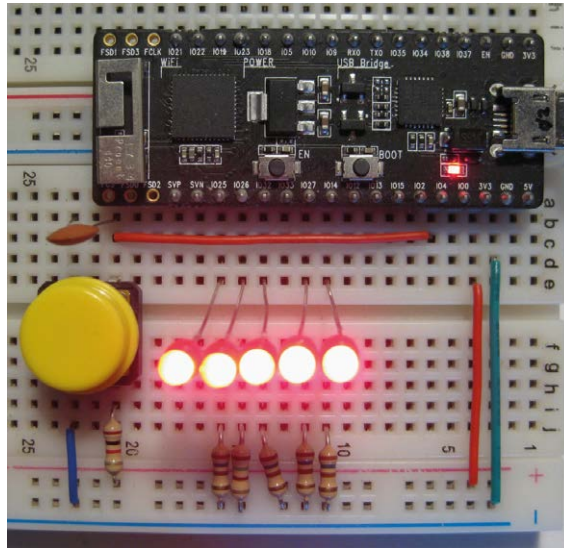


Рис.8.6: Макет светодиодного заднего фонаря.

Для переключения между режимами работы добавлена кнопка на ножке 4, включающая подтягивающий резистор и конденсатор для устранения дребезга. Кроме того, каждый светодиод теперь имеет свой добавочный резистор. При использовании общего резистора отдельные светодиоды будут светиться все слабее и слабее при включении каждого дополнительного светодиода.

Глава 9 • Использование датчиков

Оценка датчиков и электронных измерительных преобразователей является центральной задачей технологии микроконтроллеров. Поэтому в этой главе более подробно рассматривается использование MicroPython в измерительных и сенсорных технологиях. ESP32 имеет восемнадцать аналоговых входов и поэтому идеально подходит для измерительных приложений.

В дополнение к оценке сигналов датчиков сенсорная технология также включает в себя вывод и обработку измеренных значений и обработку сигналов всех видов. В частности, выдающуюся роль играет то, что называется «обработкой сигнала». Такие методы, как

- преобразование сигнала
- линеаризация
- усиление сигнала
- фильтрация

являются классическими задачами сбора данных на основе микроконтроллеров. Без этих методов многие задачи и процедуры в технике, прикладной физике или медицине были бы нереализуемы.

С MicroPython измеренные значения могут отображаться непосредственно на консоли. У Тонни даже есть специальный плоттер, позволяющий графически отображать значения. В качестве альтернативы также можно отображать значения на дисплее, так что ESP32 также может работать в автономном режиме. Второй метод более подробно описан в следующей главе.

9.1 Сбор данных измерений и датчиков

Даже с цифровыми контактами ввода-вывода уже можно считывать измеренные значения. Таким образом, двоичные уровни напряжения могут быть считаны как 0 В или 3,3 В «измеренные значения» с каждого входа GPIO контроллера ESP. Часто этого достаточно для простых задач измерения или мониторинга.

С другой стороны, реальные аналоговые, т. е. непрерывные, значения невозможно измерить с помощью цифровых портов. Как уже упоминалось в разделе о цифро-аналоговом преобразовании, природа преимущественно аналоговая. Поэтому были разработаны специальные методы и процедуры для регистрации сигналов в аналоговой области. В ESP32 для этой цели доступны так называемые аналого-цифровые преобразователи (АЦП). Они являются, так сказать, аналогом ЦАП и позволяют одновременно захватывать несколько аналоговых каналов.

АЦП стали неотъемлемой частью современного сбора сигналов. Во всех областях техники они следят за правильным функционированием компонентов и систем. В аудио- и коммуникационных устройствах, таких как смартфоны или планшеты, они преобразуют сигналы микрофона в высококачественные потоки цифровых данных. Температура, интенсивность света или давление — это лишь несколько примеров многих параметров, которые регистрируются и оцениваются АЦП в транспортных средствах или зданиях.

Преобразователи можно охарактеризовать двумя основными параметрами:

- разрешение в битах
- скорость преобразования

Точность, с которой аналоговый сигнал может быть оцифрован, определяется первым значением. Время преобразования имеет решающее значение для рабочей скорости. Оно определяет максимальную частоту, с которой сигнал может быть квантован. Это время преобразования зависит главным образом от используемого метода преобразования.

На рис.9.1 показана оцифровка аналоговой кривой напряжения с помощью АЦП. Преобразователь сканирует аналоговую кривую напряжения в определенные моменты времени и преобразует полученное значение в цифровую величину. АЦП в чипе ESP32 имеет разрешение 12 бит. Например, при нормализации до 3,3 В напряжение 2,45 В дает:

$$Q = 2,45 \text{ В} / 3,3 \text{ В} * 4095 = 3040 \text{ бит}$$

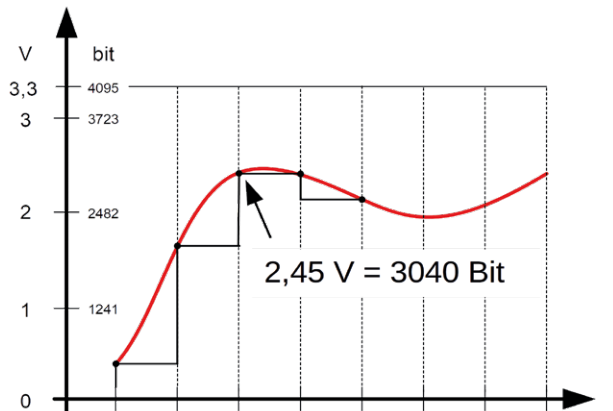


Рис.9.1: Оцифровка аналоговых значений.

Широкая область применения цифровой измерительной техники привела к развитию различных процессов АЦП. Некоторые области применения требуют максимально возможной точности измерения. Другие процессы полагаются на максимально быстрое преобразование. Поскольку обе функции не могут быть преобразованы одновременно, было разработано несколько принципов преобразования АЦП, которые явно различаются по точности и скорости. Следующая таблица суммирует их:

Процедура измерения	Приложение	Характеристики
Параллельный преобразователь	быстродействующие цифровые осциллографы, техника управления	очень быстрый, с высоким энергопотреблением
Последовательное приближение (SAR)	Процедура для большинства внутренних AD преобразователи МК, стандартный метод	быстрый, высокоточный, сложный
Односкатный или двухскатный	Мультиметр	низкая стоимость, хорошая линейность, медленный
Дельта-Сигма	Прецизионные измерения, аудиотехника	недорогой, медленный

В качестве хорошего компромисса последовательное приближение в основном используется в технологии микроконтроллеров. Два преобразователя в ESP32 также основаны на этом методе. Аналоговый мультиплексор гарантирует, что микросхема ESP способна записывать 18 аналоговых каналов со скоростью измерения несколько тысяч измерений в секунду. Достижимая скорость преобразования зависит, среди прочего, от выбранного фактического разрешения АЦП. Дополнительную информацию можно найти в следующих разделах.

9.2 Точная регистрация напряжения: самодельный вольтметр

В первом примерном приложении должна быть продемонстрирована оценка аналогово-цифровых преобразователей в среде Micro-Python. Для этого аналоговое напряжение, генерируемое потенциометром, записывается и отображается на консоли. Выводы 25-34 следует использовать как универсальные входы АЦП. Другим входам АЦП назначены важные двойные функции, поэтому их следует использовать только в том случае, если другие каналы уже заняты.

В следующем примере используется аналоговый вход 34. Следующая программа показывает самый простой способ считывания АЦП ESP32:

```
# ADC_atten_11db.py

from machine import ADC, Pin
from time import sleep

adc = ADC(Pin(34))    # инициировать АЦП на пине ADC
adc.atten(ADC.ATTN_11DB)

while True:
    print(adc.read())  # прочитанное значение, 0-1024
    sleep(1)
```

С аппаратной стороны к контакту 34 необходимо подключить только потенциометр на 10 кОм:

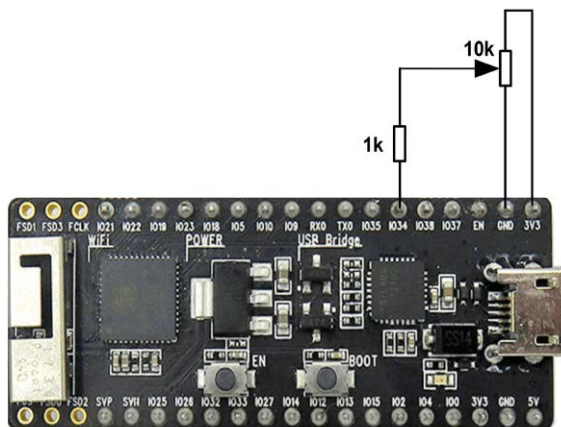


Рис.9.2: Измерение напряжения с помощью потенциометра.

Дополнительный резистор 1 кОм используется только для защиты чипа ESP. В случае неисправности максимальный ток ограничивается, если напряжение, подаваемое на потенциометр, слишком велико. Таким образом можно избежать повреждения чипа.

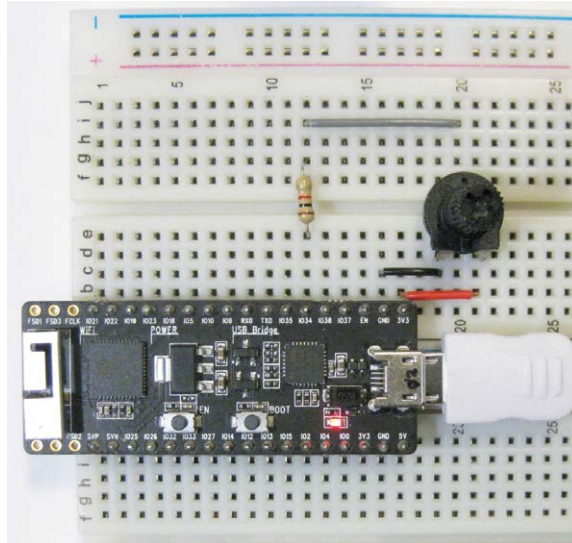


Рис.9.3: Макет для записи аналоговых измеренных значений.

Значения от 0 до 4095 выводятся для различных настроек напряжения на потенциометре. Это соответствует разрешению 12-разрядного АЦП: $2^{12}-1 = 4095$, но если вы измерите напряжения внешним мультиметром, вы обнаружите, что значение 4095 уже достигается при напряжении 1 вольт.

Таким образом, при использовании стандартной конфигурации входные напряжения на выводе ADC должны находиться в диапазоне от 0,0 В до 1,0 В. Для всех напряжений выше 1,0 В отображается значение 4095. Чтобы увеличить этот полезный диапазон напряжения, можно активировать внутренний делитель напряжения.

Использование команд

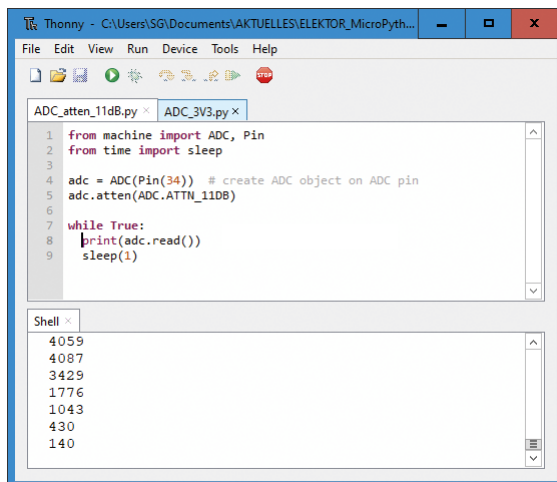
	Демпфир. (дБ)	Демпфиров. (лин.)	Ui max
dc.atten(ATTN_0DB)	0 dB	1	1.00 V
adc.atten(ATTN_2_5DB)	2.5 dB	0.75	1.34 V
adc.atten(ATTN_6DB)	6 dB	0.5	2 V
adc.atten(ATTN_11DB)	11 dB	0.282	3.6 V

можно активировать внутреннее демпфирование. Затухание 0 дБ является стандартной настройкой. Разрешение также можно настроить. Возможные варианты:

ADC.WIDTH_9BIT:	9-bit data
ADC.WIDTH_10BIT:	10-bit data
ADC.WIDTH_11BIT:	11-bit data
ADC.WIDTH_12BIT:	12-bit data (standard configuration)

Важное примечание: абсолютное максимальное номинальное напряжение для входных контактов составляет 3,6 В. Более высокие напряжения могут повредить ESP32!

После загрузки программы измеренные значения выводятся в консоль:



The screenshot shows the Thonny IDE window. The top menu bar includes File, Edit, View, Run, Device, Tools, and Help. Below the menu is a toolbar with icons for file operations, running, and stopping. Two tabs are open: 'ADC_atten_11dB.py' and 'ADC_3V3.py'. The 'ADC_atten_11dB.py' tab is active, displaying the following Python code:

```
1 from machine import ADC, Pin
2 from time import sleep
3
4 adc = ADC(Pin(34)) # create ADC object on ADC pin
5 adc.atten(ADC.ATTN_11DB)
6
7 while True:
8     print(adc.read())
9     sleep(1)
```

Below the code editor is a 'Shell' window showing the output of the program:

```
4059
4087
3429
1776
1043
430
140
```

Рис.9.4: Отображение измеренных значений в консоли Thonny.

При высоких скоростях измерения отдельные значения становятся трудно читаемыми. В этом случае функция плоттера Thonny-IDE является средством выбора. С использованием

View → Plotter

измеренные значения могут отображаться графически.

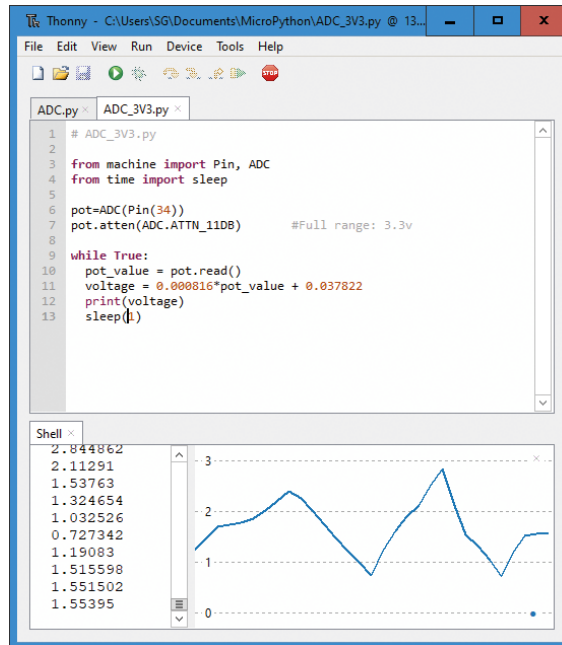


Рис.9.5: Графическое представление в плоттере.

Эта функция идеально подходит для быстрого и четкого представления обширных данных измерений, результатов или значений датчиков и т. д.

9.3 Коррекция линейности

Преобразование АЦП контроллера ESP32 имеет тот недостаток, что результирующая характеристическая кривая является нелинейной. При значениях ADC выше 3000 отчетливо видно отклонение от идеальной характеристической кривой.

Значительные отклонения возникают и вблизи нулевой точки (см. рис.9.7). В результате возникают большие ошибки измерения, особенно на краях. Тем не менее, их можно значительно уменьшить с помощью подходящих мер, таких как линеаризация с помощью полинома компенсации или ограничение диапазона измерения.

Нелинейность АЦП можно легко отобразить графически с помощью (линейного) ЦАП. Следующая программа предоставляет соответствующие данные измерений:

```

# ADC_DAC_tst.py

from machine import DAC, ADC, Pin
import time

dac0=DAC(Pin(25))
adc0=ADC(Pin(34))
adc0.atten(ADC.ATTN_11DB)

```

```

for n in range(0, 256):
    print(n, end = ' ',)
    dac0.write(n)
    time.sleep(0.1)
    print adc0.read()
    time.sleep(0.1)

```

Полученные таким образом значения можно отобразить в виде слайд-диаграммы с помощью Excel или Libre-Office:

Рис.9.6: Нелинейная характеристика АЦП.

9.4 Линеаризация путем ограничения диапазона значений

Полином компенсации позволяет линеаризовать значения во всем диапазоне измерения. Однако при этом невозможно добиться очень высокой точности. Лучше использовать только преимущественно линейные участки характеристической кривой. Из рис.9.6 видно, что АЦП работает в основном линейно в диапазоне до прибл. 3000 отсчетов. Если ограничиться этим диапазоном, получится калибровочная линия, показанная на рис. 9.7. С полученной из него формулой регрессии:

$$\text{Напряжение} = 0.000816 * \text{ADC_count} + 0.037822$$

значения напряжения от 200 мВ до 2,5 вольт теперь могут быть измерены очень точно. Обычно этого вполне достаточно для сенсорных приложений, так как многие датчики в любом случае не достигают значений менее 0,2 вольта. Программа измерения для этого выглядит следующим образом:

```

# ADC_lin.py

from machine import Pin, ADC
from time import sleep
pot = ADC(Pin(34))
pot.atten(ADC.ATTN_11DB) #Full range: 3.3v

while True:
    pot_value = pot.read()
    voltage = 0.000816*pot_value + 0.037822

```

```
print(voltage)
sleep(0.1)
```

С помощью высококачественного и точно откалиброванного вольтметра значения можно измерить повторно. Отклонения должны оставаться значительно ниже 3%. Это означает, что ничто не препятствует использованию аналоговых измерительных преобразователей, таких как фотодиоды, аналоговые датчики температуры или тензометрические датчики.

Для еще более высоких требований также могут использоваться цифровые преобразователи. ESP32 также идеально подходит для этого. Эти датчики могут напрямую связываться с процессором через подходящие системы шин. Более подробную информацию можно найти, начиная с Раздела 9.10.

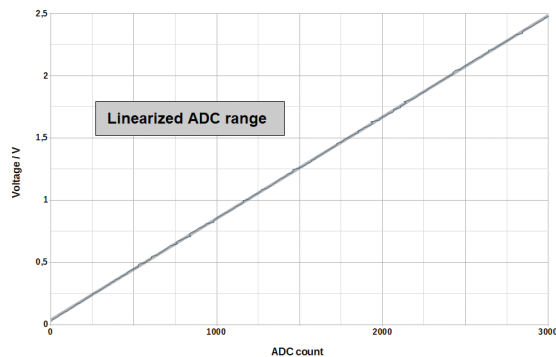


Рис.9.7: Линеаризованная характеристика АЦП.

Это означает, что полный диапазон напряжений не может использоваться для требований высокой точности. Значения напряжения должны быть соответственно ограничены. Другой возможностью является корректировка измеренных значений с помощью программного обеспечения с помощью полинома. Для этой цели доступно несколько методов. В следующем разделе более подробно объясняется стандартный метод линеаризации передаточных функций.

9.5 Линеаризация входа АЦП с помощью компенсационного полинома

Если необходимо использовать весь диапазон значений внутреннего аналого-цифрового преобразователя ESP32, требуется программная коррекция. Это делается путем использования точек интерполяции, отличных от фактической кривой измерения. Они используются для создания так называемого «многочлена кривой компенсации». Затем значения АЦП можно скорректировать с помощью параметров полинома. Определенный таким образом полином компенсации равен

$$\begin{aligned}
 y = & -0.000000000009824x^3 + \\
 & 0.000000016557283x^2 + \\
 & 0.000854596860691x + \\
 & 0.065440348345433
 \end{aligned}$$

Для этого в MicroPython реализована функция:

```
# ADC_lin_poly.py

from machine import ADC, Pin
from time import sleep

adc = ADC(Pin(34)) # инициализация АЦП на 34 пине
adc.atten(ADC.ATTN_11DB)

def ReadVoltage():
    ADC=adc.read()
    return -0.000000000009824*pow(ADC,3)+0.000000016557283*pow(ADC,
C,2)+0.000854596860691*ADC+0.065440348345433;

while True:
    print(adc.read(), " ", end="")
    print(ReadVoltage())
    sleep(0.1)
}
```

С помощью этого метода достигается хорошая линейность и точность. Теперь отклонения остаются ниже 60 мВ во всем диапазоне измерения. Этого вполне достаточно для большинства практических приложений. Для еще большей точности можно также использовать полином 4-й степени. Однако тогда уже будут заметны другие воздействия, такие как шум оцифровки, так что усилия вряд ли окупятся.

Дополнительные улучшения теперь могут быть сделаны путем дальнейшего ограничения диапазона напряжения. Однако это не обязательно для следующих приложений.

9.6 Измерение напряжения

На вход АЦП чипа ESP32 можно подавать только напряжение 3,3 В или меньше. Более высокие напряжения могут привести к разрушению всего контроллера. Однако с помощью делителя напряжения диапазон измерения можно расширить почти до бесконечности. На следующем рисунке показана схема измерения входного напряжения до прикл. 30 В. С делителем напряжения, состоящим из резистора 100 кОм и резистора 10 кОм, входное напряжение уменьшается до 1/11 его значения:

$$U_0 = R_1 / (R_1 + R_2) = 10 \text{ kohm} / 110 \text{ kohm} = 1/11$$

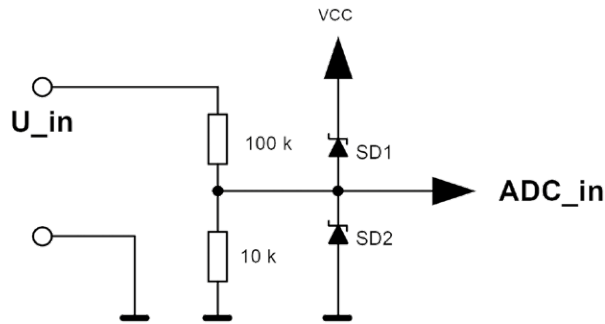


Рис.9.8: Делитель напряжения для расширения диапазона измерения АЦП.

ВАЖНО: Перед подачей внешнего напряжения необходимо тщательно проверить конструкцию. Если, например, перепутать резисторы, это может привести к разрушению платы ESP!

В электронных схемах и системах напряжения питания обычно ниже 30 В. Таким образом, диапазон измерения схемы, показанной на рис. 9.8, достаточен для многих приложений. В любом случае напряжение выше 50 В разрешается измерять только с помощью проверенных на безопасность измерительных приборов. В этом контексте всегда необходимо соблюдать следующее примечание:

Напряжение выше 50 В при особых обстоятельствах может быть опасным для жизни!

Таким образом, при использовании предделителя фактическое входное напряжение 30 В снижается до 2,73 В на входе АЦП. Это означает, что внутренний АЦП ESP32 теперь также может измерять напряжения в расширенном диапазоне до 30 В.

Диоды Шоттки (SD1 и SD2), показанные на принципиальной схеме, не требуются для фактического измерения. Они только защищают входы микросхемы ESP от перегрузки в случае ошибки. Как только напряжение на входе контроллера падает ниже 0 В или превышает 3,3 В, диоды становятся проводящими и тем самым предотвращают возможные перенапряжения на АЦП.

Влияние делителя напряжения, конечно же, должно быть учтено в программном обеспечении. За это отвечает калибровочный коэффициент `cal`. Он используется для расчета коэффициента масштабирования делителя напряжения.

```
# Voltmeter_30V.py

from machine import ADC, Pin
from time import sleep

adc = ADC(Pin(34)) # Инициировать АЦП на 34 пине
adc atten(ADC.ATTN_11DB)
```

```
Vref=3.30      # внутреннее опорное напряжение
R1=100         # для делителя напряжения
R2=10         # для деления напряжения
cal=1*(R1+R2)/R2 # калибровочный коэффициент
```

```
while True:
    output="V = {:.1f} V"
    print(output.format(adc.read()*Vref/4095*cal))
    sleep(1)
```

Сравнение измеренных значений с качественно откалиброванным мультиметром выявляет уже упомянутую нелинейность ESP32-ADC. Ошибка может быть более одного вольта. Следующая программа обеспечивает гораздо более точные значения:

```
# Voltmeter_30V_lin.py

from machine import ADC, Pin
from time import sleep

adc = ADC(Pin(34)) # Инициировать АЦП на 34 пине
adc.atten(ADC.ATTN_11DB)

Vref=3.30      # внутреннее опорное напряжение
R1=100         # для делителя напряжения
R2=10         # для деления напряжения
cal=1*(R1+R2)/R2 # калибровочный коэффициент

def ReadVoltage():
    ADC=adc.read()
    return -0.00000000009824*pow(ADC,3)+0.000000016557283*pow(ADC,
C,2)+0.000854596860691*ADC+0.065440348345433;

while True:
    output="V = {:.1f} V"
    print(output.format(ReadVoltage()*cal))
    sleep(1)
```

Пошаговое сравнение с калиброванным настольным вольтметром дает следующую таблицу:

U_Reference/V	U_ESP32/V
0.201	0.22
0.496	0.49
1.007	1.01
2.005	1.99

3.007	3.02
5.022	5.05
10.055	10.09
15.013	15.12
20.039	20.04
25.019	25.06
30.018	30.16

Константы, используемые здесь в программе в виде коэффициента калибровки и эталонного напряжения, называются параметрами калибровки программного обеспечения. В качестве альтернативы аппаратная калибровка может быть выполнена с помощью прецизионного потенциометра или триммера. Триммеры, специально разработанные для данного применения, могут быть отрегулированы очень точно и имеют хорошую долговременную стабильность благодаря своей закрытой конструкции. При сравнении с калиброванным измерительным прибором триммер настраивается до тех пор, пока вольтметр ESP32 не покажет правильное значение. Тогда больше нет необходимости изменять параметры в программе. Таким образом, в особых условиях окружающей среды (например, при экстремальных температурах) повторная калибровка может быть выполнена без изменения программы.

Недостатком аппаратной калибровки является то, что требуются дополнительные компоненты. Потенциометры и особенно триммеры сравнительно дороги и всегда показывают некоторый дрейф. Воздействие окружающей среды приводит к изменению электрических параметров, что ухудшает долговременную стабильность измерительного устройства. Если для потенциометра выбрано общее сопротивление 1 кОм, измеренное значение может быть изменено примерно на 5%. Более высокие значения сопротивления обеспечивают более широкий диапазон регулировки, но также снижают точность регулировки.

9.7 Перекрестные помехи: побочные эффекты сенсорной техники

Измерение значений аналогового напряжения с помощью АЦП ESP-32 больше не должно быть проблемой. Это открывает путь для оценки датчиков с аналоговым выходным напряжением. С этими зондами или датчиками доступны компоненты, которые могут преобразовывать физические величины в электрические значения. Почти все физические величины могут быть записаны в электронном виде. В дополнение к уже упомянутым параметрам, таким как температура, влажность или интенсивность света, это также относится к другим важным переменным, таким как

- Громкость или интенсивность звука
- Механическая печать
- Радио, тепловое или инфракрасное излучение
- Уровни радиоактивного излучения
- Ускорения или механические силы
- Все типы магнитных и электрических полей
- Химические параметры, такие как значение pH или электропроводность растворов.
- и т.д.

Сотни различных измерительных трансформаторов установлены на химических заводах, самолетах, спутниках или современных автомобилях. Без качественных датчиков все эти технические средства были бы немыслимы.

В идеале датчики измеряют точно определенное конкретное значение. Это значение воспроизводимо и с максимально возможной точностью преобразуется в электрическую величину, такую как напряжение или значение сопротивления. Желательна линейная зависимость между измеряемой переменной и электрическим значением. Однако для микроконтроллеров, таких как ESP32, даже нелинейная передаточная функция не представляет проблемы, поскольку линеаризация может быть выполнена с помощью подходящих программных методов без особых усилий.

Так называемые «перекрестные помехи» устранить не так-то просто. Многие датчики реагируют не только на желаемую измеряемую переменную, но и на другие физические величины. Некоторые примеры:

- Температурная зависимость фотодатчиков
- Влияние вибраций на преобразователи звука
- Влияние влажности воздуха на датчики электрических полей
- Реакция газовых сенсоров на некоторые химические соединения
- Тепловые помехи датчикам влажности или давления

Устранение таких влияний может быть связано со значительными усилиями. Один из методов заключается в дополнительном и независимом измерении возмущающей переменной и основанной на этом математической коррекции измеренного значения. В любом случае, при использовании датчиков всегда следует проверять, не могут ли перекрестные чувствительности повлиять на точность измерения или исказить ее недопустимым образом.

9.8 Прикосновение разрешено: емкостные сенсорные датчики

Сенсорные датчики — это классика сенсорной техники. В 1970-х и 80-х годах они были обязательными для каждого телевизора и стереосистемы. Они заменили привычные кнопки управления и дали возможность менять программы или регулировать громкость просто на ощупь. Но и экраны современных смартфонов — это, в принципе, сенсорные датчики, которыми можно управлять без клавиатуры или механических элементов управления.

Обычный сенсорный датчик состоит из металлических электродов. Иногда они покрыты тонкой защитной поверхностью. Если к датчику прикоснуться пальцем, это можно определить по изменению емкости электрода.

Чтобы использовать сенсорные датчики ESP32, в машинном модуле должен быть загружен класс `TouchPad`. После этого доступен метод `TouchPad.read()`:

```
from machine import TouchPad, Pin
t=TouchPad(Pin(14))
t.read() # Возвращает меньшее число при касании
```

`TouchPad.read()` возвращает изменение относительной емкости. Без прикосновения к сенсорным поверхностям, подключенным к ESP, возвращаются большие числа, то есть обычно значения больше 800. При прикосновении к активному электроду значения падают ниже 200. Однако результаты являются относительными и могут различаться в зависимости от формы и размера электрода. В следующей таблице приведены некоторые сведения об ожидаемых значениях:

Неподключенный контакт:	> 1000
С подключенным электродом - без прикосновения:	800 to 1000
С прикосновением:	< 200

Таким образом, может потребоваться калибровка, прежде чем сенсорные панели можно будет использовать в конкретной прикладной среде.

ESP32 предоставляет десять входов сенсорной панели (площадок). Они расположены на контактах 0, 2, 4, 12, 13, 14, 15, 27, 32 и 33.

Для проверки работы сенсорных входов можно использовать следующую программу:

```
# touch_test.py

from machine import TouchPad, Pin
from time import sleep

from machine import TouchPad, Pin

t1 = TouchPad(Pin(2))
t2 = TouchPad(Pin(4))

while True:
    cap1 = t1.read()
    print(cap1, end=" ")

    cap2 = t2.read()
    print(cap2)

    sleep(1)
```

В качестве электродов можно использовать такие предметы, как чертежные кнопки или мелкие медные монеты. На следующем рисунке показана предполагаемая схема управления входами 2 и 4:

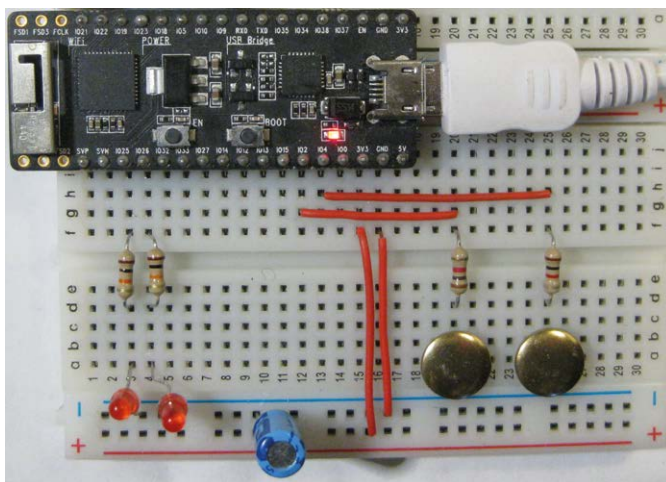


Рис.9.9: Датчики, подключенные к ESP32.

После загрузки программы выводятся значения датчиков. Если ни к одному из датчиков не прикасаться, числовые значения будут больше 800. Если слегка коснуться пальцем одного из электродов, значения соответствующего канала упадут ниже 200. На плоттере можно проследить ход сигнала:

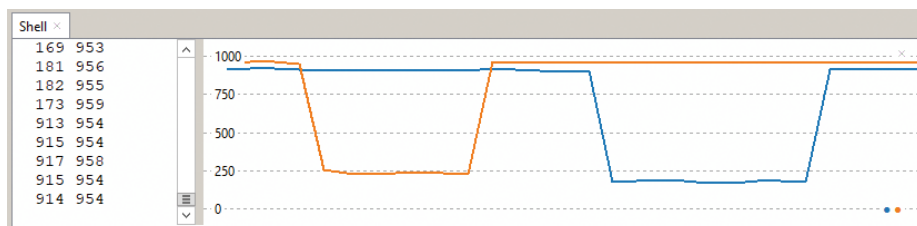


Рис.9.10: Изменение сигнала при прикосновении к сенсорным датчикам.

С помощью следующего скрипта два светодиода на портах 25 и 26 можно переключать с помощью сенсорных площадок. При касании площадки загорается соответствующий светодиод:

```
# touch_LED.py

from machine import TouchPad, Pin
from time import sleep

t1 = TouchPad(Pin(2))
t2 = TouchPad(Pin(4))

led1 = Pin(25, Pin.OUT)
led2 = Pin(26, Pin.OUT)

while True:
```

```

if t1.read() < 500:
    led1.value(1)
else:
    led1.value(0)

if t2.read() < 500:
    led2.value(1)
else:
    led2.value(0)

```

Примечание: ESP32 также можно вывести из спящего режима с помощью сенсорных панелей. Дополнительные сведения об использовании методов сна см. в главе 17.

9.9 Хорошо охлажденный или перегретый: датчики температуры обеспечивают четкость

Точное измерение температуры является одним из наиболее важных применений датчиков в электронной измерительной технике. Термодатчики и датчики температуры используются для контроля процессоров современных ПК или химических реакций на промышленных предприятиях, а также температуры масла в моторных лодках или серверных больших компьютерных систем. Точный и непрерывный мониторинг температуры также часто чрезвычайно важен в области медицины.

Классическими спиртовыми или ртутными термометрами можно измерять только текущую температуру. С другой стороны, электронный термометр также позволяет проводить измерения с временным разрешением. Это чрезвычайно важно для мониторинга температуры силовых транзисторов, микропроцессоров, инкубаторов или серверных ферм, поскольку только наблюдение в течение более длительных периодов времени обеспечивает безопасную работу. В случае чрезвычайной ситуации может быть активирован дополнительный блок охлаждения или даже может быть остановлена работа всей линейки серверов.

Простые датчики NTC (с отрицательным температурным коэффициентом) недороги и обеспечивают достоверные показания температуры, но существенным недостатком является необходимость калибровки. Для одиночных датчиков и в непрофессиональных приложениях это обычно приемлемо. Однако в профессиональной среде и при использовании большого количества датчиков усилия по калибровке могут быстро стать нерентабельными. Здесь предпочтительны датчики с заводской калибровкой, такие как LM35 или TMP36. Высокая точность измерения достигается за счет индивидуальной настройки производителем. Таким образом, эти датчики можно использовать напрямую без какой-либо дополнительной подготовительной работы. Цоколевку широко используемого датчика типа TMP36 можно увидеть на следующем рисунке:

Рис.9.11: Распиновка датчика TMP36.

Следующие данные относятся к TMP36:

Напряжение питания:	от 2,7 В до 5,5 В
Калибровка:	заводская настройка в °C
Масштабный коэффициент:	10 мВ/°C
Точность:	±2 °C (типичная)
Линейность:	±0,5 °C (тип.)
Диапазон температур:	от -40 °C до +125 °C
Ток замкнутой цепи:	<50 мкА

АЦП на выводе 34 можно использовать для считывания аналоговых измеренных значений. В результате получается следующая схема подключения:

Рис.9.12: TMP36 на плате ESP.

Основным преимуществом датчика TMP36 является заводская калибровка, гарантирующая градиент ровно 10 мВ/°C. Датчики обеспечивают выходное напряжение 1,0 В при температуре 50,0 °C. Это приводит к определению температуры в градусах Цельсия:

$$\text{Temp} = (\text{mV} - 500.0) / 10.0$$

Программа для записи температуры может выглядеть так:

```
# TMP36_TempSens_test.py

from machine import Pin, ADC
from time import sleep
```

```

TMP36pin = ADC(Pin(34))
TMP36pin.atten(ADC.ATTN_0DB) # диапазон ввода 0 ... 1 В

averages=10

while True:
    T=0
    for average in range(averages):
        TMP36data=TMP36pin.read()
        TMP36_mV=TMP36data/4.096 # напряжение датчика в милливольтax
        T+=(TMP36_mV-500.0)/10.0;
    print(int(10*T/averages)/10.0)
    sleep(1)

```

Измеренные значения выводятся на консоль. Для проверки работоспособности сенсора к TMP36 можно прикоснуться пальцем или слегка подуть феном. Это должно привести к увеличению отображаемых значений температуры.

Значения могут колебаться относительно сильно, поскольку во внутренние перекрестные помехи от других функциональных блоков в ESP вызывают определенный уровень шума. Изменение напряжения всего на 10 мВ/°C является относительно небольшим. Таким образом, амплитуды шумовых сигналов в милливольтном диапазоне уже могут существенно влиять на результат измерения. Колебания уменьшаются за счет усреднения.

С помощью этой программы ESP32 превращается в компьютерный термометр, с помощью которого можно записывать измеренные значения в течение более длительных периодов времени. Например, температуру в винном погребе, функцию морозильной камеры или контроль температуры в теплице и т.д. можно постоянно контролировать.

Здесь используется классический вариант форматирования вывода данных:

```
print(int(10*T/averages)/10.0)
```

Этот вариант гарантирует, что значения выводятся только с одним десятичным знаком. Для этого измеренное значение сначала умножается на 10. Затем десятичные разряды обрезаются с помощью функции `Int()`. Наконец, при делении на 10,0 генерируется значение только с одной десятичной позицией. В качестве альтернативы в MicroPython также можно использовать следующую версию:

```

output = "T = {:.1f} °C"
print(output.format(T/10))

```

для вывода значений температуры с одним десятичным знаком.

9.10 Цифровая регистрация температуры для безошибочной передачи данных

Если вы хотите записывать точные значения температуры даже в труднодоступных или удаленных местах, без электронного измерения не обойтись. С помощью обычных термометров можно определить только температуру в месте расположения самого термометра. Мониторинг температуры в масляном поддоне дизельного двигателя, в реакторе химической реакции или в серверной части большой компьютерной системы вряд ли можно осуществить с помощью бытовых термометров.

С другой стороны, электронный термометр лучше всего подходит для измерения температуры двигателя. Температуру можно измерить в цилиндре и отобразить в кабине. В промышленных условиях также без проблем возможны измерения на оси мегаваттного генератора, в трансформаторе или в масляной ванне.

Однако могут возникать ошибки измерения, если расстояние между датчиком и блоком обработки данных становится слишком большим. Даже на расстоянии измерения в один метр возмущающие воздействия, такие как сопротивление линии или электромагнитные помехи, могут оказывать негативное влияние на аналоговые датчики. Контактные напряжения на штекерных соединениях и т. д. также могут искажать аналоговые измерения.

Цифровая передача измеренных значений может свести к минимуму упомянутые выше ошибки измерения. Экранированные кабели обеспечивают расстояние до ста метров и более между местом измерения и блоком отображения. Даже неэкранированные плоские ленточные кабели передают цифровые сигналы на несколько метров со сравнительно низким уровнем помех. Это позволяет, например, контролировать температуру в холодильнике или морозильной камере. Широко используется цифровой датчик температуры — DS18(x)20. В следующем разделе более подробно объясняется использование этого преобразователя.

9.11 Однопроводной датчик DS18x20

Датчики серии DS18x20 обмениваются данными через так называемую однопроводную шину. Линия данных датчика подключается к ESP32 через любой пин ввода-вывода. Кроме того, требуется только подтягивающий резистор на 4,7 кОм. Если используется внутренняя подтяжка ESP32, даже это можно не делать. Датчик имеет следующие особенности:

Напряжение питания:	от 3,0 В до 5,5 В
Диапазон температур:	от -55 °C до +125 °C
Точность измерения:	±0,5°C (от -10°C до +85°C)
Разрешение:	9 бит, что соответствует прил. 1/10 °C
Период измерения:	750 мс (макс.)

Кроме того, можно считывать несколько датчиков. Благодаря специальному протоколу шинной системы One-Wire почти любое количество датчиков температуры может быть опрошено параллельно через один пин контроллера.

На следующем изображении показано подключение DS18x20:

Рис.9.13: Разводка термодатчика DS18x20.

Соответствующая программа предоставляет значения всех подключенных датчиков:

```
# DS18x20_TempSens_demo.py

from machine import Pin
import onewire
import ds18x20
import time

ow = onewire.OneWire(Pin(25)) # инициализация однопроводной шины
ow.scan()
ds=ds18x20.DS18X20(ow)      # создать объект ds18x20

while True:
    units=ds.scan() # сканировать объект ds18x20
    ds.convert_temp() # преобразование единиц измерения температуры в единицы измерения:
    print(ds.read_temp(unit)) # отобразить
    time.sleep(1)
    print()
```

Модули Python для чтения датчика снова доступны в стандартной комплектации прошивки MicroPython.

Рис.9.14: Датчик с одним проводом, подключенный к ESP32.

9.12 Мощность данных: мультисенсорная матрица с термодатчиком DS18x20

Измерение температуры — это просто прогулка по парку благодаря протоколу One-Wire, даже когда требуется несколько точек измерения. Все датчики могут быть подключены, так сказать, параллельно (см. иллюстрацию), то есть они занимают только один контакт. Это стало возможным благодаря индивидуальному адресу каждого датчика в массиве.

Рис.9.15: Несколько однопроводных датчиков, подключенных к ESP32.

Эта технология может быть очень эффективно использована в программном обеспечении. Инструкция

```
units=ds.scan()
```

предоставляет информацию о количестве подключенных датчиков. Подпрограмма возвращает массив, содержащий отдельную запись для каждого подключенного датчика. Например, для четырех подключенных датчиков это выглядит так:

```
[bytearray(b'\x10\r\xc8\x03\x08\x00F'),  
 bytearray(b'\x10\xe\xda\x03\x08\x00n'),  
 bytearray(b'\x10j\xd0\x03\x08\x00B'),  
 bytearray(b'\x10\x18\xd5\x03\x08\x00f')]
```

Здесь также может сыграть роль еще одно преимущество MicroPython. Вывод значений температуры осуществляется с помощью конструкции for/loop:

```
for unit in units:
```

Таким образом, здесь используется тот факт, что в Python операторы for могут использоваться не только для чисто целочисленных значений, но и для элементов массива.

9.13 В полный рост: оптические датчики

Подобно термодатчикам, светочувствительные компоненты стали очень важными в технике и повседневной жизни. Первоначально для этой задачи использовались LDR (фоторезисторы). Однако сейчас они считаются устаревшими. Кроме того, они содержат вещества, вредные для окружающей среды, и только по этой причине их нельзя больше использовать. Современными преемниками LDR являются фотодиоды и фототранзисторы. Область их применения варьируется от управления автоматическими системами освещения до световых барьеров и систем лазерного мониторинга. Фотодатчики также используются в современных смартфонах для автоматической регулировки яркости экрана в зависимости от условий окружающей среды.

Фотодиоды характеризуются быстрым откликом. Поэтому они предпочтительно используются в телекоммуникационных системах. Здесь они обеспечивают скорость передачи данных до терабитного диапазона. Если высокие скорости не столь важны, фототранзистор является хорошим выбором. Световые барьеры, экспонометры для фото- и кинокамер, а также приемники для инфракрасных пультов дистанционного управления являются типичными приложениями для этих компонентов.

Фототранзисторы обладают высокой чувствительностью. Кроме того, поведение отклика значительно лучше, чем у LDR. С точки зрения механической конструкции их обычно трудно отличить от белого светодиода. Фототранзисторы дешевы и широко используются. Они работают аналогично обычным транзисторам, с той разницей, что в случае фототранзистора свет попадает на полупроводник прямо через прозрачный корпус. Там носители заряда высвобождаются за счет внутреннего фотоэффекта. Затем фототок усиливается в самом компоненте. Таким образом, фоторезистор может напрямую коммутировать меньшие токи до нескольких миллиампер. Фототранзисторы обычно имеют только два вывода — коллектор и эмиттер. Однако существуют также версии с выводным базовым соединением,

например для регулировки рабочей точки. Если база остается неподключенной, проходит относительно много времени, пока зона база-эмиттер не освободится от носителей заряда. Это одна из причин, почему фототранзистор выключается медленнее, чем фотодиоды.

На следующем рисунке показан фототранзистор типа BPW40:



Рис.9.16: Фототранзистор.

Кремниевые фототранзисторы достигают максимальной чувствительности при длине волны света ок.850нм. Она находится в невидимом ближнем инфракрасном диапазоне. Благодаря широкому спектральному диапазону хорошие характеристики отклика достигаются даже на длинах волн видимого диапазона (400–800 нм). В следующей таблице приведены наиболее важные данные фототранзистора BPW40:

Макс. напряжение питания:	30 В
Темновой ток:	100 нА
Ток коллектора при 1 мВт/см ² :	2.5 мА
Макс. ток коллектора:	20 мА
Диапазон длин волн:	видим. свет в ближнем инфракрасном диапоз.

В следующей таблице приведены значения фототоков, ожидаемых при определенных условиях освещения:

	Освещенность	Ток коллектора
Laser pointer (5mW)	> 400.000 lx	Насыщение при 20 мА
Лазерная указка (5 мВт)	10.000 lx	20 мА
Телевизионная студия	2.000 lx	4 мА
Освещение офиса или комнаты	1.000 lx	2 мА
Типичное освещение коридора	100 lx	0.2 мА
Абсолютная темнота	0 lx	100 нА

9.14 Для кино- и фотопрофессионалов: электронный люксметр

Фототранзистор можно использовать для точного определения яркости окружающей среды. Это чрезвычайно важно для кино или фотосессий. Фотографии и фильмы могут быть по-настоящему убедительными только при наличии достаточного освещения. Поэтому профессиональные фотографы и операторы всегда имеют под рукой так называемый люксметр. Это позволяет быстро и легко проверить, подходят ли условия освещения для хороших снимков.

Для самодельного люксметра можно снова использовать внутренний АЦП ESP32, аналогичный TMP36. Датчик температуры просто заменяется измерительной схемой с BPW40.

На следующем рисунке показана аппаратная структура. Как и любой транзистор, фототранзистор должен быть установлен с соблюдением полярности. Более короткий соединительный провод (коллектор) должен быть подключен к напряжению питания 3,3 В. Это соединение также отмечено уплотнением на корпусе. Если измеренные значения не меняются или меняются очень незначительно при попадании света, полярность BPW40 должна быть изменена. На следующем рисунке показана схема люксметра:

Рис.9.17: Схема подключения электронного люксметра.

Программа для этого следующая:

```
# BPW40_luxmeter.py

from machine import Pin, ADC
from time import sleep

fotoPin = ADC(Pin(34))
fotoPin atten(ADC.ATTN_11DB) # входной диапазон 0 ... 3,3 В

averages=10

while True:
    luxValue=0
    for average in range(averages):
        luxData=fotoPin.read()
        luxValue+=luxData/40.96
    luxValue/=averages
    output="Light level: {:.1f} %"
    print(output.format(luxValue))
    sleep(1)
```

Текущие значения яркости отображаются на консоли. Там, где освещение обеспечивается электрическими лампами, работающими на 230 В, 50 Гц, нефильтрованные измерения показывают сильные колебания. Этот эффект особенно заметен для современных светодиодных ламп. Это явный признак того, что светодиоды на самом деле мерцают сильнее, чем обычные лампы накаливания, даже если глаз сознательно не воспринимает эти быстрые колебания света. Таким образом, измеренные значения рассчитываются по ряду средних значений:

```
для среднего в диапазоне (средних):
luxData=fotoPin.read()
luxValue+=luxData/40.96
```

При масштабировании до 40,96 ($= 2^{12}/100$) уровни освещенности отображаются в процентах, то есть от 0,0 до 100,0. Значения выводятся в формате с десятичной точкой:

```
output="Light level: {:6.1f} %"
print(output.format(luxValue))
```

В полной темноте должно отображаться значение значительно ниже 1%. С другой стороны, при ярком освещении можно достичь почти 100%.

В качестве рабочего сопротивления можно использовать значение 100 кОм или 10 кОм. При использовании сопротивления 100 кОм люксметр достигает очень высокой чувствительности. Это позволяет обнаруживать даже самые низкие уровни освещенности, едва заметные глазу. Отображение производится в относительных значениях. Для определения абсолютных значений, например, для измерений на промышленных рабочих местах или путях эвакуации система должна быть откалибрована с использованием сравнительного прибора. Тогда также возможны значения в люменах, люксах или канделах.

В разделе 16.4 фототранзистор используется для визуального наблюдения за помещением.

.15 Электронные летучие мыши: измерение расстояния с помощью ультразвука

В принципе, расстояния можно измерять и оптическими датчиками. Однако методы измерения отражения на оптической основе имеют тот недостаток, что результаты сильно зависят от свойств поверхности светоотражающих материалов. Из-за высокой скорости света для измерения оптического времени прохождения требуются сложные электронные процедуры оценки. По этим причинам ультразвук часто используется для измерения расстояния. Для измерения расстояний в диапазоне средних расстояний, т. е. от нескольких сантиметров до нескольких метров, подходят соответствующие методы. Так как скорость звука в воздухе при 20°C составляет ок. 343 м/с, время прохождения звука на расстояние в один сантиметр составляет около 30 микросекунд. Этот промежуток времени можно очень точно измерить с помощью MicroPython.

Применения этого метода измерения включают определение высоты помещения, измерение высоты просвета моста или измерение числа квадратных метров в помещениях и площадях. Высокая надежность ультразвуковых измерительных систем позволяет использовать их как в робототехнике, так и в обычных или автономных транспортных средствах. С помощью ультразвука роботизированные системы способны обнаруживать препятствия. При парковке расстояние автомобиля от стены можно точно определить с помощью датчиков в кузове автомобиля.

На рынке появились очень экономичные приемо-передающие модули с резонансной частотой 40 кГц. Таким образом, самостоятельная сборка ультразвуковых систем вряд ли имеет смысл. На следующем рисунке показан широко используемый модуль типа HC-SR04.



Рис.9.18: Ультразвуковой модуль с двумя преобразователями.

Это экономичный ультразвуковой модуль со встроенными капсулями передатчика и приемника. Эта версия идеально подходит для активации с помощью контроллера ESP32. HC-SR04 может обнаруживать объекты на расстоянии от 2 см до прикл. 3 м. Разрешение измерения составляет около 3 мм. Точность зависит от скорости звука и, следовательно, от температуры воздуха, а также от уровня шума. В следующей таблице приведены технические данные модуля:

Напряжение питания V_{CC} :	+5 В $\pm 10\%$,
Потребление тока:	ок. 2 мА
Уровень сигнала (триггер):	от 3 до 5 В
Измеряемые расстояния:	от 2 см до примерно 3 м
Разрешение измерения:	ок. 3 мм
Измерений в секунду:	максимум 50
Размеры (д, ш, г) в мм:	45 x 20 x 20 мм

Доступна библиотека для MicroPython, что значительно упрощает работу с датчиком. Определенным недостатком модуля является то, что он требует напряжения питания 5 В. Однако, поскольку выводы ввода/вывода ESP32 рассчитаны только на 3,3 В, уровень сигнала на выходе модуля необходимо изменить с помощью делителя напряжения. В результате получается следующая схема:

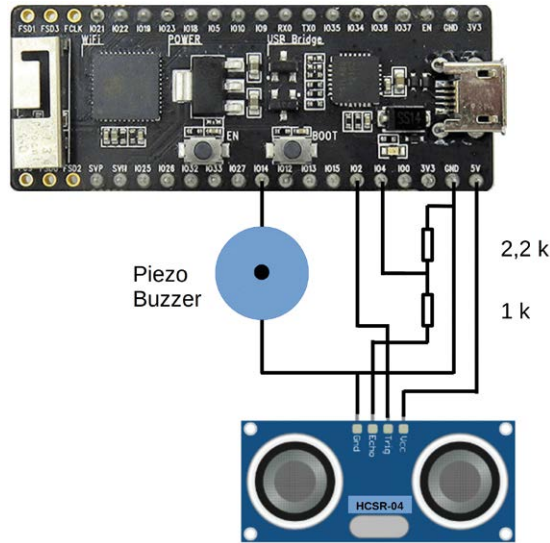


Рис.9.19: Ультразвуковой модуль на ESP32.

Устройство на макетной плате может выглядеть так:

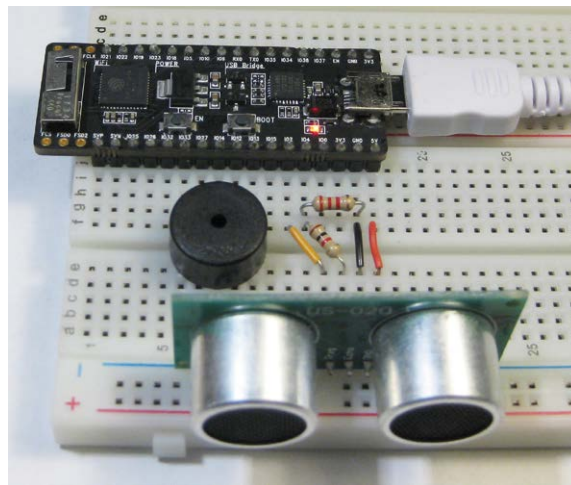


Рис.9.20: Устройство для ультразвукового измерения расстояния.

Для триггерного входа достаточно 3,3 В выхода ESP, так что преобразователь уровня не нужен. Показанный также зуммер нужен только в следующей главе, и его в начале можно не использовать.

С программной стороны управление модулем очень простое. После срабатывания по заднему фронту модуль автоматически измеряет расстояние и преобразует его в широтно-импульсный сигнал, поступающий на выход модуля. Интервал измерения имеет длительность 20 мс. Это означает, что можно выполнять 50 измерений в секунду.

После запуска измерительного цикла модуль посылает ультразвуковой измерительный импульс длительностью прилб. 200 мкс. Сразу после этого выход переходит на высокий уровень, и модуль ожидает приема эхо-сигнала. Если он обнаружен, эхо-выход возвращается к низкому уровню. Через двадцать миллисекунд после срабатывания может быть выполнено другое измерение. Если эхо не обнаружено, выходной сигнал остается на высоком уровне в течение 200 мс, указывая, таким образом, сообщением «превышение диапазона», что ни один объект не находится в пределах диапазона. Затем модуль ожидает следующего спадающего фронта на входе триггера, и измерение начинается снова.

Наилучшие результаты измерения получаются при отражении от гладких и плоских поверхностей. На расстояниях до одного метра материал поверхности не столь критичен. Угол к объекту может быть от менее 1° до примерно 45° на коротких расстояниях. Объекты размером с карандаш надежно обнаруживаются.

Разрешение измерения 3 мм определяется внутренней скоростью сканирования модуля. Точность измерений в основном ограничивается температурной зависимостью скорости звука в воздухе. В приближении скорость звука как функцию температуры можно рассчитать в диапазоне от -20°C до +40°C по следующей формуле:

$$V_s = (331.5 + 0.6 * T_{\text{air}} / ^\circ\text{C}) \text{ м/с.}$$

Для комнатной температуры 20°C это дает

$$c = 331,5 + (0,6 \times 20) \text{ м/с} = 343,5 \text{ м/с}$$

На основе этого значения следующая программа обеспечивает значения измерения расстояния в сантиметрах:

```
# HCSR04_US_distance_demo.py
# измерение расстояния с помощью ультразвука

from hcsr04 import HCSR04
from time import sleep

sensor = HCSR04(trigger_pin=2, echo_pin=4)

while True:
    distance = sensor.distance_cm()
    output="Distance: {:.1f} cm"
    print(output.format(distance))
    sleep(.1)
```

Библиотека «HCSR04» генерирует импульс запуска, который запускает соответствующий импульс передачи в ультразвуковом модуле. После этого система ожидает прихода эхо-сигнала. Информация о расстоянии передается по ширине принимаемого импульса. На осциллографе можно увидеть два сигнала:

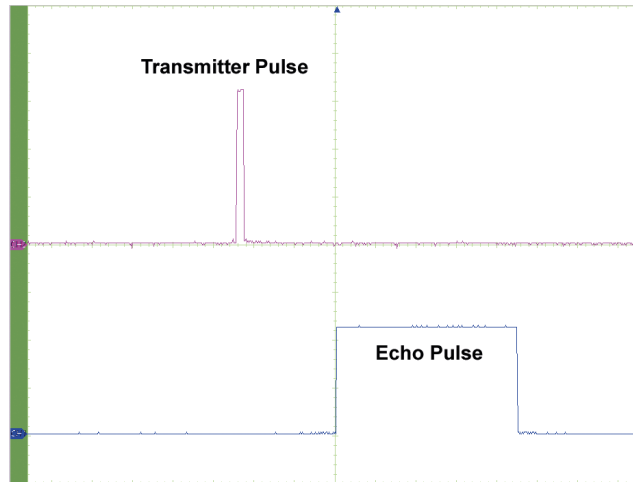


Рис.9.21: Импульс передачи ультразвука (верхняя кривая) и импульс приема (нижняя кривая).

ESP32 нужно только определить ширину эхо-импульса. Это делается библиотечной функцией. Функция

```
sensor.distance_cm()
```

непосредственно возвращает измеренное расстояние в сантиметрах. Затем измеренное значение может быть выведено на оболочку через

```
output="Distance: {:.1f} cm" print(output.format(distance))
```

и форматируется с одним десятичным знаком.

При разнице температур в 20 °C уже возникает погрешность в 3,4 %. Поэтому для использования вне помещений и измерения больших расстояний полезна температурная компенсация. Для простых приложений, таких как помощь при парковке, температурная компенсация не требуется, как показывает пример приложения в следующем разделе.

9.16 Больше никаких вмятин и царапин: дальномер для гаражей

Современные гаражи обычно очень компактны с точки зрения затрат. В частности, подземные парковочные места становятся все более узкими. Здесь повреждения автомобиля могут быстро произойти при парковке. Устройство предупреждения о расстоянии очень полезно в таких ситуациях.

Ультразвуковой дальномер может быть легко модернизирован до такого устройства. Для этого требуется только пороговое значение. Если расстояние между автомобилем и стеной меньше этого значения, должен прозвучать предупредительный сигнал. Если ультразвуковой модуль установить в подходящем месте, можно избежать поломки собственного автомобиля.

Функцию ШИМ можно использовать для вывода предупредительного звукового сигнала. В следующей строке отчетливо слышен сигнал частотой 1 кГц:

```
PWM(Pin(14), freq=1000, duty=512)
```

Чтобы предупреждающий сигнал был слышен, пассивный пьезо-зуммер должен быть подключен к контакту D14 платы ESP. На следующем рисунке показан подходящий зуммер:



Рис.9.22: Пьезо-зуммер.

Это дополнит программу из предыдущей главы:

```
# HCSR04_anti_colission.py

from machine import Pin, PWM
from hcsr04 import HCSR04
from time import sleep

sensor = HCSR04(trigger_pin=2, echo_pin=4)
beeper = PWM(Pin(14), freq=1000, duty=512)
beeper.deinit()

minDist=10

while True:
    distance = sensor.distance_cm()
    output="Distance: {:6.1f} cm"
    print(output.format(distance))
    if (distance<minDist):
        beeper = PWM(Pin(14),freq=1000,duty=512)
        sleep(.5)
        beeper.deinit()
    sleep(.1)
```

Как только расстояние между ультразвуковым модулем и автомобилем падает ниже значения «minDist» (в см), звучит предупредительный звуковой сигнал. Аналогичные системы также используются в промышленном производстве и все чаще в робототехнике.

9.17 Оптимальный микроклимат в помещении для флоры и фауны

Когда речь идет об общем самочувствии, важна не только температура, но и влажность. Даже при приятных 22 °C вы не будете чувствовать себя комфортно, если воздух очень сухой. С другой стороны, слишком высокий уровень влажности также не способствует хорошему самочувствию и здоровью. Влажность воздуха также имеет решающее значение для комнатных растений. Многим комнатным растениям, даже кактусам, требуется влажность воздуха не ниже 40%. Тропические или субтропические растения даже предпочитают гораздо более влажный воздух. Ориентировочное значение, оптимальное для многих растений, составляет около 60% относительной влажности (RH).

В то же время растения обеспечивают регулируемое содержание влаги в воздухе за счет испарения через листья. Поэтому для оптимального микроклимата в помещении полезно контролировать как температуру, так и влажность воздуха. При необходимости можно вмешаться, чтобы урегулировать ситуацию, например, с дополнительными комнатными растениями.

Комбинированные датчики для определения влажности и температуры позволяют одновременно измерять оба значения. Если модуль датчика уже содержит необходимые аналого-цифровые преобразователи в дополнение к самому датчику, считывание очень просто. В этом случае требуется только один контакт данных, через который передается информация об измеренном значении.

DHT11 — относительно недорогой датчик температуры и влажности. Влажность воздуха обоих типов измеряется емкостным методом измерения. Для измерения температуры воздуха используется термистор. Поскольку преобразователь выводит готовый цифровой сигнал на свой вывод данных, аналоговый ввод не требуется. Если требуются более точные значения, можно использовать DHT22 или его более оптимизированные версии AM2320, HTU-21 или Si7021. Они имеют более высокую точность, лучшее разрешение и более широкий диапазон измерений. Однако и цена их соответственно выше.

Модуль выдает новое измеренное значение каждые две секунды. Поскольку комнатная температура и влажность воздуха обычно не изменяются очень быстро, в большинстве случаев такой скорости измерения должно быть достаточно.

Следующие два рисунка показывают фотографию модуля и соответствующее назначение ножек.

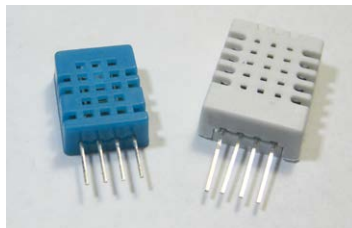


Рис.9.23: Датчики температуры/влажности типов DHT11 и DHT22.

Рис.9.24: Назначение выводов DHT11/22.

Обозначение «пс» здесь означает «не подключен», т. е. штифт не подключен внутри. Характеристики датчиков приведены в следующей таблице:

	DHT11	DHT22
Напряжение питания	от 3,3 до 5,5 В	от 3,3 до 5,5 В
Потребляемая мощность активна	0,3 мА тип. - 2,5 мА макс.	от 1 до 1,5 мА
Точность повторения влажности	±1% RF	±1% RF
Точность измерения влажности при 25°C	±2% RF (в диапазоне от 20 до 90 %)	±2% RF (в диапазоне от 20 до 100 %)
Время реакции влажности при 25°C	6 с	6 с
Долговременная стабильность	<±0,5 % RF/год	<±0,5 % RF/год
Точность измерения температуры	±0,5 °C (в диапазоне от 0 до 50°C)	±0,5 °C (в диапазоне -40 до 80°C)
Разрешение	1% RF, 1 °C	0.1% RF; 0.1 °C
Время температурной реакции	10 с	10 с
Распиновка	1: Vdd, 2: DATA, 3: NC, 4: GND	1: Vdd, 2: DATA, 3: GND, 4: GND
Скорость передачи данных	1 измерение в 2 секунды	1 измерение в 2 секунды

Более высокая точность, лучшее разрешение и более широкий диапазон измерения температуры и влажности DHT22 требуют дополнительных усилий при изготовлении и тестировании, что также отражается на его цене.

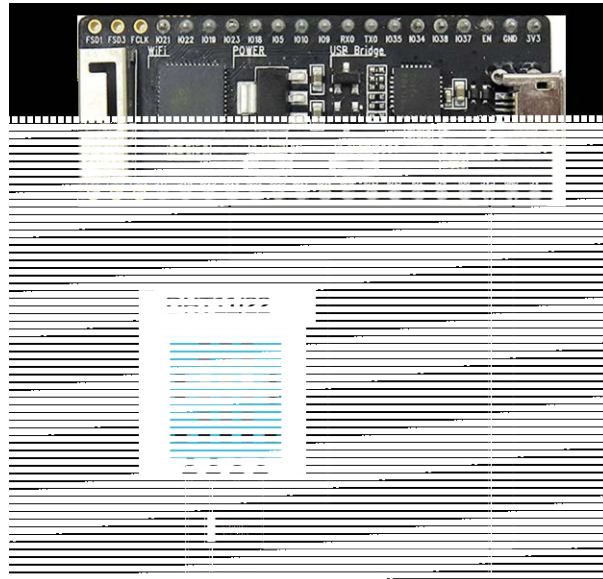


Рис.9.25: Принципиальная схема модуля DHT, подключенного к плате ESP.

Подтяжка 10 кОм гарантирует, что сигнальный контакт датчика DHTxx будет подтянут к ВЫСОКОМУ уровню, если не применяется фиксированный уровень. Это подтягивание не всегда необходимо при работе с платой ESP, так как внутреннее подтягивание активируется в библиотеке DHT. Если, вопреки ожиданиям, с резистором проблем нет, его можно использовать, как показано.

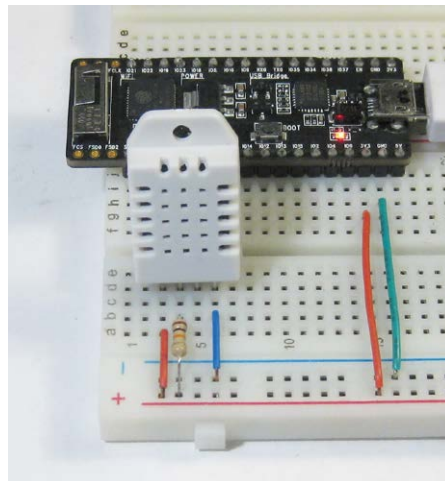


Рис.9.26: Модуль DHT22 и модуль ESP на макетной плате.

Датчики считываются через соответствующую библиотеку. Он автоматически загружается на ESP32 вместе с прошивкой MicroPython. Таким образом, датчики могут быть считаны с помощью следующей программы.

```
# DHT11_HumiTemp_sensor_demo.py

from machine import Pin
from time import sleep
import dht

#sensor = dht.DHT11(Pin(14))
sensor = dht.DHT22(Pin(14))

while True:
    try:
        sleep(2)
        sensor.measure()
        temp = sensor.temperature()
        hum = sensor.humidity()
        print(,Temperature: %3.1f C' %temp)
        print(,Humidity: %3.1f %%' %hum)
        print()

    except OSError as e:
        print(,Failed to read sensor.')
```

можно активировать или закомментировать.

Здесь можно еще раз продемонстрировать функцию конструкции try/except, используемую в программе. Если датчик будет удален во время работы для имитации дефекта, программа не будет прервана. Вместо этого отображается предупреждающее сообщение «Не удалось прочитать датчик», и программа продолжает работу:

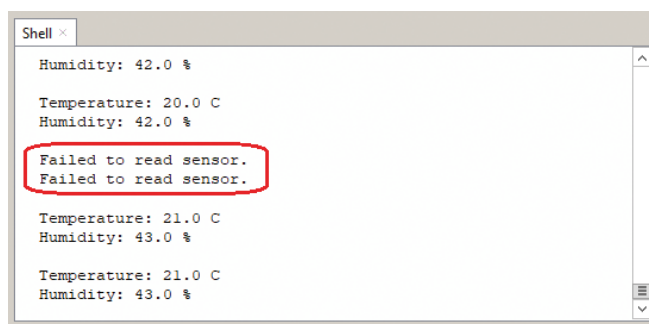


Рис.9.27: Предупреждающее сообщение о сбое датчика.

Когда датчик вставлен снова, программа автоматически возобновляет свою первоначальную работу. Так, если, например, в удаленной измерительной станции активны несколько датчиков, неисправность одного датчика не приводит к полному выходу из строя всей станции из-за прерывания программы.

9.18 «Верь мне...»: сравнение датчиков

В MicroPython несколько типов датчиков могут работать одновременно на одном контроллере без каких-либо проблем. Следующая программа отображает значения одного DHT11 и одного DHT22 на консоли:

«Я доверяю только той статистике, которую сам сфальсифицировал», — так, как говорят, сказал Уинстон Черчилль. С измеренными значениями дело обстоит не так уж сильно. Поэтому всегда интересно сравнить результаты разных датчиков.

```
# DHT11_22_double HumiTemp_sensor.py

from machine import Pin
from time import sleep
import dht

sensor1 = dht.DHT11(Pin(26))
sensor2 = dht.DHT22(Pin(14))

while True:
    try:
        sleep(2)
        sensor1.measure()
        temp1 = sensor1.temperature()
        hum1 = sensor1.humidity()
        print("DHT11:")
        print(,Temperature: %3.1f C' %temp1)
        print(,Humidity: %3.1f %%' %hum1)
        print()

        sensor2.measure()
        temp2 = sensor2.temperature()
        hum2 = sensor2.humidity()
        print("DHT22:")
        print(,Temperature: %3.1f C' %temp2)
        print(,Humidity: %3.1f %%' %hum2)
        print()

    except OSError as e:
        print(,Failed to read sensor.')
```

Она также позволяет проводить очень информативные сравнительные измерения. Лучший способ сделать это — удалить дополнительную информацию в инструкциях `print()`. Тогда будут выводиться только чистые данные. Затем данные можно сравнить непосредственно в плоттере.

На следующем рисунке показано сравнение, полученное таким образом:

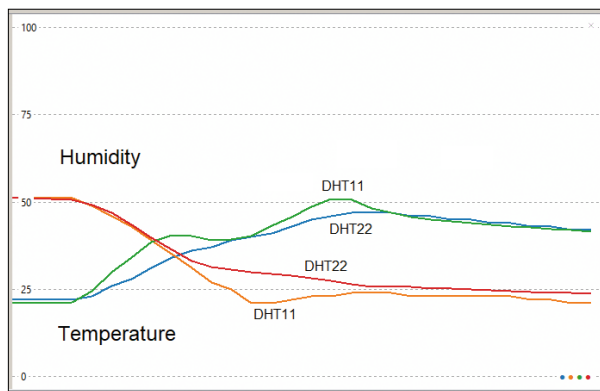


Рис.9.28: Сравнительное измерение DHT11/DHT22.

Подув теплым воздухом из фена, можно продемонстрировать, как датчики реагируют на изменения температуры и влажности. Видно, что со временем есть отклонения. В целом, однако, измеренные значения образцов, использованных здесь, хорошо согласуются.

9.19 Измерение атмосферного давления и высоты над уровнем моря

Наряду с температурой и влажностью, атмосферное давление является наиболее важным параметром, когда речь идет о климатических данных и прогнозировании погоды. Доступен датчик BME280 от Bosch — современный измерительный преобразователь, способный регистрировать все три параметра. Данные датчиков снова выводятся через интерфейс I2C. Технические данные BME280 приведены в следующей таблице:

Размеры (Д x Ш)	14 x 10 mm	
Рабочее напряжение:	от 1,8 В до 5 В	
Интерфейс:	I ² C	
Диапазоны измерения:	Температура:	−40 °C до +85 °C
	Давление воздуха:	от 300 гПа до 1100 гПа ± 0,25 %
	Влажность:	относительная точность ±3%, ≤2% Гистерезис

Тестовые платы с датчиком BME280 имеют 4 контакта, которые подключаются к ESP32 следующим образом.

BME280ESP32	
VIN	3.3 V
GND	GND
SCL	22
SDA	21

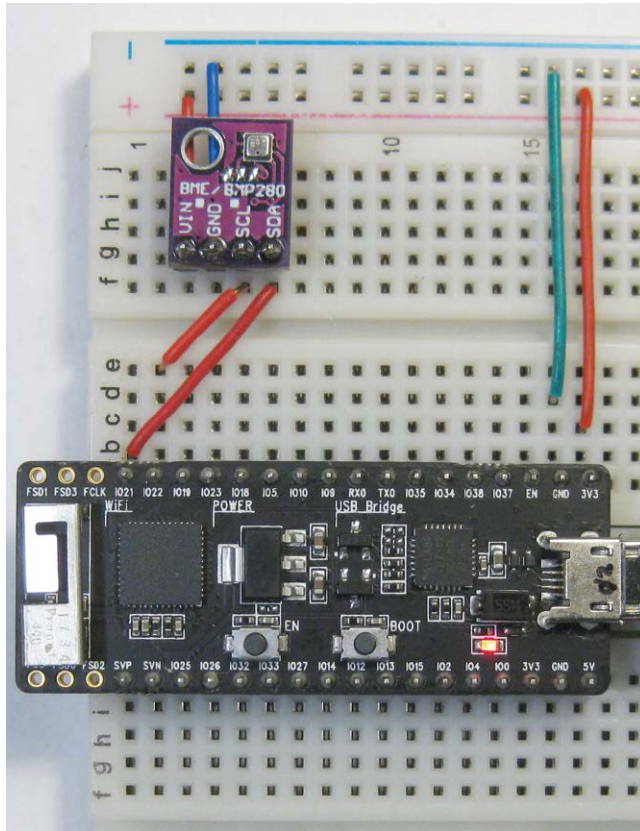


Рис.9.29: BME280 и ESP32 на макетной плате.

Доступны различные библиотеки для считывания показаний датчика. Ссылки можно найти в папке с кодами. Файл BME280.py необходимо перезагрузить в ESP под user_lib. Затем текущие измеренные значения могут быть считаны с помощью следующего скетча.

```
# BME280 Temp_Humit_Press-sensor_test.py

from machine import Pin, I2C
from time import sleep
import BME280

# ESP32 - Pin assignment
i2c = I2C(scl=Pin(22), sda=Pin(21), freq=10000)
# ESP8266 - Pin assignment
#i2c = I2C(scl=Pin(5), sda=Pin(4), freq=10000)

while True:
    bme = BME280.BME280(i2c=i2c)
    temp = bme.read_temperature()/100.0
```

```

hum = bme.read_humidity()/1024.0
pres = bme.read_pressure()/25600.0
# uncomment for temperature in Fahrenheit
#temp = (bme.read_temperature()/100) * (9/5) + 32
#temp = str(round(temp, 2)) + ,F'
print(,Temperature: %5.1f C' %temp)
print(,Humidity   : %5.1f %%' %hum)
print(,Pressure   : %5.1f hPa' %pres)
print("-----")

sleep(1)

```

Атмосферное давление выводится в гектопаскалях (гПа) с одним десятичным знаком. Такая точность позволяет барометрически измерять разницу высот всего в один метр. Следующая программа предоставляет гектопаскаля в десятичной форме с двумя знаками после запятой.

```

# BME280 Press-sensor_numeric.py

from machine import Pin, I2C
from time import sleep
import BME280

i2c = I2C(scl=Pin(22), sda=Pin(21), freq=100000)
averages=30

def pressure():
    p=bme.read_pressure() // 256
    pi=p// 100
    pd=p-pi*100
    return "{:.02d}".format(pi, pd)

while True:
    bme=BME280.BME280(i2c=i2c)
    press=0
    for i in range(averages):
        press += float(pressure())
        sleep(.1)
    print(press/averages)

```

Кроме того, значения датчиков усредняются 30 раз. Это позволяет плоттеру обнаруживать даже малейшие различия в высоте. На следующем графике высота датчика была изменена на один метр. Вы можете четко видеть снижение и повышение давления.

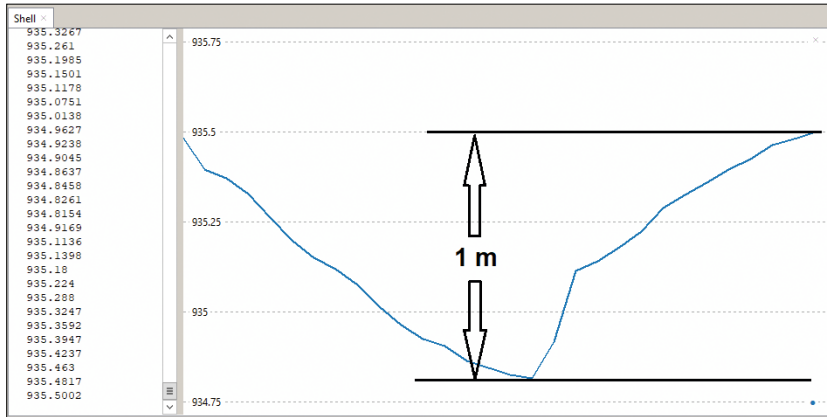


Рис.9.30: Измерение высоты с помощью BME280.

В разделе 16.1 датчик служит основой для виртуальной метеостанции, которую можно вызывать по всему миру.

9.20 Обнаружение магнитных полей с помощью датчика Холла

С чипом ESP32 поставляется бесплатный датчик. В дополнение ко многим другим функциям микросхема также содержит датчик Холла, который позволяет измерять напряженность магнитного поля вблизи ИС. Датчик Холла — или просто «Холла» — выдает напряжение, которое увеличивается с увеличением абсолютной напряженности магнитного поля вблизи преобразователя.

Датчик даже позволяет различать магнитный северный и южный полюса. Это означает, что его можно использовать для многих различных приложений. Единственным недостатком является то, что датчик постоянно встроен в чип. Это явно дает вам меньшую гибкость, чем внешний датчик Холла. Тем не менее, многие приложения реализуемы, например:

- Измерение частоты вращения двигателя
- Магнитный бесконтактный переключатель
- Магнитный переключатель для обнаружения открытых окон или дверей

Датчик Холла выдает аналоговые значения. Однако функции переключения также могут быть реализованы с помощью программного распознавания порогового значения. Датчик можно протестировать с помощью следующей программы. В основном цикле исходные данные датчика передаются на консоль:

```
# Hall_sensor_test.py

import esp32
from time import sleep

esp32.hall_sensor()    # read the internal hall sensor

while True:
```

```
print(esp32.hall_sensor())
sleep(1)
```

Датчик можно легко проверить с помощью магнита. Если магнит приблизить к чипу ESP, значения меняются. Лучше всего это видно с помощью функции плоттера оболочки:

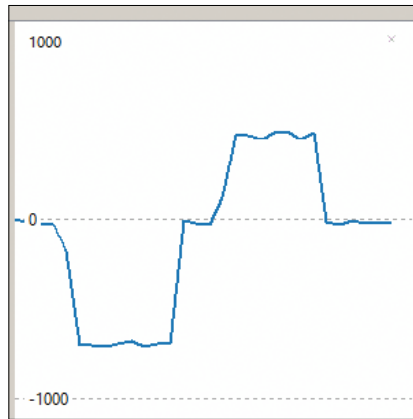


Рис.9.31: Измеренные значения датчика Холла.

Если, например, южный полюс указывает на чип, значения уменьшаются. В зависимости от силы магнита может быть достигнут диапазон от -500 до -1000. Если северный полюс приближается к чипу, измеренные значения возрастают от +500 до +1000.

9.21 Датчики тревоги контролируют дверь и ворота

Датчик Холла можно использовать, например, для контроля дверей, ворот или окон. Следующая программа переключает зеленый светодиод на красный, как только магнитное поле вблизи чипа выходит за пределы определенного предельного значения.

```
#!/# Hall_sensor_alarm.py
import esp32
from time import sleep
from machine import Pin

ledRed = Pin(2, Pin.OUT)
ledGreen = Pin(4, Pin.OUT)

esp32.hall_sensor()      # read the internal hall sensor
threshold=100

while True:
    Hfield=int(esp32.hall_sensor())
    print(Hfield)
    if (Hfield>threshold):
```

```
    ledRed.value(0)
    ledGreen.value(1)
else:
    ledRed.value(1)
    ledGreen.value(0)
sleep(.1)
```

Для задач мониторинга к дверному или оконному датчику необходимо прикрепить постоянный магнит. Модуль ESP монтируется так, чтобы магнит находился рядом с модулем при закрытых окнах или дверях. При открытии магнит удаляется из модуля ESP, и напряженность магнитного поля вблизи чипа падает практически до нуля. Обнаружение порога генерирует цифровой сигнал. В примере программы он выключает зеленый светодиод и активирует красный светодиод. Сигнал можно использовать и для других целей. Например, его можно перенаправить через WLAN на ПК или смартфон, где он может вызвать соответствующий сигнал тревоги.

Глава 10 • Технология отображения и экраны малого размера

Отображение текстов и значений в консоли вполне подходит для многих целей тестирования и простых приложений. Однако этот вариант имеет большой недостаток в том, что всегда требуется ПК или хотя бы ноутбук. Если система находится в постоянной работе в течение нескольких дней или недель, работа этих устройств будет потреблять ненужное количество энергии. Кроме того, не всегда желательно или целесообразно использовать отдельный компьютер для каждого приложения микроконтроллера.

Если речь идет только об отображении времени или измеренных данных от климатических датчиков, гораздо лучше подключить к контроллеру собственный небольшой дисплей. Широко используемый тип — SSD1306. Этот дисплей имеет разрешение 128 x 64 пикселей и размер всего 0,96 дюйма (чуть менее 2,5 см).

Стандартно MicroPython поставляется с драйвером для этой версии дисплея. Этот драйвер уже загружен при загрузке файловой системы в ESP32. Драйвер позволяет отображать текстовые и числовые данные, а также простую графику. Дисплей SSD1306 оснащен внутренней оперативной памятью и собственным генератором. Поэтому он может работать без каких-либо внешних компонентов. Кроме того, он имеет регулировку яркости с 256 регулируемыми уровнями.

Основные характеристики дисплея SSD1306:

- Разрешение: матрица 128 x 64 точек
- Источник питания: от 1,65 В до 3,3 В.
- Диапазон рабочих температур: от -40 °С до +85 °С
- Встроенный 128 x 64-битный буфер дисплея SRAM
- Функция непрерывной прокрутки в горизонтальном и вертикальном направлении
- Встроенный генератор

OLED-дисплеям не требуется фоновое освещение, поскольку каждый отдельный пиксель способен излучать свет. По этой причине этот вариант по-прежнему легко читается даже при неблагоприятных условиях освещения. Кроме того, контрастность значительно выше по сравнению с жидкокристаллическими дисплеями (ЖК-дисплеями). Поскольку пиксель потребляет энергию только тогда, когда он действительно загорается, OLED-дисплеи очень энергоэффективны.

Самые простые версии плат SSD1306 имеют всего четыре контакта. Этого достаточно для управления дисплеем по шине I2C. Другие версии имеют контакты сброса или дополнительный интерфейс SPI. Однако для большинства приложений достаточно простой конструкции. В следующей таблице показаны все необходимые соединения.

OLED-Pin	ESP32
VDD	3V3
GND	GND
SCK	GPIO 22
SDA	GPIO 21

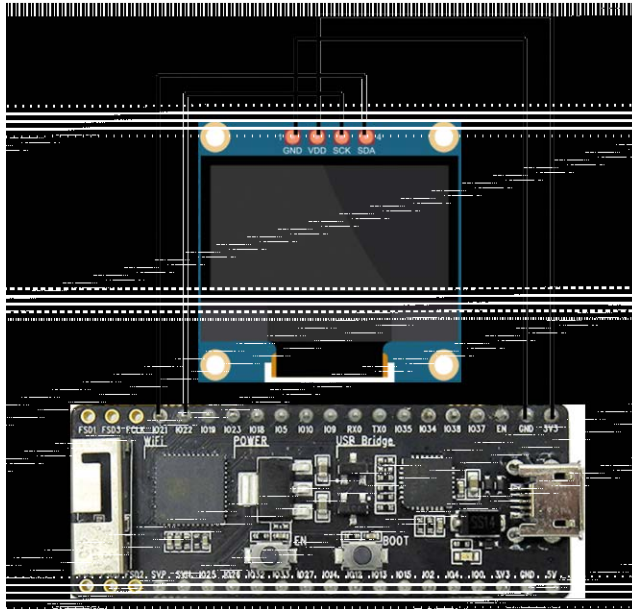


Рис.10.1: Дисплей SSD1306 на ESP32.

Следующий скрипт выводит на дисплей текстовое сообщение и простой графический элемент в виде фрейма:

```
# SSD1306_DEMO.py

from machine import Pin, I2C
from ssd1306 import SSD1306_I2C

i2c=I2C(-1,scl=Pin(22),sda=Pin(21))

# I2C Pin assignment
oled_width=128
oled_height=64
oled = SSD1306_I2C(oled_width, oled_height, i2c)
lin_hight = 9
col_width = 8
```

```
def text_write(text,lin, col):
    oled.text(text,col*col_width,lin*lin_hight)

oled.fill(0)
text_write("MicroPython", 1, 2)
text_write("for", 3, 6)
text_write("ESP32", 5, 5)

oled.rect(5, 5, 116, 52, 1)
oled.show()
```

и дает такой результат:

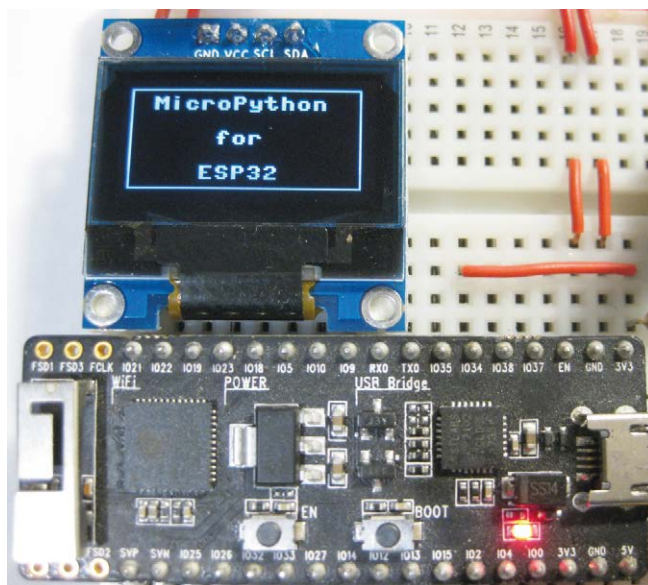


Рис.10.2: Дисплей SSD1306 с текстовым и графическим выводом.

Для управления дисплеем необходимо импортировать необходимые модули. Как уже упоминалось, библиотеки «машина» и «SSD1306» обычно уже доступны в качестве стандартных библиотек. Однако при необходимости файл `ssd1306.py` можно загрузить на плату отдельно. Пин-декларация для шины I2C выполняется с использованием:

```
i2c = I2C (-1, scl = pin (22), sda = pin (21))
```

Количество пикселей подключенного модуля записывается со следующими переменными:

```
oled_width = 128
oled_height = 64
```

Параметр «-1» указывает, что используемый модуль не имеет контактов сброса или прерывания. С помощью этой информации можно создать объект SSD1306_I2C с именем `oled`. Здесь берутся ранее определенные данные:

```
oled = ssd1306.SSD1306_I2C (oled_width, oled_height, i2c)
```

Теперь дисплей готов к работе. Функция `"text()"` выводит информацию на дисплей. Метод `show()` используется для обновления дисплея. Функция `text()` принимает следующие аргументы:

- Сообщение (строка)
- X-позиция и Y-позиция текстового поля в пикселях.
- Необязательный цвет текста: 0 = черный (темный) и 1 = белый (светлый)

Следующая инструкция выводит сообщение белого или синего цвета на темном фоне. Текст начинается с позиции $x = 0$ и $y = 0$:

```
oled.text ('MicroPython!', 0, 0)
```

Метод `show()` делает изменения видимыми на дисплее. Библиотека также содержит другие полезные методы. Функция `fill(1)` создает полностью белый экран, т. е. все пиксели светятся. При `oled.fill (0)` все пиксели становятся черными или темными.

Метод `pixel()` позволяет графическое представление. Он принимает следующие аргументы:

- X-координата: горизонтальное положение в пикселях.
- Y-координата: вертикальная позиция в пикселях
- Цвет пикселя: 0 = черный, 1 = белый

Один белый пиксель в верхнем левом углу можно создать так:

```
oled.pixel (0, 0, 1)
```

и используя

```
oled.invert (True).
```

цвета OLED инвертированы. Белое становится черным и наоборот. Чтобы вернуться к исходным цветам, можно использовать `oled.invert (False)`.

10.1 Графическое представление

В дополнение к пиксельным инструкциям доступны и другие графические команды. Горизонтальные и вертикальные линии можно рисовать с помощью `.hline()` или `.vline()`. Задается начальная позиция XY, а также длина и цвет линии.

Например, следующая программа рисует на дисплее концентрические прямоугольные рамки.:

```
# SSD1306_frames.py
from machine import Pin, I2C
import ssd1306

i2c = I2C(-1, scl=Pin(22), sda=Pin(21))
oled_width = 128
oled_height = 64
oled = ssd1306.SSD1306_I2C(oled_width, oled_height, i2c)

for n in [0,5,10,15,20,25]:
    oled.hline(n, n, oled_width-1-2*n, 1-2*n)
    oled.hline(n, oled_height-1-n, oled_width-1-2*n, 1-2*n)
    oled.vline(n, n, oled_height-1-2*n, 1-2*n)
    oled.vline(oled_width-1-n, n, oled_height-2*n, 1-2*n)
    oled.show()
```

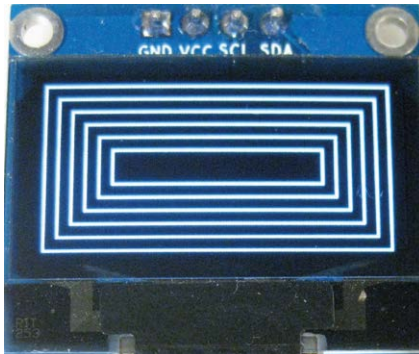


Рис.10.3: Рамка на OLED-дисплее.

Диагонали рисуются с помощью: линии (x1, y1, x2, y2, c) между двумя определенными точками (x1, y1) и (x2, y2). Параметр c управляет цветом нарисованной линии. Для простой графики растровые изображения могут записываться попиксельно в буфер дисплея. Следующая программа показывает соо

```
# SSD1306_bitmap_DEMO.py
from machine import Pin, I2C
import ssd1306
import urandom

i2c = I2C(-1, scl=Pin(22), sda=Pin(21))

oled_width = 128
oled_height = 64
```

```
oled = ssd1306.SSD1306_I2C(oled_width, oled_height, i2c)

# рамка
oled.hline(0, 0, oled_width-1, 1)
oled.hline(0, oled_height-1, oled_width-1, 1)
oled.vline(0, 0, oled_height-1, 1)
oled.vline(oled_width-1, 0, oled_height, 1)
oled.show()

ICON = [
    [0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0],
    [0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0],
    [0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0],
    [0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0],
    [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
    [0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0],
    [0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0],
    [0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0],
    [0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0],
    [0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0],
]

for n in range(12):
    xofs = urandom.randint(1, oled_width-12)
    yofs = urandom.randint(1, oled_height-12)
    for y, row in enumerate(ICON):
        for x, c in enumerate(row):
            oled.pixel(x+xofs, y+yofs, c)

oled.show()
```

со следующим результатом на дисплее:

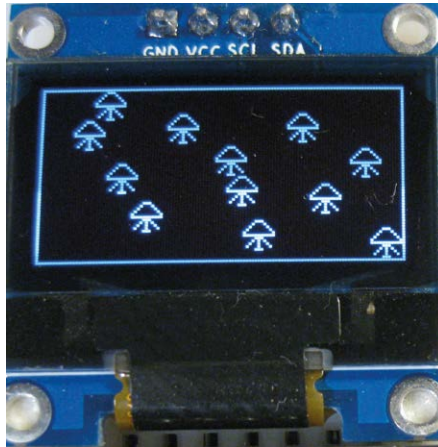


Рис.10.4: Графические элементы на OLED-дисплее.

10.2 OLED-дисплей в качестве плоттера данных

В дополнение к растровым изображениям данные измерений также могут отображаться графически на OLED-дисплее. Это означает, что вы больше не зависите от функции плоттера в Thonny, но также можете настроить автономные устройства с графическим выводом.

Следующая программа обеспечивает прокручивающийся дисплей, известный по профессиональным устройствам ЭКГ в больницах.

```
# Rolling_ECG_display.py

from machine import Pin, ADC, I2C
from time import sleep
import ssd1306

i2c = I2C(-1, scl=Pin(22), sda=Pin(21))

oled_width = 128
oled_height = 64
oled = ssd1306.SSD1306_I2C(oled_width, oled_height, i2c)

pot = ADC(Pin(34))
pot.atten(ADC.ATTN_11DB)      # Полный диапазон: 3,3 В
vMax = 3.4
dotPos_old = int(oled_height/2)

while True:
    pot_value = pot.read()
    voltage = 0.000816*pot_value + 0.037822
    # print(voltage)

    dotPos_new = int(voltage/vMax*oled_height)
```

```
oled.line(0, dotPos_new, 0, dotPos_old, 1)
oled.scroll(1, 0)
oled.line(0, dotPos_new, 0, dotPos_old, 0)

dotPos_old = dotPos_new
oled.pixel(0, int(oled_height/2), 1)
oled.show()
```

Для регистрации измеренных значений к входу АЦП 34 можно подключить потенциометр (см. раздел 9.2). После запуска программы значения напряжения непрерывно отображаются на дисплее.

Это стало возможным благодаря встроенной функции прокрутки. Через инструкцию

```
oled.scroll(1, 0)
```

все содержимое экрана перемещается на один пиксель. Если вы хотите записывать не отдельные пиксели, а непрерывную кривую, вам нужно соединить точки линиями. Для этого необходимы две переменные:

```
dotPos_old
```

и

```
dotPos_new
```

Они рисуют линию между текущим и последним измеренным значением. Затем изображение смещается на один пиксель. Наконец новая позиция переносится во временную память:

```
dotPos_old = dotPos_new
```

и игра начинается сначала. На следующем рисунке показан пример непрерывного изменения значений потенциометра:

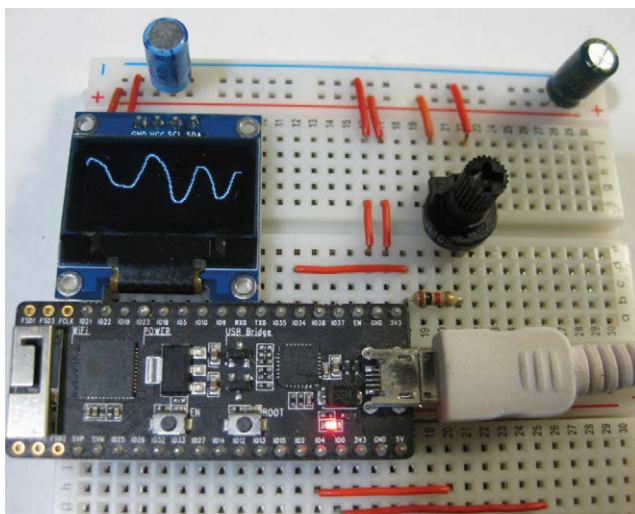


Рис.10.5: Вывод данных на OLED-дисплей.

Если у вас есть усилитель ЭКГ, вы также можете таким же образом записывать электрические сигналы человеческого сердца. В результате получается типичная электрокардиограмма, известная в отделениях интенсивной терапии в больницах:

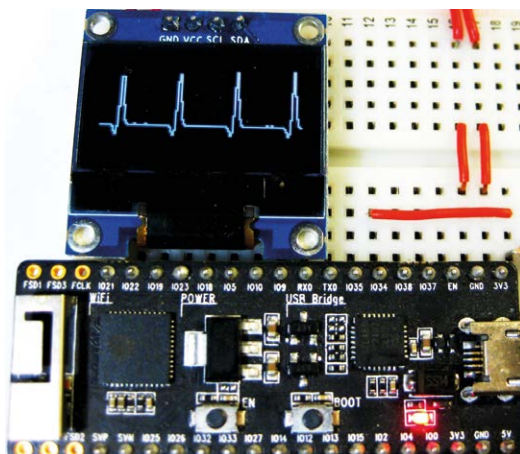


Рис.10.6: Сигнал ЭКГ на OLED-дисплее.

10.3 Пожалуйста, укажите точное время: цифровые часы с OLED-дисплеем

Еще одним полезным приложением для дисплея является вывод времени. Это превращает ESP32 вместе с SSD1307 в практичные цифровые часы. Функция `time()` из модуля `time` возвращает количество секунд с момента включения или сброса платы. Через переменную

```
time_offset=20*3600+00*60+0 # 44.MM.CC
```

можно указать время начала. В этом случае часы начинаются со времени 20:00:00. Подпрограмма "time_text()":

```
def time_text(time):
    secs=time%60
    mins=(time//60)%60
    hours=(time//3600)%24
    return "{:02d}:{:02d}:{:02d}".format(hours,mins,secs)
```

преобразует последовательные секунды в часы, минуты и секунды, то есть в обычное представление чч:мм:сс.

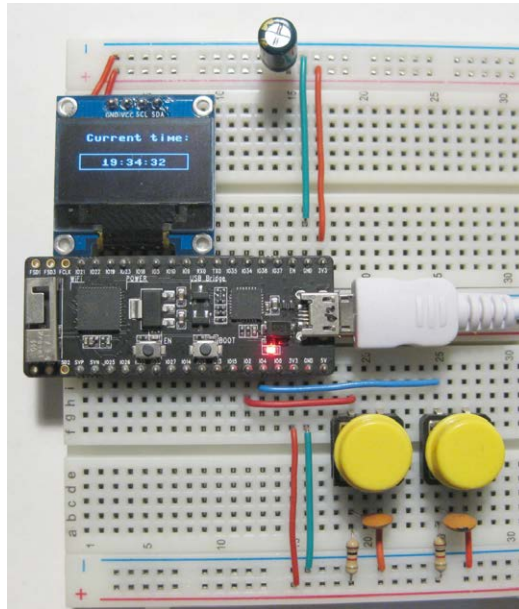


Рис.10.7: Цифровые часы с OLED-дисплеем

Часы можно настроить с помощью двух кнопок (Ta1 и Ta2) на портах 02 и 04.

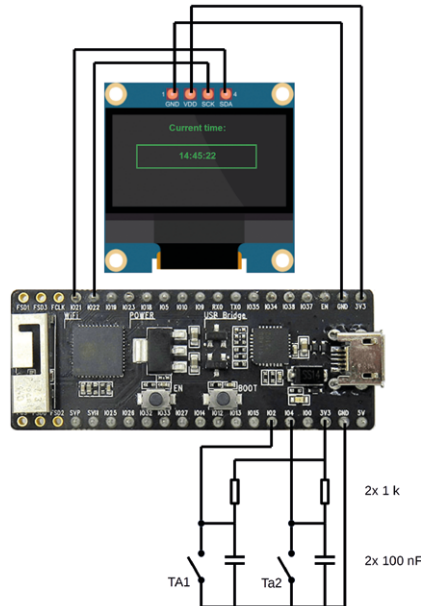


Рис.10.8: Принципиальная схема цифровых часов.

Кнопки снова шунтируются конденсаторами емкостью 100 нФ для устранения дребезга. Резисторы номиналом 1 кОм служат подтягивающими резисторами.

Связанные процедуры прерывания

```
def handle_interrupt_min(pin):
    global time_offset
    time_offset+=60
    time.sleep(.2)
```

и

```
def handle_interrupt_hr(pin):
    global time_offset
    time_offset+=3600
    time.sleep(.2)
```

убедитесь, что минуты (60 секунд) или часы (3600 секунд) увеличиваются на единицу при каждом нажатии кнопки. Полная программа цифровых часов выглядит так:

```
# setable_clock.py

from machine import Pin,I2C
import time
from ssd1306 import SSD1306_I2C
```

```
i2c = I2C(-1,scl=Pin(22),sda=Pin(21))
oled_width=128
oled_height=64
oled = SSD1306_I2C(oled_width,oled_height,i2c)

time_offset=20*3600+00*60+0 # hh:mm:ss
lin_hight=5
col_width=8

def handle_interrupt_min(pin):
    global time_offset
    time_offset+=60
    time.sleep(.2)

def handle_interrupt_hr(pin):
    global time_offset
    time_offset+=3600
    time.sleep(.2)

button_min = Pin(4, Pin.IN)
button_min.irq(trigger=Pin.IRQ_RISING, handler=handle_interrupt_min)

button_hr = Pin(2, Pin.IN)
button_hr.irq(trigger=Pin.IRQ_RISING, handler=handle_interrupt_hr)

def text_write(text, lin, col=0):
    oled.text(text, col*col_width, lin*lin_hight)

def time_text(time):
    secs=time%60
    mins=(time//60)%60
    hours=(time//3600)%24
    return "{:02d}:{:02d}:{:02d}".format(hours,mins,secs)

def show():
    oled.fill(0)
    text_write("Current time:",1,2)
    current_text = time_text(current_time)
    text_write(current_text,6,4)
    oled.rect(10,25,108,16,1)
    oled.show()

while True:
    current_time=time_offset+time.time()
    show()
```

10.4 Не только для спортсменов: секундомер

Также может быть легко реализован секундомер. Разрешение до 1/1000 секунды возможно даже без особых усилий. Аппаратную структуру, показанную на рис.10.8, можно использовать без изменений. Однако у кнопок появились новые функции:

Ta1: Start / Reset Ta2: Stop

Если вы хотите отказаться от подтягивающих резисторов на 1 кОм, вы также можете активировать внутренние подтягивающие резисторы:

```
start_button = Pin(2,Pin.IN,Pin.PULL_UP)
stop_button = Pin(4,Pin.IN,Pin.PULL_UP)
```

В результате получается следующая программа:

```
# stopwatch.py

from machine import Pin, I2C
import time
from ssd1306 import SSD1306_I2C

i2c = I2C(-1, scl=Pin(22), sda=Pin(21))
oled_width = 128
oled_height = 64
oled = SSD1306_I2C(oled_width,oled_height,i2c)

start_button = Pin(2,Pin.IN,Pin.PULL_UP)
stop_button = Pin(4,Pin.IN,Pin.PULL_UP)
start_time = 0
total_time = 0
active = False

lin_hight = 5
col_width = 8

led = Pin(25, Pin.OUT)

def text_write(text, lin, col=0):
    oled.text(text, col*col_width, lin*lin_hight)

def time_text(time):
    mills = time%1000
    secs = (time//1000)%60
    mins = (time//60000)%60
    hours = (time//3600000)%24
    return "{:02d}:{:02d}:{:02d}.{:03d}".format(hours,mins,secs,mills)
```

```

def show():
    oled.fill(0)
    total_text=time_text(total_time)
    text_write(total_text,6,2)
    oled.show()

while True:
    if not active:
        led.value(0)
        if not start_button.value():
            start_time = time.ticks_ms()
            active = True
    if active:
        led.value(1)
        total_time = time.ticks_ms() - start_time
        if not stop_button.value():
            active = False
            time.sleep(3)
            total_time=0
    show()

```

В основном цикле два состояния предполагаются через переменную «**active**». Когда секундомер остановлен, система ждет, пока не будет нажата кнопка запуска:

```
if not start_button.value():
```

Затем запускаются часы. С помощью второй кнопки часы останавливаются, и остановленное время можно прочитать. Повторное нажатие Ta1 сбрасывает время, и часы становятся доступными для новых показаний времени.

Кнопки настроены на «низкую активность». Это означает, что в нормальном состоянии на контактах 02 и 04 присутствует ВЫСОКИЙ сигнал (3,3 В). Когда клавиша нажата, она подтягивается к LOW (GND) т.е. к земле. Поэтому в программе необходимо использовать конструкцию `if not....`

10.5 Just Touch: секундомер с сенсорными кнопками

Секундомер также может управляться емкостными датчиками. Тогда кнопки можно сохранить и заменить простыми металлическими накладками. Кроме того, устранена проблемадребезга клавиш. На следующем рисунке показан вариант компоновки для этого. Защитные резисторы номиналом 1 кОм здесь не использовались. Однако, если на практике часы будут использоваться в течение более длительного периода времени, их следует установить, как показано на рисунке 9.9. Это снижает риск повреждения контроллера ESP электростатическим напряжением.

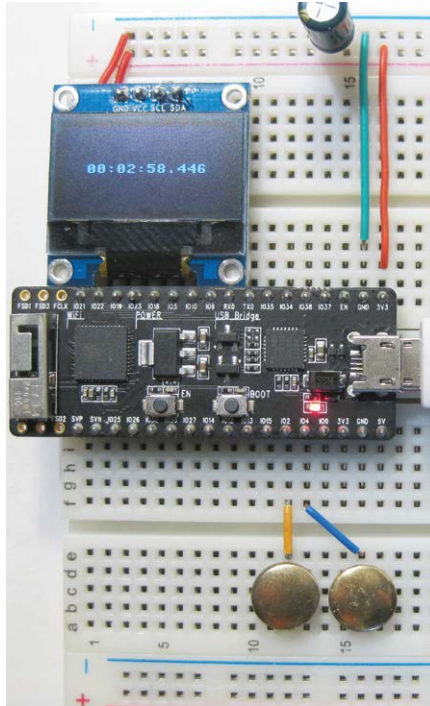


Рис.10.9: Секундомер с сенсорными кнопками.

В программе опрос кнопок заменен двумя подпрограммами для емкостных датчиков:

```
def start_touch():
    if t1.read() < 500:
        return True
    else:
        return False

def stop_touch():
    if t2.read() < 500:
        return True
    else:
        return False
```

При необходимости пороговое значение 500 можно адаптировать к реальным условиям. В основном цикле только запросы кнопок должны быть заменены новыми подпрограммами:

```
# stopwatch_touch_sensor.py

from machine import Pin, I2C, TouchPad
import time
from ssd1306 import SSD1306_I2C
```

```
i2c = I2C(-1, scl=Pin(22), sda=Pin(21))
oled_width = 128
oled_height = 64
oled = SSD1306_I2C(oled_width,oled_height,i2c)

t1 = TouchPad(Pin(2)) # start sensor pad
t2 = TouchPad(Pin(4)) # stop sensor pad

start_time = 0
total_time = 0
active = False

lin_hight = 5
col_width = 8

led = Pin(25, Pin.OUT)

def text_write(text, lin, col=0):
    oled.text(text, col*col_width, lin*lin_hight)

def time_text(time):
    mills = time%1000
    secs = (time//1000)%60
    mins = (time//60000)%60
    hours = (time//3600000)%24
    return "{:02d}:{:02d}:{:02d}.{:03d}".format(hours,mins,secs,mills)

def show():
    oled.fill(0)
    total_text=time_text(total_time)
    text_write(total_text,6,2)
    oled.show()

def start_touch():
    if t1.read() < 500:
        return True
    else:
        return False

def stop_touch():
    if t2.read() < 500:
        return True
    else:
        return False
```

```

while True:
    if not active:
        led.value(0)
        if start_touch():
            start_time = time.ticks_ms()
            active = True
    if active:
        led.value(1)
        total_time = time.ticks_ms() - start_time
        if stop_touch():
            active = False
            time.sleep(3)
            total_time=0
    show()

```

Это приложение демонстрирует, как можно легко и с пользой использовать встроенную функцию сенсорного датчика ESP32.

10.6 Отличный климат с датчиком BME280!

OLED-дисплеи также идеально подходят для вывода значений датчиков, как показано на следующем рисунке:

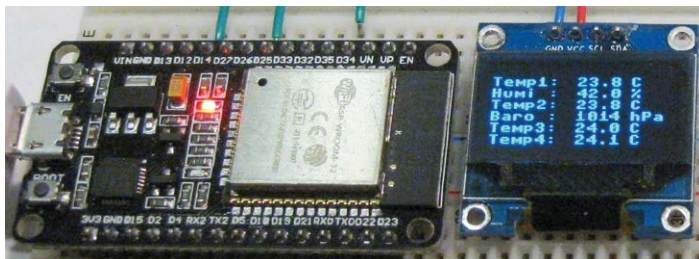


Рис.10.10: Значения датчиков на OLED-дисплее.

Следующая программа демонстрирует, как значения датчика BME280 (см. раздел 9.19) могут отображаться на дисплее. Это делает вас независимым от ПК или ноутбука в качестве терминала данных:

```

# BME280 Temp_Humit_Press-sensor_to_SSD1306.py

from machine import Pin, I2C
from time import sleep
import ssd1306
import BME280

# ESP32 - Pin assignment
i2c = I2C(scl=Pin(22), sda=Pin(21), freq=10000)

```

```
# OLED setup
oled_width = 128
oled_height = 64
oled = ssd1306.SSD1306_I2C(oled_width, oled_height, i2c)

while True:
    oled.fill(0)
    oled.text(',BME280 climate', 0, 0)
    bme = BME280.BME280(i2c=i2c)
    temp = bme.read_temperature()/100
    hum = bme.read_humidity()/1024
    pres = bme.read_pressure()/25600

    print(Temp: %7.2f' %temp)
    print(Humi: %7.2f' %hum)
    print(Pres: %7.2f' %pres)
    print()

    temp_string=str(Temp:%7.1f' %temp)
    hum_string =str(Humi:%7.1f' %hum)
    pres_string=str(Pres:%7.1f' %pres)

    oled.text(temp_string, 0, 10)
    oled.text(', C', 90 ,10)
    oled.text(hum_string, 0, 20)
    oled.text(', %', 90 ,20)
    oled.text(pres_string, 0, 30)
    oled.text(', hPa', 90 ,30)
    oled.show()

    sleep(1)
```

Если ожидается, что ESP32 запустится автоматически после прерывания питания, программа должна быть перезагружена как [main.py](#) вместе с файлом [dummy-boot.py](#). Таким образом, теперь вы можете создавать небольшие и компактные устройства, которые могут выполнять полезные задачи в повседневной жизни.

Обратитесь к Разделу 14.6 для получения информации о методах передачи климатических данных в Интернет через WLAN.

Глава 11 • Светодиодные матрицы и большие дисплеи

После подробного описания использования OLED-дисплеев в последней главе в следующих разделах основное внимание будет уделено управлению другим очень популярным дисплеем с использованием MicroPython. С помощью так называемых матричных дисплеев вы также можете отображать числа, буквы, символы и графику.

Основным отличием от OLED-дисплея является достижимая яркость и размер дисплея. Размер светодиодных матриц может достигать нескольких метров. С помощью современных светодиодов также можно достичь очень высоких уровней яркости. Поэтому этот вариант дисплея часто используется для дисплеев с большой площадью или рекламных щитов на оживленных площадях, вокзалах или аэропортах. В залах фондовых бирж мира текущие финансовые данные появляются на так называемых «живых тикерах».

Точечные матрицы особенно популярны в Азии, поскольку их также можно использовать для представления иероглифов, как показано на следующем рисунке:



Рис.11.1: Светодиодные точечные матрицы очень популярны в Азии.

Широко используются светодиодные матрицы с $5 \times 7 = 35$ светодиодами. Но также доступны различные другие матрицы. Также часто встречаются конструкции с 8×8 светодиодами.

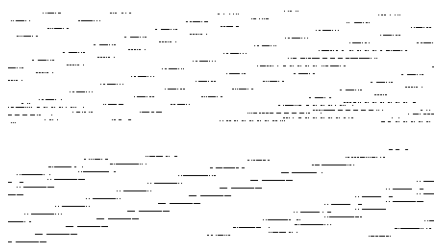


Рис.11.2: Принцип точечно-матричного управления.

Прямое управление матрицей 8×8 потребовало бы 65 соединений, поскольку необходимо было бы подключить 64 штыревых соединения и общий катод. При использовании матрицы требуется значительно меньше соединений. Здесь 8 светодиодов соединены в один столбец и один ряд. Таким образом, необходимо всего $8 + 8 = 16$ соединений. На рисунке выше показан принцип работы матрицы 3×4 . Вместо 13 соединений требуется только 7 соединений для индивидуального управления светодиодами.

Например, для управления вторым светодиодом во второй линии все линейные соединения, кроме второй, должны быть установлены на HIGH -ВЫСОКИЙ уровень. Однако второе соединение должно иметь потенциал GND. В случае столбцов ВЫСОКИЙ потенциал может присутствовать только во втором столбце.

Непосредственное управление точечными матрицами требует значительной части ресурсов контроллера. Если также необходимо записывать значения датчиков или управлять исполнительными механизмами, даже мощный контроллер ESP32 быстро достигает своих пределов. Управление большими дисплеями с сотней и более светодиодов также быстро становится проблемой, с одной стороны, из-за требуемой вычислительной мощности, а с другой — из-за ограниченного количества доступных контактов на контроллере.

Здесь могут помочь экономичные драйверы дисплея, такие как MAX7219. Эти устройства имеют SPI-совместимый интерфейс и, таким образом, могут управлять дисплеями с матричными элементами до $8 \times 8 = 64$ с помощью всего трех цифровых контактов. Драйверы доступны в виде готовых модулей. На следующем рисунке показан пример:

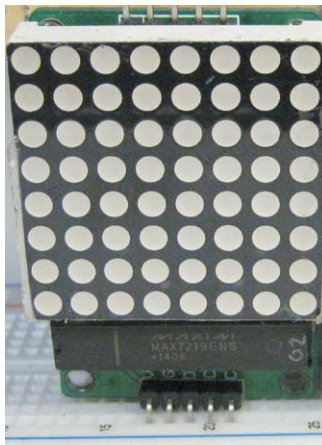


Рис.11.3: Светодиодный матричный модуль.

Подключить модули драйверов к ESP32 очень просто. Должны быть подключены только три контакта SPI к плате контроллера:

- MAX7219_data (DIN) Port D02
- MAX7219_load (CS) Port D05
- MAX7219_clock (CLK)→ Port D04
- MAX7219_GNDESP31 GND

Из-за относительно высокого энергопотребления целесообразно обеспечить внешний источник питания. Этот метод позволяет настроить дисплей практически любого размера. Контроллер почти не перегружается, так как по шине SPI приходится отправлять только отдельные команды. Кроме того, достаточное количество контактов остается свободным для управления внешними датчиками или другими периферийными устройствами.

Теперь ничто не мешает реализации широкоформатных дисплеев, например, в рекламных целях или на спортивных мероприятиях.

Ссылку на библиотеку Python для управления микросхемами Maxim можно найти в папке с кодами. После загрузки файла драйвера "Max7219.py" в контроллер доступны следующие инструкции:

```
spi = SPI(1, baudrate=10000000, polarity=1, phase=0, sck=Pin(CLK), mosi=Pin(DIN))
ss = Pin(CS, Pin.OUT)
```

для приведенного выше назначения контактов установите

- CLK = 4
- DIN = 2
- CS = 5

Далее можно создать дисплейный «объект»:

```
display = max7219.Matrix8x8(spi, ss, MN)
```

при этом для MN необходимо ввести количество используемых матричных элементов. После этого текст и графика могут быть разработаны с помощью следующих команд:

<code>display.pixel(x,y,1)</code>	установить пиксель в координатах x, y
<code>display.pixel(x,y,0)</code>	удалить пиксель с координатой x,y
<code>display.hline(x,y,l,1)</code>	горизонтальная линия от x,y - длина
<code>display.vline(x,y,l,1)</code>	вертикальная линия от x,y - длина
<code>display.line(a,b,c,d,1)</code>	строка от a,b до c,d
<code>display.rect(a,b,c,d,1)</code>	прямоугольник с углами a, b, c, d
<code>display.text('text',x,y,1)</code>	текст в позиции x,y
<code>display.scroll(x,0)</code>	прокрутить на x пикселей
<code>display.show()</code>	обновить дисплей

Это позволяет создавать эффективную и легко читаемую рекламу событий с расстояния в несколько метров.

11.1 Светодиодная матрица в действии

В следующем коде показан пример дисплея с шестью матричными элементами 8 x 8:

```
# LED_matrix_test.py

import max7219
from machine import Pin, SPI

spi = SPI(1, baudrate=10000000, polarity=1, phase=0, sck=Pin(4),
mosi=Pin(2))
ss = Pin(5, Pin.OUT)
```

```
display = max7219.Matrix8x8(spi, ss, 6)
display.text('Python',0,0,1)
display.show()
```

After loading the program onto the ESP controller, the text is shown on the display:



Рис.11.4: Точечный матричный дисплей с шестью матричными элементами 8 x 8.

Дисплеи и тексты могут быть не статичными. Даже движущуюся графику можно реализовать без проблем. В следующем разделе показан соответствующий пример.

11.2 Запуск анимированных текстов и графики

Функцию прокрутки также можно использовать для создания запущенных скриптов и тому подобного. Это позволяет выводить более длинные тексты на маленькие или короткие дисплеи. Следующая программа позволяет надписи «Python» проходить справа налево по точно-матричному дисплею:

```
# LED_matrix_ticker.py

import max7219
from machine import Pin, SPI
from time import sleep

spi = SPI(1,baudrate=100000000, polarity=1, phase=0, sck=Pin(4), mosi=Pin(2))
ss = Pin(5,Pin.OUT)

speedFactor=0.05
pixelDistance=7
display = max7219.Matrix8x8(spi, ss, 6)

def moveLeft(Pixel):
    for i in range(Pixel):
        display.scroll(-1,0)
        sleep(speedFactor)
        display.show()

while True:
    display.text('P',40,0,1)
    moveLeft(pixelDistance)
    display.text('y',40,0,1)
    moveLeft(pixelDistance)
    display.text('t',40,0,1)
    moveLeft(pixelDistance)
    display.text('h',40,0,1)
```

```
moveLeft(pixelDistance)
display.text('o',40,0,1)
moveLeft(pixelDistance)
display.text('n',40,0,1)
moveLeft(pixelDistance)
display.text(' ',40,0,1)
moveLeft(pixelDistance)
```

Отдельные буквы генерируются с помощью

```
display.text('....',40,0,1)
```

у правого края дисплея. Далее они перемещаются влево на значение «pix-elDistance» с помощью функции `moveLeft`:

```
for i in range(8):
    display.scroll(-1,0)
    sleep(speedFactor)
```

Кажущаяся скорость бега может быть изменена с помощью значения «speedFactor».

Глава 12 • Физические вычисления: сервоприводы приносят движение в игру

«Дайте мне точку опоры, и рычагом я переверну весь мир». Говорят, что Архимед использовал это предложение, чтобы проиллюстрировать закон рычагов. Сегодня он мог бы сказать: «Дайте мне сервопривод, и я переверну весь мир!» Физические вычисления связаны со сбором данных об окружающей среде, а также контролем и регулированием механических переменных. Используя датчики, можно легко собирать и количественно оценивать важные данные. С другой стороны, управление механическими переменными обычно связано с большими усилиями.

Физические вычисления все чаще находят свое применение в проектах, таких как робототехника и при разработке автономных машин. Удивительные проекты, основанные на PhysCom, уже были реализованы с микроконтроллерами, включая самобалансирующихся роботов, автоматизированную высокоскоростную фотографию или широкий спектр проектов домашней автоматике.

Доступными приводами являются, например, электромагниты, двигатели постоянного тока, шаговые двигатели или серводвигатели. Если они должны управляться контроллером, требуются специальные меры для подавления индуктивных напряжений, такие как блокировочные конденсаторы и обратноходовые диоды.

Самый простой способ — управлять серводвигателями при изготовлении моделей. Они содержат полные приводные системы, включая систему управления двигателем и систему позиционирования. Последний состоит из редуктора и потенциометра. Потенциометр жестко соединен с осью двигателя через редуктор. Таким образом, положение ползунка потенциометра обеспечивает прямое изменение текущего положения оси двигателя. Контур управления обеспечивает точное позиционирование сервопривода с помощью управляющего сигнала. Таким образом, такие системы идеально подходят для управления микроконтроллерами.

12.1 Сервотестер

Метод широтно-импульсной модуляции уже применялся для генерации квазианалоговых сигналов. Это позволило плавно изменять яркость светодиодов. Для управления сервоприводом также используется ШИМ. На следующем рисунке показана соответствующая кривая сигнала и соответствующие положения сервоприводов.

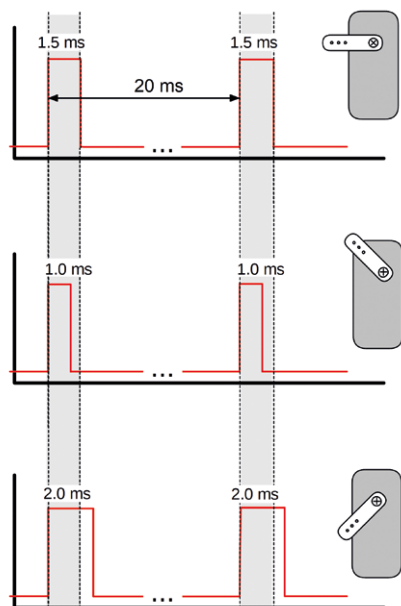


Рис.12.1: Сигнал управления сервоприводом.

Период импульсов ок. 20 мс, или частота повторения 50 Гц имеет второстепенное значение. Здесь допустимы и большие допуски. Информация о положении заключается в длительности управляющего импульса. Для этого установлен следующий квазистандарт:

Длит. импульса	Позиция
1 мс	Левая
1.5 мс	Средняя
2 мс	Правая

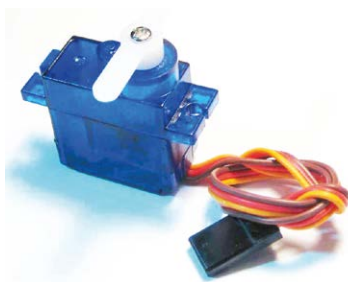


Рис.12.2: Сервопривод для использования в моделировании.

Благодаря встроенной управляющей электронике требуется только одна линия управления в дополнение к источнику питания. Его можно подключить к любому контакту ввода/вывода. Кабели стандартно маркируются следующими цветами:

Красный:	+5 V
Чёрный:	GND
Оранже. или корич.:	Управляющ. сигнал

Для более крупных сервоприводов рекомендуется использовать отдельный источник 5 В. Здесь можно использовать сетевой адаптер с рабочим током от 2А.

Для управления сервоприводами с помощью MicroPython требуется машинный модуль:

```
from machine import Pin
```

Его можно использовать для указания вывода, на котором должен работать сервопривод, например. контакт 18:

```
p4 = machine.pin(18)
```

Частота ШИМ установлена на 50 Гц, что соответствует периоду 20 мс:

```
servo = machine.PWM(p4, freq=50)
```

Теперь необходимо определить рабочий цикл (скважность). ШИМ имеет разрешение 10 бит, т.е. $2^{10} = 1024$ значения. Чтобы достичь среднего положения сервопривода, длительность импульса

$$1.5 \text{ ms} / (20 \text{ ms} / 1023) = 76.7 \text{ ms},$$

то есть значение 77 является наилучшим значением. Соответственно значения 52 и 102 приводят к правому и левому крайним положениям соответственно, так что эти параметры можно определить в программе:

```
# duty for servo is between 52...102
```

```
duty_min=52 # 52*20/1023 ms=1.02 ms
duty_mid=77 # 77*20/1023 ms=1.51 ms
duty_max=102 # 102*20/1023 ms=1.99 ms
```

Для этих значений рычаг сервопривода перемещается в следующие положения:

52:	левый стоп	(-45° position)
77:	Среднее положение	(0° position)
102:	правый стоп	(+45° position)

Поскольку в некоторых версиях сервоприводов большие отклонения приводят к повреждению, следует избегать значений, превышающих примерно 45°.

Сервотест теперь можно выполнить с помощью следующей программы Python:

```
# Servo_tst.py

import machine
from machine import Pin
from time import sleep

p4 = machine.Pin(18)
servo = machine.PWM(p4,freq=50)

# duty for servo is between 52...102
duty_min=52 # 52*20/1023 ms=1.02 ms
duty_mid=77 # 77*20/1023 ms=1.51 ms
duty_max=102 # 102*20/1023 ms=1.99 ms

while True:
    for pos in (duty_min,duty_mid,duty_max):
        print(pos)
        servo.duty(pos)
        sleep(1)
```

После подключения сервопривода (см. рисунок) и загрузки программы рычаг сервопривода должен перемещаться от минимального положения через центр к максимальному и обратно.

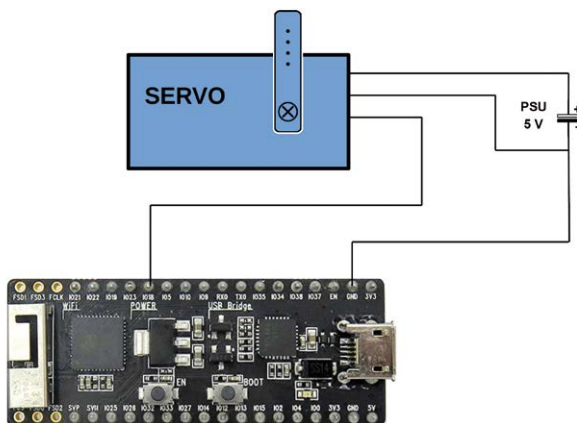


Рис.12.3: Сервопривод на ESP32

12.2 Сервотермометр с мегадисплеем

В дополнение к приложениям для построения моделей сервоприводы также можно использовать во многих других областях. Одним из интересных приложений являются широкоформатные дисплеи, которые легко читаются даже с больших расстояний. С OLED-дисплеем уже сложно читать цифры с расстояния более одного метра. С другой стороны, отображаемое значение сервопривода, представленного здесь, можно легко увидеть с нескольких метров.

Когда указатель прикреплен к сервоприводу, создается очень гибкий квазианалоговый индикаторный прибор. Затем с помощью шкалы можно отображать значения, как на классическом измерительном приборе. Программа для отображения температуры состоит из комбинации оценочного кода для DS18B20 из главы 9.11 и сервоуправления:

```
# servoDisplay_thermometer.py

import machine
from machine import Pin
import onewire
import ds18x20
from time import sleep

p4 = machine.Pin(18)
servo = machine.PWM(p4, freq=50)

# duty for servo is between 40 - 115
duty_min=52 # 52*20/1023 ms=1.02 ms
duty_mid=77 # 77*20/1023 ms=1.51 ms
duty_max=102 # 102*20/1023 ms=1.99 ms

ow=onewire.OneWire(Pin(25))      #инициализировать wire
ow.scan()
ds=ds18x20.DS18X20(ow)          #инициализировать ds18x20 object

while True:
    roms=ds.scan()               #сканировать ds18x20
    ds.convert_temp()            #преобразование температуры
    for rom in roms:
        T=ds.read_temp(rom)
    print(T)                     # тестовая выходная температура
    pos = int(duty_max-(duty_max-duty_min)*T/50)
    print(pos)                   # проверка положения сервопривода на выходе
    servo.duty(pos)
    sleep(1)
```

Для работы термометра требуется только один датчик DS18(x)20. Если к шине One-Wire подключено несколько измерительных преобразователей, то оценивается тот, у которого наиболее значимый идентификатор.

Контакт 18 был выбран для подключения сервосигнала. Кроме того, только линии питания должны быть подключены к GND и VIN (5 В) на плате ESP. Однако для сервопривода лучше использовать отдельный источник питания. На следующих рисунках показана принципиальная схема с использованием внешнего источника питания и рекомендуемая компоновка дисплея.

Рис.12.4: Принципиальная схема термометра с сервоприводом.

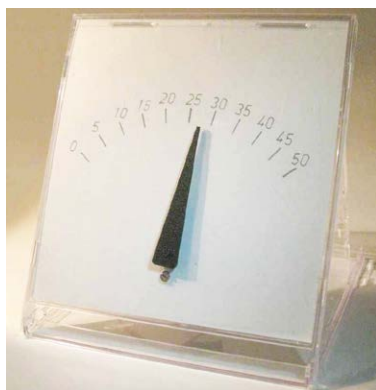


Рис.12.5: Настройка термометра со стрелкой с сервоприводом.

Глава 13 • RFID и беспроводная передача данных

RFID расшифровывается как радиочастотная идентификация. Считывающие устройства с этой функциональностью способны считывать данные с пассивных транспондеров, называемых RFID-метками, по беспроводной сети, т. е. по радио. Транспондер (передатчик/ответчик) может быть помечен специальным кодом, который делает его легко идентифицируемым. MicroPython предоставляет специальную библиотеку, позволяющую считывать эти коды по беспроводной сети (см. папку с кодами).

Это позволяет использовать ESP32 для реализации систем блокировки или подобных проектов, в которых человек должен идентифицировать себя с помощью передатчика.

Транспондеры доступны в самых разных формах и вариантах. На следующих иллюстрациях сначала показан модуль приемника, а затем две разные метки RFID. Одна из двух меток имеет форму брелока, другая состоит из пластиковой карты формата кредитной карты.



Рис.13.1: Модуль RFID.



Рис.13.2: RFID-карта и брелок.

Транспондеры могут поглощать энергию электромагнитного поля, создаваемого передатчиком. Это означает, что встроенный в транспондер передатчик может работать без собственного источника энергии. Таким образом, классическая RFID-метка не требует встроенных батарей или аккумуляторов и постоянно готова к использованию. Транспондер посылает обратно свой интегрированный код на фиксированной частоте. Затем этот код принимается и обрабатывается дальше.

Метки RFID также могут быть записаны с новыми кодами. Однако это требует очень много времени, поэтому данная процедура здесь далее не рассматривается. Большинство приложений также можно выполнять с кодами, уже присутствующими в тегах.

13.1 Чтение карт и чипов

С помощью библиотеки `mfr522.py` можно считать номер «уникального идентификатора» (UID), который содержит индивидуальные идентификационные данные RFID-метки. Программа Python для этого выглядит следующим образом:

```
# RFID_test.py
import mfr522

# RFID RX pinning
sck = 0
mosi= 2
miso= 4
rst = 5
cs  =14    # SDA на платах RFID-RC522

def do_read():
    rdr = mfr522.MFR522(sck, mosi, miso, rst, cs)
    print("Place card before reader")
    try:
        while True:
            (stat, tag_type) = rdr.request(rdr.REQIDL)
            if stat == rdr.OK:
                (stat, raw_uid) = rdr.anticoll()
                if stat == rdr.OK:
                    print("Card detected")
                    print("type: 0x%02x" % tag_type)
                    print(raw_uid[0], raw_uid[1], raw_uid[2], raw_uid[3])
                    print("")
    except KeyboardInterrupt:
        print("Bye")

while True:
    do_read()
```

Здесь сначала определяются контакты для подключения модуля RFID к ESP32. На следующем рисунке показана схема подключения:

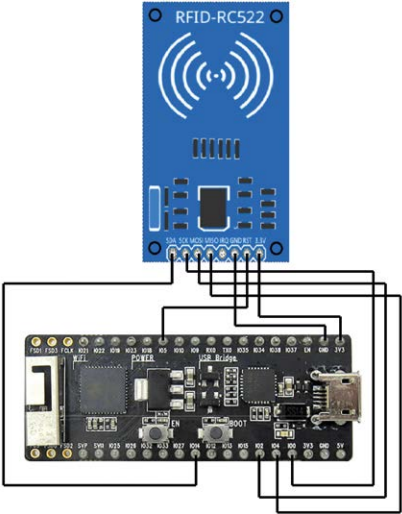


Рис.13.3: Модуль RFID на ESP32

В следующей таблице перечислены необходимые соединения:

RFID module	ESP32
SCK	I/O 00
MOSI	I/O 02
MISO	I/O 04
RST	I/O 05
CS (SDA)	I/O 14
RST	пустой

Подпрограмма, называемая `do_read()`, затем позаботится о чтении тегов.

После запуска программы карту или другую метку необходимо поднести к зоне действия RFID-модуля. После этого в консоли должен появиться вывод в следующем виде:

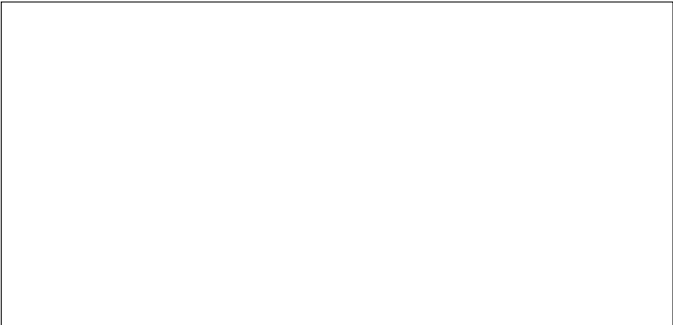


Рис.13.4: Идентификационные номера различных RFID-меток

13.2 Бесконтактный и безопасный: замок RFID

Технология RFID работает бесконтактно и, таким образом, очень гигиенична и защищена от загрязнений. Процессы включения или выключения могут запускаться через несколько сантиметров. Возможное применение: открытие замков, дверей и ворот с помощью карт с кодовым ключом. Поскольку эта процедура не требует прямого физического контакта, она также часто используется в больницах и домах престарелых по гигиеническим причинам.

Человеку, которому разрешено входить в определенную комнату, выдается его или ее собственный TAG в виде карты или брелка. Модуль RFID, прикрепленный к двери, открывает запорный механизм только в том случае, если в его зону действия попадает действующая метка RFID.

Есть несколько преимуществ по сравнению с замками с соответствующими ключами:

1. TAG значительно дешевле, чем ключи безопасности
2. Если ID-TAG утерян, соответствующий код удаляется из списка действительных идентификаторов.
3. Некоторые люди, например гости отеля, которые случайно забрали свой TAG домой, могут быть заблокированы. В этом случае блокировка соответствующего кода может восстановить безопасность.

В практических приложениях определенный UID можно сравнить с номером карты с авторизацией доступа. Следующая программа показывает пример:

```
# RFID_access_control.py

import mfrc522
from machine import Pin, PWM
from time import sleep

# Закрепление RFID RX
sck = 0
mosi = 2
miso = 4
rst = 5
cs = 14 # или SDA на платах RFID-RC522

ledRed = Pin(18, Pin.OUT)
ledGreen = Pin(19, Pin.OUT)

p4 = Pin(21)
servo = PWM(p4, freq=50)

duty_min=52 # 52*20/1023 ms=1.02 ms
duty_max=102 # 102*20/1023 ms=1.99 ms
```

```
servo.duty(duty_max)

def open_lock():
    servo.duty(duty_min)
    sleep(1)
    servo.duty(duty_max)

def do_read():
    rdr = mfrc522.MFRC522(sck, mosi, miso, rst, cs)

    print("")
    print("Place card before reader to read from address 0x08")
    print("")

    try:
        while True:
            (stat, tag_type) = rdr.request(rdr.REQIDL)
            if stat == rdr.OK:
                (stat, raw_uid) = rdr.anticoll()
                if stat == rdr.OK:
                    print("New card detected")
                    print(" - tag type: 0x%02x" % tag_type)
                    print(raw_uid[0], raw_uid[1], raw_uid[2], raw_uid[3])
                    print("")
                    if raw_uid[0]==124 and raw_uid[1]==24 and raw_uid[2]==62 and
raw_uid[3]==17:
                        print("OK!")
                        ledGreen.value(1)
                        open_lock()
                        sleep(1)
                        ledGreen.value(0)
                    else:
                        print("NOT OK!")
                        ledRed.value(1)
                        sleep(1)
                        ledRed.value(0)

    except KeyboardInterrupt:
        print("Bye")

while True:
    do_read()
```

После определения PIN-кодов для RFID-транспондера активируются два светодиода. Красный указывает на недействительную карту, зеленый загорается при обнаружении утвержденного UID.

Для запуска операции переключения можно использовать знания из предыдущей главы. Закрывающий механизм можно перемещать с помощью сервопривода для изготовления модели, здесь на штифте 21.

В процедуре "do_read" данные карты считываются через

```
rdr = mfrc522.MFRC522(sck, mosi, miso, rst, cs)
```

Карта, которая читает

```
print("Place card before reader to read from address 0x08")  
("Поместите карту перед считывателем для чтения с адреса 0x08")
```

Решающая строка программы

```
if raw_uid[0]==136 and raw_uid[1]==4 and raw_uid[2]==97 and raw_uid[3]==65:
```

сравнивает прочитанный UID с заданными данными в примере: 124 24 62 17

При совпадении срабатывает запирающий механизм. В противном случае кратковременно загорается только красный светодиод. Для считывания кодов карт можно использовать программу из прошлой главы.

На следующем рисунке показано возможное применение в виде ящика с замком, куда встроена вся электроника. Чтобы открыть коробку, поднесите к ней действующую RFID-метку.

В качестве меры предосторожности вы всегда должны иметь доступ к внешнему источнику питания. Если батарейки разряжены, коробку в противном случае придется открывать силой.

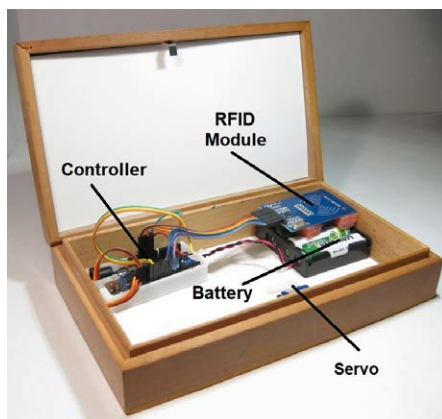


Рис.13.5: Хранилище, контролируемое RFID.

Глава 14 • MicroPython и Интернет вещей (IoT)

В этой главе плата ESP интегрируется в сеть. В дополнение к проводным соединениям большинство сетей теперь также имеют WLAN (беспроводную локальную сеть). Это означает, что преимущества компьютерной сети можно использовать без необходимости прокладки кабелей.

В принципе каждое соединение двух компьютеров уже представляет собой простую сеть. Если включается все больше и больше компьютеров, ноутбуков, ПК или планшетов, создаются все более сложные структуры. Их можно настроить с помощью различных сетевых компонентов, таких как концентраторы, роутеры и коммутаторы. Для этого требуется подходящий протокол передачи. В глобальном Интернете для этой цели используется протокол TCP-IP.

Стандартный протокол в Ethernet, а, следовательно, и в WLAN, называется TCP (протокол управления передачей). Этот протокол позволяет передавать данные через локальные или даже глобальные сети и обеспечивает практически безошибочную передачу данных.

IP означает интернет-протокол. Здесь происходит адресация пакетов данных. Поскольку в сети всегда есть несколько отправителей и получателей, необходима процедура, которая всегда передает пакеты данных, подлежащие передаче, правильному получателю. Протокол IP обеспечивает правильную адресацию пакетов данных. IP-адреса в нотации IPv4 состоят из 4 байтов по 8 бит каждый, т. е. всего 32 бита. Итак, теоретически,

$$2^{32} = 4,294,967,296$$

доступных адресов. Даже этого большого числа уже недостаточно для глобального Интернета. Приложения IoT и межмашинная связь, в частности, требуют, чтобы система была расширена до IPv6.

Однако домашние сети продолжают работать с 4-байтными адресами. В частном роутере каждому устройству назначается адрес вида

xxx.xxx.xxx.xxx

Например, IP-адрес 192.168.178.32.

Назначение IP-адреса также может выполняться полностью автоматически. Если ESP32 должен быть интегрирован в WLAN, необходимо знать только имя сети и код доступа. Дальнейшие подробности объясняются в следующих главах.

Примечание: далее различная информация, такая как IP-адреса, выделена серым цветом из соображений защиты данных, даже если это не обязательно очень конфиденциальная информация. Эту процедуру также следует понимать как указание на осторожное обращение с данными в целом.

Так называемый MAC-адрес (для управления доступом к среде) также играет важную роль. Это глобально уникальный идентификатор, который назначается каждому сетевому совместимому устройству. Он состоит из шести байтов. Первые три байта указывают идентификационный код производителя, за которым следует индивидуальный код, присвоенный производителем. Типичный MAC-адрес выглядит так:

22-2C-DA-3A-A4-1B

MAC-адрес требуется, если сетевые компоненты должны быть явно адресованы в интересах услуг, предлагаемых на более высоких уровнях связи. Поскольку это также относится к чипу ESP32, ему также назначается собственный MAC-адрес.

Если роутер домашней сети подключен к Интернету через линию DSL или даже оптоволокно, он называется «шлюзом». Для функции шлюза также назначается специальный IP-адрес. Когда пакеты данных отправляются на этот адрес, шлюз пересылает данные через провайдера в Интернет. Таким образом, можно обмениваться данными с контроллером через Интернет. Например, в следующих главах глобальная связь устанавливается через сервер ThingSpeak.

14.1 Для современных детективов: сетевой сканер

Когда полицейское радио было еще на 100% аналоговым, мошенники часто использовали так называемые «радиосканеры», чтобы подслушивать сообщения полиции. Внедрение цифровой радиопередачи и изоощренных методов шифрования положило конец этой деятельности. Однако, что касается WLAN, сканеры полностью легальны и могут использоваться кем угодно. В первом приложении плата ESP используется как сканер. Его можно использовать для обнаружения всех активных сетей поблизости. Функция

```
station.scan()
```

возвращает список с информацией обо всех локально доступных точках доступа WLAN:

- ssid: «Идентификатор набора услуг», т. е. имя сети WLAN.
- bssid: «Идентификация базового набора услуг» — соответствует MAC-адресу беспроводной точки доступа.
- channel: сетевой канал
- RSSI: «Индикатор уровня принимаемого сигнала»: уровень сигнала в дБ.
- authmode: режим аутентификации:
 - open
 - WEP
 - WPA-PSK
 - WPA2-PSK
 - WPA/WPA2-PSK
- hidden: скрытая сеть?
 - False – станция видна
 - True – станция скрыта

Соответственно, сканер MicroPython WLAN может выглядеть так:

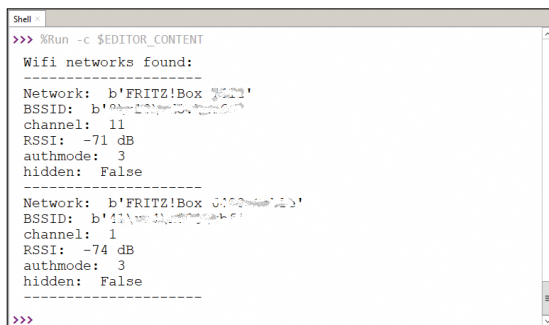
```
# boot.py : выполняется при каждой загрузке
# WLAN сканнер

import network
station = network.WLAN(network.STA_IF)
station.active(True)

n=station.scan()
print("Wifi networks found:")
print("-----")

for i in n:
    print("Network: ", i[0])
    print("BSSID: ", i[1])
    print("channel: ", i[2])
    print("RSSI: ", i[3], "dB")
    print("authmode: ", i[4])
    print("hidden: ", i[5])
    print("-----")
```

Данные выводятся на консоль:



```
Shell >
>>> %Run -c $EDITOR_CONTENT
Wifi networks found:
-----
Network:  b'FRITZ!Box
BSSID:  b'42:42:42:42:42:42
channel:  11
RSSI:  -71 dB
authmode:  3
hidden:  False
-----
Network:  b'FRITZ!Box
BSSID:  b'42:42:42:42:42:42
channel:  1
RSSI:  -74 dB
authmode:  3
hidden:  False
-----
>>>
```

Рис.14.1: Сканирование WLAN в оболочке.

Программа обнаруживает все сети WLAN в пределах досягаемости. Эта функция очень полезна, например, если вы хотите быстро и легко проверить, доступна ли WLAN в определенном месте.

14.2 Подключено, но нет кабелей: WLAN

Интерфейс WLAN ESP32 также можно использовать для обмена информацией с сетью. Это позволяет управлять устройствами, системами или устройствами по беспроводной сети через WLAN с помощью веб-сервера.

Чтобы сервер на основе ESP автоматически перезапустился после возможной перезагрузки, необходимы два файла

- boot.py
- main.py

Файл **boot.py** содержит части программы, необходимые для запуска сетевого соединения. К ним относятся импорт библиотек, информация для входа в сеть и установления соединения с локальной WLAN:

```
# boot.py für WLAN connection

try:
    import usocket as socket
except:
    import socket

import network
from machine import Pin
import dht

import esp
esp.osdebug(None)

import gc
gc.collect()

ssid = ,xxxxxxxxxxxxx'
password = ,12345678901234567890'

station = network.WLAN(network.STA_IF)

station.active(True)
station.connect(ssid, password)

while station.isconnected() == False:
    pass

print(,Connection successful')
print(station.ifconfig())
```

Информация для доступа к сети

- `ssid = ,xxxxxxxxxxxxx'`
- `password = ,12345678901234567890'`

должны быть заменены соответствующими данными внутренней WLAN. Это можно найти, например, на задней панели роутера или в его инструкции по эксплуатации.

Веб-серверы работают с так называемыми «сокетами» Python API. Сокеты — это программные блоки, которые служат конечными точками связи. Они обеспечивают основу для обмена данными с другими программами или компьютерами. Коммуникационные партнеры могут быть установлены на том же компьютере, на сетевом элементе или на компьютерах, доступных через сеть. Связь через сокеты может быть двунаправленной. Таким образом, данные могут быть одновременно получены и отправлены. Таким образом, сокеты образуют независимый от платформы стандартизированный интерфейс между сетевым протоколом и базовым прикладным программным обеспечением. Библиотека сокетов для ESPcontroller импортируется следующим образом (см. `boot.py`):

```
try:
    import usocket as socket
except:
    import socket
```

Итак, если более универсальный модуль `usocket` недоступен, используется стандартная версия.

Затем импортируется сетевая библиотека. Она позволяет ESP32 обмениваться данными с сетями WLAN. Для беспроводного управления пинами GPIO требуется только класс контактов машинного модуля:

```
from machine import Pin
```

Строки

```
import esp
esp.osdebug(None)
```

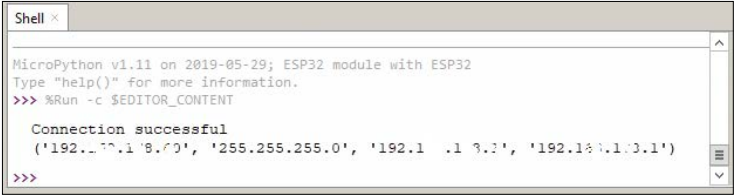
отключают отладочные сообщения. Если требуется вывод соответствующей информации, строки могут быть опущены или закомментированы.

Затем активируется так называемый «сборщик мусора» (`gc`). Он представляет собой возможную форму автоматического управления памятью. Это освобождает место в памяти, которое может быть занято объектами, которые больше не используются программой.

Имея информацию для входа в WLAN (сетевой SSID и пароль используемого роутера), ESP32 может установить соединение с локальным роутером. Затем ESP32 настраивается и активируется как станция WLAN:

```
station = network.WLAN(network.STA_IF)
station.active(True)
```

После этого контроллер устанавливает соединение с роутером. Цикл `while` гарантирует, что программа не будет продолжена, пока модуль ESP не подключен к локальной сети WLAN. После успешного подключения выводятся сетевые параметры, включая IP-адрес ESP32:



```
Shell x
MicroPython v1.11 on 2019-05-29; ESP32 module with ESP32
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT

Connection successful
('192.168.1.8', '255.255.255.0', '192.168.1.1', '192.168.1.1')
>>>
```

Рис.14.2: Вывод сетевых параметров

14.3 Переключение и управление с помощью веб-сервера

После того, как плата установила связь с WLAN, можно настроить независимый веб-сервер. Это можно использовать, например, для управления различными выходами. Сервер должен предлагать следующие возможности:

- Управление отдельными каналами для GPIO платы ESP.
- Доступ через IP-адрес.
- Управление через любой браузер в локальной домашней сети.
- Нажатие на кнопки веб-сервера должно немедленно изменить статус активных каналов.
- Обратная связь путем отображения текущего состояния порта на веб-странице.

Для тестирования сервера к используемым портам (22 и 23) сначала подключаются два светодиода. При необходимости светодиоды, конечно, также могут быть заменены реле или другими электронными компонентами, такими как силовые транзисторы или симисторы и т. д., для переключения более высоких выходных уровней. После того, как в файле `boot.py` уже установлен контакт с сетью, построение веб-страницы сервера можно осуществить в файле `main.py`. Здесь объекты выводов сначала определяются для управляемых GPIO. В следующем примере «светодиод 1» должен быть подключен к контакту 22, а «светодиод 2» — к

```
from machine import Pin
led1 = Pin(22,Pin.OUT)
led2 = Pin(23,Pin.OUT)
```

Веб-страница имеет задачу отображать текущие состояния GPIO. Перед созданием HTML-текста необходимо проверить состояния светодиодов. Они записываются в переменную `gpio_state`:

```
if led1.value()==1:
    gpio_state1="ON"
else:
    gpio_state1="OFF"
if led2.value()==1:
```

```

        gpio_state2="ON"
    else:
        gpio_state2="OFF"

```

Затем определяется конструкция веб-страницы. Основные моменты этой конструкции будут кратко объяснены ниже. В первом блоке HTML основные параметры, такие как

- заголовок
- размер
- шрифт

страницы определяются:

```

html = """<html><head> <title>WLAN server</title> <meta name="viewport" con-
tent="width=device-width, initial-scale=1">
    <link rel="icon" href="data:,">

    <style>html{font-family: Arial; display:inline-block; margin: 0px auto;
text-align: left}
    h1{color: black; padding: 2vh; font-size: 20px;}

```

Затем следует конструкция двух виртуальных кнопок красного и зеленого цветов:

```

.buttonGreen{display: inline-block; background-color: green; border: none;
font-size: 20px; margin: 10px; cursor: pointer;}
.buttonRed{display: inline-block; background-color: red; border: none;
font-size: 20px; margin: 10px; cursor: pointer;}

```

Теперь веб-страницу можно вывести с помощью

- Заголовок
- информация о состоянии
- кнопки

следующее:

```

</style></head><body> <h1>WLAN server V 1.2</h1; text-align: left>

<p>Channel 1: "" + gpio_state1 + ""
</p><p><a href="/led1=on"><button class="buttonGreen">ON</button></a>
    <a href="/led1=off"><button class="buttonRed">OFF</button></a></p>

<p>Channel 2: "" + gpio_state2 + ""
</p><p><a href="/led2=on"><button class="buttonGreen">ON</button></a>
    <a href="/led2=off"><button class="buttonRed">OFF</button></a></p>
</body></html>"""

```

После создания HTML-кода для веб-сайта требуется прослушивающий сокет, чтобы ожидать входящие запросы и отправлять HTML-текст в качестве ответа. Для этого создается новый объект сокета "s" с соответствующим типом сокета (STREAM TCP). Затем этот объект сокета может быть привязан к адресу (сетевому интерфейсу и номеру порта) с помощью метода `bind()`.

```
s=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(('', 80))
s.listen(5)
```

Пустая строка «» предоставляет IP-адрес локального хоста и, следовательно, текущий IP-адрес ESP32. Порт номер 80 служит стандартным портом.

Теперь сервер может подключаться к сети. Соответствующая строка позволяет серверу установить соединение «read» (чтение). Аргумент соответствующего метода `listen()` определяет максимальное количество соединений в очереди, максимум пять.

Цикл `while` ожидает запросов. Когда клиент подключается, сервер вызывает метод `accept()`, чтобы принять соединение. Затем он сохраняет новый объект сокета для приема и отправки данных для переменной `conn`. Адрес клиента сохраняется для подключения к серверу для переменной «addr».

Через строку

```
print('Got a connection from %s' % str(addr))
```

адрес клиента выводится на консоль для управления. Обмен данными между клиентом и сервером происходит с помощью методов `send()` и `recv()`. Метод `recv()` получает данные из клиентского сокета. Аргумент метода указывает максимальное количество данных, которое может быть получено за один раз.

Переменная «response» (ответ) содержит текст HTML, возвращаемый функцией `web_page()`. Init выполняется поиск клавиши "ledx=on/off" и соответственно включается или выключается светодиод.

Наконец, ответ отправляется клиенту сокета с помощью методов `send()` и `sendall()`, а сокет закрывается с помощью `conn.close()`.

Весь файл `main.py` для создания веб-страницы сервера выглядит следующим образом:

```
# main.py: WebServer_WLAN_switch
from machine import Pin
led1 = Pin(22,Pin.OUT)
led2 = Pin(23,Pin.OUT)
def web_page():
```

```

    if led1.value()==1:
        gpio_state1="ON"
    else:
        gpio_state1="OFF"

    if led2.value()==1:
        gpio_state2="ON"
    else:
        gpio_state2="OFF"

    html = """<html><head> <title>WLAN server</title> <meta name="viewport"
content="width=device-width, initial-scale=1">
    <link rel="icon" href="data:,">

    <style>html{font-family: Arial; display:inline-block; margin: 0px auto;
text-align: left}
    h1{color: black; padding: 2vh; font-size: 20px;}

    .buttonGreen{display: inline-block; background-color: green; border: none;
font-size: 20px; margin: 10px; cursor: pointer;}
    .buttonRed{display: inline-block; background-color: red; border: none;
font-size: 20px; margin: 10px; cursor: pointer;}

</style></head><body> <h1>WLAN server V 1.2</h1> text-align: left>

<p>Channel 1: "" + gpio_state1 + ""
</p><p><a href="/led1=on"><button class="buttonGreen">ON</button></a>
    <a href="/led1=off"><button class="buttonRed">OFF</button></a></p>

<p>Channel 2: "" + gpio_state2 + ""
</p><p><a href="/led2=on"><button class="buttonGreen">ON</button></a>
    <a href="/led2=off"><button class="buttonRed">OFF</button></a></p>
</body></html>"""

    return html

s=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(('', 80))
s.listen(5)

while True:
    conn, addr = s.accept()
    print(,Got a connection from %s' % str(addr))
    request = conn.recv(1024)
    request = str(request)
    print(,Content = %s' % request)

```

```
led1_on = request.find(/led1=on')
led1_off = request.find(/led1=off')
led2_on = request.find(/led2=on')
led2_off = request.find(/led2=off')
if led1_on==6:
    print(,LED ON')
    led1.value(1)
if led1_off==6:
    print(,LED OFF')
    led1.value(0)
if led2_on==6:
    print(,LED ON')
    led2.value(1)
if led2_off==6:
    print(,LED OFF')
    led2.value(0)

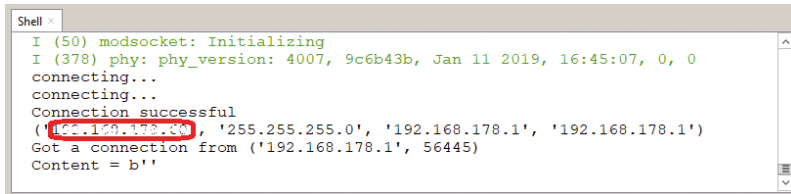
response = web_page()
conn.send(response)
conn.close()
```

14.4 Веб-сервер WLAN в действии

В веб-сервер входит только плата ESP с одним светодиодом на портах GPIO 22 и 23 и соответствующие последовательные резисторы (2 x 150 Ом).

Рисунок 14.3: Веб-сервер ESP32.

После загрузки файлов [main.py](#) и [boot.py](#) в папку «device» ESP32 можно нажать кнопку ESP EN/RST. Через несколько секунд будет установлено соединение с локальным роутером WLAN. Связанный IP-адрес отображается в оболочке.



```

Shell <
I (50) modsocket: Initializing
I (378) phy: phy_version: 4007, 9c6b43b, Jan 11 2019, 16:45:07, 0, 0
connecting...
connecting...
Connection successful
('192.168.178.1', '255.255.255.0', '192.168.178.1', '192.168.178.1')
Got a connection from ('192.168.178.1', 56445)
Content = b''

```

Рис.14.4: Настройка подключения и IP-адрес платы ESP32.

Для вызова веб-страницы требуется компьютер, подключенный к той же локальной сети. На этом компьютере можно открыть любой браузер с указанным выше IP-адресом ESP в поле ввода (см. изображение).

Рис.14.5: Веб-сайт коммутатора WLAN.

После нажатия виртуальной кнопки «ON» загорается соответствующий светодиод. Кроме того, сразу обновляется статус GPIO на странице.

14.5 Считывание данных датчика через WLAN

Помимо коммутации устройств, одной из важнейших задач веб-сервера является считывание и передача данных датчиков. Например, три аналоговых канала должны быть отправлены веб-клиенту через WLAN. Чтобы гарантировать, что заданные значения применяются к каналам, они подключены к потенциометрам. В качестве альтернативы можно использовать любые датчики с аналоговыми выходами, такие как фотодиоды или аналоговые датчики температуры (см. главу 9).

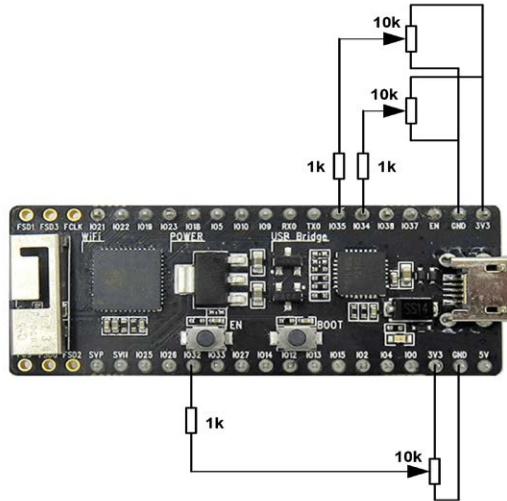


Рис.14.6: Веб-сервер для передачи аналоговых каналов.

Соединение с WLAN устанавливается, как обычно, через файл boot.py. Следовательно, он может быть принят без изменений. После импорта вывода и АЦП каналы АЦП определяются в файле main.py:

```
adcs = [ADC(Pin(i)) for i in (32,34,35)]
```

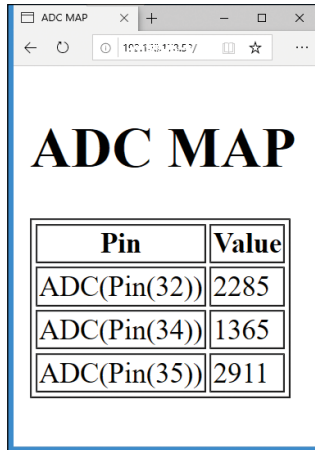
Danach erfolgt der Aufbau der Web-Page:

```
html = """<!DOCTYPE html>
<html>
  <head> <title>ADC MAP</title> </head>
  <body> <h1>ADC MAP</h1>
    <table border="1"> <tr><th>Pin</th><th>Value</th></tr> %s </table>
  </body>
</html>
"""
```

в комплекте с названием, заголовком и предварительно отформатированной таблицей. Затем сокет снова активируется. В основном цикле каналы, определенные в «adcs», выводятся через

```
rows = [, <tr><td>%s</td><td>%d</td></tr>' % (str(p), p.read()) for p in
adcs]
```

с помощью созданной таблицы. На следующем рисунке показан пример для трех аналоговых значений:



Pin	Value
ADC(Pin(32))	2285
ADC(Pin(34))	1365
ADC(Pin(35))	2911

Рис.14.7: Аналоговые измеренные значения в браузере

Однако описанная здесь процедура позволяет распространять данные только в внутренней сети дома через локальную или беспроводную локальную сеть. Доступ извне невозможен без дальнейших изменений. Настройки по умолчанию домашнего роутера обычно запрещают внешний доступ к внутренним сетевым ресурсам. Это также имеет смысл, так как обычно нежелательно, чтобы неизвестные лица управляли собственной домашней автоматикой.

Специальные порты могут быть освобождены для внешнего доступа. Тогда можно было бы получить доступ к контроллеру в домашней сети из любой точки мира. Эта процедура, однако, требует глубоких знаний о том, как обращаться с соответствующими мерами предосторожности, чтобы исключить неправильное использование. Из-за этих сопутствующих рисков этот метод не будет здесь описываться.

Гораздо более простой и лучший метод представлен в главе 16 «Отправка данных в Интернет через ThingSpeak». Там бесплатная интернет-платформа используется для извлечения данных измерений из собственной домашней автоматике по всему миру. Кроме того, платформа предлагает широкие возможности для графического отображения измеренных значений и данных. Также предусмотрены статистические методы сбора и обработки данных.

Доступ к платформе можно регулировать и ограничивать с помощью различных настроек. Это обеспечивает разумный уровень безопасности, хотя, конечно, ни одна система не может предложить абсолютную защиту от нежелательного вторжения.

14.6 Запись параметров окружающей среды: Термометр/гигрометр WLAN

Если измеренные значения необходимо регистрировать из удаленных, изменяющихся или труднодоступных мест, желательно не прокладывать измерительные кабели. В этом случае радиопередача может быть методом выбора. Для этой цели можно использовать систему записи с поддержкой WLAN. Если должны передаваться реальные измеренные значения, только потенциометры из последней главы необходимо заменить подходящими датчиками. Например, для беспроводного дистанционного измерения температуры снова можно использовать настройку, показанную на рис. 9.12. В соответствующей программе передача данных WLAN объединена с программой термометра.

В качестве альтернативы можно использовать датчик DHT22 и резистор на 10 кОм. Подключение и оценка этого и других датчиков уже объяснялись в главе 9. Однако теперь зарегистрированные значения должны не только отображаться локально на дисплее, но и передаваться по беспроводной сети через WLAN.

Опять же, для этого проекта необходимы файлы `boot.py` и `main.py`. `Boot.py` идентичен версии для LED-сервера. В строке `main.py` теперь указан пин датчика вместо ножки светодиода:

```
sensor = dht.DHT22(Pin(14))
```

Температура и влажность считываются через

```
def read_sensor ():
```

Две переменные «temp» (температура) и «hum» (влажность) хранят данные о температуре и влажности, считанные с датчика.

При следующей конструкции try/except выполнение программы продолжается, даже если возникает исключение. Это предотвращает сбой веб-сервера, если данные не могут быть считаны с датчика. Если показания достоверны, подготавливается сообщение, содержащее показания температуры и влажности:

```
msg = (b '{0: 3.1f}, {1: 3.1f}'. format (temp, hum))
```

Наконец, данные возвращаются с помощью `return (msg)`. Если датчик не может быть прочитан, например, из-за того, что связь с измерительным блоком прервана, генерируется сообщение об ошибке ("Failed to read sensor - Не удалось прочитать датчик").

Функция `web_page()` снова возвращает HTML-страницу. Как и в случае со светодиодным сервером, здесь сначала определяется общая конструкция страницы. Затем создаются две подпрограммы для отображения температуры и влажности:

```
<p>
  <i class="fas fa-thermometer-half" style="color:#059e8a;"></i>
  <span class="dht-labels">Temperature</span>
  <span>"""+str(temp)+"""</span>
  <sup class="units">°C</sup>
</p>
<p>
  <i class="fas fa-tint" style="color:#00add6;"></i>
  <span class="dht-labels">Humidity</span>
  <span>"""+str(hum)+"""</span>
  <sup class="units">%</sup>
</p>
```

Затем выполняются шаги программы, уже рассмотренные в разделе 14.9, для передачи веб-страницы.

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(('', 80))
s.listen(5)
```

В цикле `while` вызывается функция `read_sensor()` для вывода значений датчика и обновления глобальных переменных `temp` и `hum`. Полный файл `main.py` выглядит так:

```
# main.py: ENVIRO Server DHT11

import dht
sensor = dht.DHT11(Pin(14))

def read_sensor():
    global temp, hum
    temp = hum = 0
    try:
        sensor.measure()
        temp = sensor.temperature()
        hum = sensor.humidity()
        if (isinstance(temp, int) and isinstance(hum, int)):
            msg = (b'{0:3.1f},{1:3.1f}'.format(temp, hum))
            print(msg)
            return(msg)
        else:
            return(Invalid sensor readings.')
    except OSError as e:
        return(Failed to read sensor.')

def web_page():
    html = """<!DOCTYPE HTML><html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <style>
    html { font-family: Arial; display: inline-block; margin: 0px auto;
text-align: center;    }
    h2 { font-size: 2.0rem; } p { font-size: 2.0rem; }
    .units { font-size: 2.0rem; }
    .dht-labels{ font-size: 2.0rem; vertical-align:bottom; padding-bottom:
0px; }
  </style>
</head>
<body>
  <h2>ESP32 ENVIRO Server</h2>
```

```

<p>
  <span class="dht-labels">Temperature:</span>
  <span>""+str(temp)+"</span>
  <span class="units">&deg;C</span>
</p>
<p>
  <span class="dht-labels">Humidity:</span>
  <span>""+str(hum)+"</span>
  <span class="units">%</span>
</p>
</body>
</html>""
    return html

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(('', 80))
s.listen(5)

while True:
    conn, addr = s.accept()
    print('Got a connection from %s' % str(addr))
    request = conn.recv(1024)
    print('Content = %s' % str(request))
    sensor_readings = read_sensor()
    print(sensor_readings)
    response = web_page()
    conn.sendall(response)
    conn.close()

```

После загрузки файлов на ESP32 можно снова запустить веб-браузер и ввести текущий IP-адрес ESP32. Данные измерений теперь будут отображаться в виде веб-страницы:

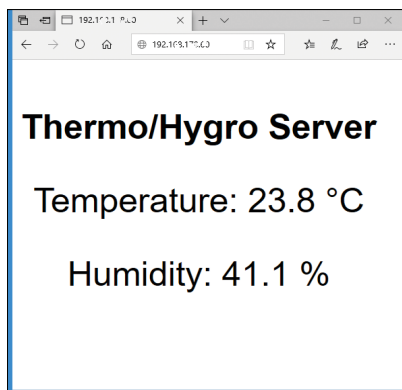


Рис.14.8: Сервер Thermo/Hygro работает и отображается в браузере.

Глава 15 • Просто и хорошо: протокол MQTT

В этой главе рассматриваются основы протокола MQTT. MQTT расшифровывается как Message Queuing Telemetry Transport. Это простая система публикации и подписки, с помощью которой можно публиковать и получать сообщения. Протокол был разработан, в том числе, для передачи данных с низкой пропускной способностью. Это делает его предпочтительным решением для приложений IoT. С помощью MQTT можно отправлять команды для управления выходами. Кроме того, также возможно считывать данные с сенсорного узла и публиковать их.

При разработке протокола была централизована необходимость иметь возможность устанавливать связь между несколькими устройствами без особых усилий. MQTT включает в себя следующие основные понятия:

- Publish/Subscribe - Опубликовать/подписаться
- Messages - Сообщения
- Topics - Темы
- Broker - Маклер

Первый подход — это система публикации и подписки. Это позволяет устройству публиковать сообщения по определенной теме. Кроме того, можно подписаться на определенную тему, чтобы получать сообщения.

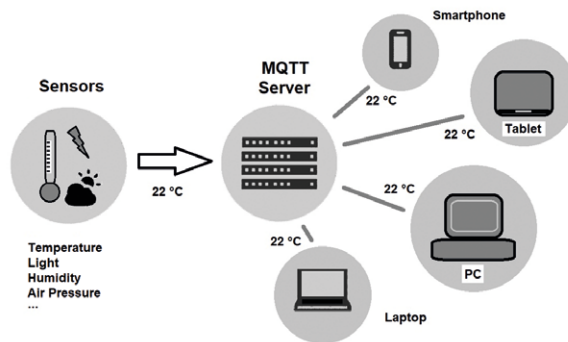


Рис.15.1: MQTT: основная процедура.

Сообщения — это информация, которой обмениваются различные устройства. Это могут быть команды или данные.

Еще одним важным понятием являются темы. Темы содержат информацию о том, какие новости интересуют участника или где новости должны быть опубликованы.

Темы представлены строками, разделенными косой чертой ("/"). Каждая косая черта указывает на уровень темы. Например, тему «кофемашина» в офисе можно представить так:

work/office/coffeemachine

Темы чувствительны к регистру:

`work/office/coffeemachine`

и

`work/office/CoffeeMachine`

следовательно, это две разные темы.

Итак, если вы хотите включить кофемашину в офисе через MQTT, должен произойти следующий сценарий:

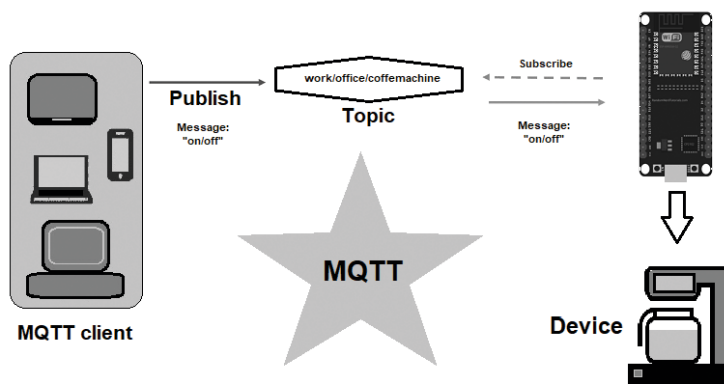


Рис.15.2: Сценарий выполнения MQTT.

Терминальное устройство, такое как ноутбук или смартфон, выдает команду «оп - включить» или «off - выключить» для темы «работа/офис/кофемашина».

На эту тему подписано другое устройство, например ESP32. Поэтому, когда в этой теме публикуется новое сообщение, ESP32 получает сообщение «Вкл.» или «Выкл.» и включает или выключает кофемашину.

Наконец, также требуется брокер . Он в первую очередь отвечает за получение всех сообщений, их фильтрацию и определение того, кто в них заинтересован. У него также есть задача публикации информации для всех подписанных клиентов.

Для этой цели доступны различные брокеры. Широко известной версией является брокер Mosquitto, который можно установить на Raspberry Pi. В качестве альтернативы можно использовать брокера cloudMQTT, например ThingSpeak.

15.1 MQTT через ThingSpeak

В ThingSpeak данные могут быть представлены в самых разных формах. Веб-сайт предлагает всестороннюю поддержку для различных систем микроконтроллеров. Вход на платформу ThingSpeak осуществляется через страницу входа. После успешного входа в систему можно запросить уникальные коды назначения, так называемые ключи API (для интерфейса прикладного программирования).

Channels → MyChannels Каналы → Мои каналы

Для связи через MQTT требуется следующая информация о канале ThingSpeak:

- Идентификатор канала (в верхней части каждой страницы «канала»)
- Идентификатор пользователя (Account → My Profile)
- Чтение ключа API (API Keys TAB)
- Запись ключа API (API Keys TAB)
- Ключ API MQTT (Account → My Profile)

На следующем рисунке показан пример страницы ключей API.

Channel ID: 45511
Author: tonismarthome
Access: Private

Private View Public View Channel Settings Sharing **API Keys**

Write API Key

Key: 8431472121012721

Generate New Write API Key

Read API Keys

Key: 70XCVTGG TW1Q2G

Note:

Save Note Delete API Key

Add New Read API Key

Рис.15.3: Ключи API в ThingSpeak.

Для передачи данных на Thingspeak.com снова требуются два файла `boot.py` и `main.py` (в папке с кодами под MQTT_thingspeak). `boot.py` остается без изменений, как обычно, и устанавливает соединение с WLAN.

Сначала в `main.py` создается клиент со свободно выбираемым номером:

```
myMqttClient = bytes("client_"+str(2151139), 'utf-8')
```

Затем этот клиент подключается к брокеру ThingspeakMQTT. Соединение осуществляется через порт TCP.

```
THINGSPEAK_URL=b"mqtt.thingspeak.com"
THINGSPEAK_USER_ID=b'USER_ID'
THINGSPEAK_MQTT_API_KEY=b'MQTT_API_KEY'
client=MQTTClient(client_id=myMqttClient,
                  server=THINGSPEAK_URL,
                  user=THINGSPEAK_USER_ID,
                  password=THINGSPEAK_MQTT_API_KEY,
                  ssl=False)
```

Для заполнителей

- USER_ID
- MQTT_API_KEY

здесь должны использоваться данные, взятые с веб-страницы ThingSpeak. Затем делается попытка установить соединение:

```
try:
    client.connect()
except Exception as e:
    print(,could not connect to MQTT server {}{}'.format(type(e).__name__,
e))
    sys.exit()
```

В случае ошибки выводится соответствующее сообщение. После указания

1. THINGSPEAK_CHANNEL_ID = b'XXXXXX'
2. THINGSPEAK_CHANNEL_WRITE_API_KEY = b'1234567890ABCDFG'

и определение периода времени между двумя передачами данных (минимум 15 секунд),

```
PUB_TIME_SEC = 15
```

вы можете начать отправлять значения:

```

dummyData = adc.read()
credentials = bytes("channels/{:s}/publish/{:s}".format(THINGSPEAK_CHANNEL_
ID, THINGSPEAK_CHANNEL_WRITE_API_KEY), ,utf-8')
payload = bytes("field1={:.1f}\n".format(dummyData), ,utf-8')
client.publish(credentials, payload)
time.sleep(PUB_TIME_SEC)

```

Чтобы предотвратить возможное завершение программы, последовательность передачи была встроена в конструкцию `try/except`. В том числе, вся программа выглядит так:

```

# main: MQTT_ThingSpeak

from umqtt.robust import MQTTClient
import time
import os
#импортировать сборщик мусора
import sys
from machine import ADC, Pin

adc = ADC(Pin(34)) # create ADC object on ADC pin
adc atten(ADC.ATTN_11DB)

# создать случайный идентификатор клиента MQTT
randomNum = int.from_bytes(os.urandom(3), ,little')
myMqttClient = bytes("client_"+str(randomNum), ,utf-8')
# подключиться к брокеру Thingspeak MQTT через незащищенный TCP (порт 1883)

THINGSPEAK_URL=b"mqtt.thingspeak.com"
THINGSPEAK_USER_ID=b'tonismarthome'
THINGSPEAK_MQTT_API_KEY=b'65A4YORGS7N6HPFB'
client=MQTTClient(
    client_id=myMqttClient,
    server=THINGSPEAK_URL,
    user=THINGSPEAK_USER_ID,
    password=THINGSPEAK_MQTT_API_KEY,
    ssl=False)

try:
    client.connect()
except Exception as e:
    print(,could not connect to MQTT server {}{}' .format(type(e).__name__,
e))
    sys.exit()

# опубликовать в Thingspeak с помощью MQTT
THINGSPEAK_CHANNEL_ID = b'123456'
THINGSPEAK_CHANNEL_WRITE_API_KEY = b'1234567890123456'

```

```

PUB_TIME_SEC = 15
while True:
    try:
        dummyData = adc.read()
        print(dummyData)
        credentials = bytes("channels/{:s}/publish/{:s}".format(THINGSPEAK_
CHANNEL_ID, THINGSPEAK_CHANNEL_WRITE_API_KEY), ,utf-8')
        payload = bytes("field1={:.1f}\n".format(dummyData), ,utf-8')
        client.publish(credentials, payload)
        time.sleep(PUB_TIME_SEC)
    except KeyboardInterrupt:
        print(,exit...)
        client.disconnect()
        sys.exit()

```

Наконец, на ESP необходимо загрузить файл `robust.py` (из папки с кодами). Он содержит основные процедуры для протокола MQTT.

Затем последовательность программ может быть запущена через перезагрузку. ESP автоматически входит в указанную сеть WLAN и подключается к серверу ThingSpeak.

Теперь данные уже можно загружать на веб-страницу, а затем запрашивать по всему миру. Для этого вы можете настроить свою индивидуальную веб-страницу ThingSpeak. Каналы данных могут быть сконфигурированы индивидуально в:

MyChannels → New Channel Мои каналы→Новый канал

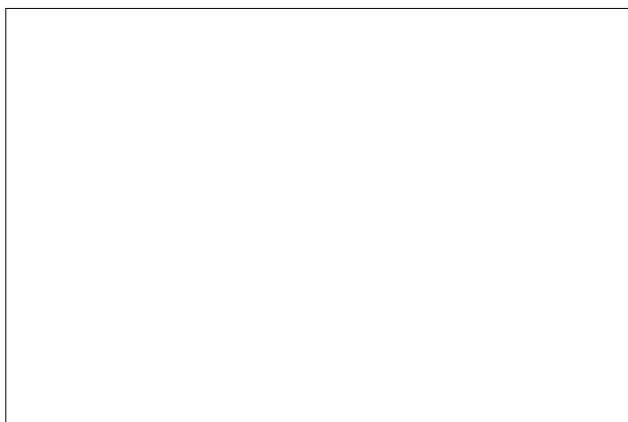


Рис.15.4: MyChannels на ThingSpeak.com.

На этом этапе можно определить несколько каналов. В приведенной выше программе так называемое поле уже определено.

На странице канала графические представления можно создать в разделе «Add Visualizations - Добавить визуализации». Для фиктивных данных АЦП результат выглядит следующим образом:

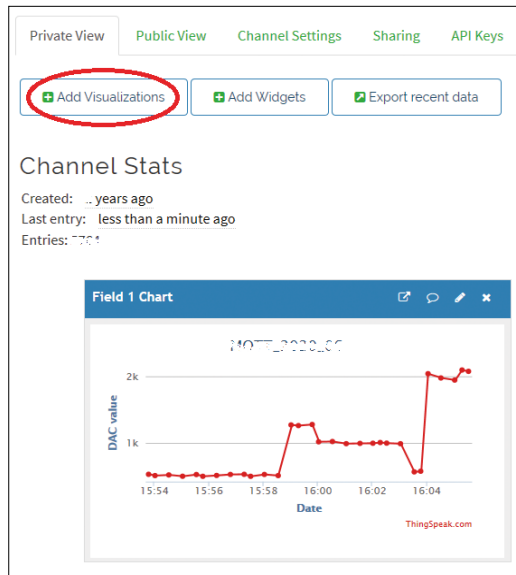


Рис.15.5: Фиктивные данные на странице ThingSpeak.

Что касается компонентов, здесь снова можно использовать настройку, показанную на рис. 9.2, т.е. потенциометр на порту 34.

Глава 16 • Отправка данных в Интернет через ThingSpeak

В WLAN данные изначально доступны только в ограниченной среде. Преимущество этого заключается в том, что измеренные значения и т. п. также относительно безопасны. Однако также бывают ситуации, когда вы хотите получить данные измерений или значения датчиков с большого расстояния или даже со всего мира. Во многих случаях желательно иметь возможность проверять домашние данные в дороге, с работы или в отпуске. В принципе, можно было бы сделать все значения из домашней WLAN доступными глобально, если мы разрешим внешний доступ к домашней WLAN. Однако это связано с различными рисками безопасности. Именно здесь онлайн-платформа, такая как ThingSpeak, может использовать свои сильные стороны.

С помощью службы IoT можно регистрировать значения измерений или датчиков, после чего они становятся доступными по всему миру. Помимо доступа через Интернет, также доступно приложение для мобильных устройств (см. ниже). Защищенный паролем доступ к платформе ThingSpeak также обеспечивает достаточную защиту данных.

Контроллер ESP32 практически предназначен для использования в ThingSpeak благодаря встроенным возможностям беспроводной связи. Это встроенное соединение WLAN является решающим преимуществом по сравнению с другими системами, такими как классические платы Arduino. Им требовался собственный «шилд WLAN» для связи через беспроводную сеть. С другой стороны, в чипе ESP32 эта функция уже интегрирована.

16.1 Дождь или гроза? Виртуальная метеостанция доступна по всему миру

В общем, нужно отправлять не только фиктивные данные значений потенциометра, но и реальные измеренные значения. Данные о погоде или климате имеют особое значение в этом контексте. Для надежных прогнозов погоды важно знать климат или данные о погоде даже из отдаленных мест. Например, если вы в отпуске или в командировке, может быть интересно проверить метеостанцию дома. Например, при угрозе заморозков можно попросить соседа включить отопление. Или вы можете проверить данные о погоде в вашем загородном доме из дома. При необходимости там можно заранее включить кондиционер и по приезду найти комнаты с приятной температурой.

Для этой цели можно использовать датчик BME280. Его основная функция и процедура считывания данных уже объяснялись в разделе 9.19. Теперь эти данные нужно передать на страницу Thingspeak.com. Со встроенными датчиками для

- Температура
- Влажность
- Атмосферное давление

вы можете сделать самые важные данные доступными через ThingSpeak. Для этого в тестовой программе из последнего раздела вместо фиктивных данных передаются только реальные значения:

```
def collectData():
```

```

bme =BME280.BME280(i2c=i2c)
temp=0.01*int(bme.read_temperature())
humi=0.01*int(bme.read_humidity())/10.24)
pres=0.01*int(bme.read_pressure())/256)
return temp,humi,pres

```

Чтение данных через шину I2C уже было объяснено в главе 9, а дополнительные сведения можно найти в разделе 18.1. Помимо передачи на сторону ThingSpeak, климатические значения также должны отображаться локально на дисплее. Поэтому включены соответствующие части программы из Раздела 10.6. Снова требуется загрузка и основной файл, чтобы климатическая станция WLAN правильно запускалась после возможного отключения питания. Доступ к WLAN осуществляется, как обычно, через файл boot.py, который можно взять без изменений из предыдущего раздела. С другой стороны, полный файл main.py теперь выглядит так:

```

# ClimateStation.py
import network
from umqtt.simple import MQTTClient
from time import sleep
from machine import Pin, I2C
import ssd1306
import BME280

PUB_TIME_SEC=15 # final->300

# данные связи ThingSpeak
SERVER="mqtt.thingspeak.com"
CHANNEL_ID="123456"
WRITE_API_KEY="ABCDEFGH1234567890"

# Объект клиента MQTT и строка темы MQTT
client=MQTTClient("umqtt_client", SERVER)
topic="channels/"+CHANNEL_ID+"/publish/"+WRITE_API_KEY

# ESP32 - Назначение пинов
i2c = I2C(scl=Pin(22), sda=Pin(21), freq=100000)

# Настройка OLED
oled_width = 128
oled_height = 64
oled = ssd1306.SSD1306_I2C(oled_width, oled_height, i2c)

# Функция чтения всех данных:
def collectData():
    bme =BME280.BME280(i2c=i2c)
    temp=0.01*int(bme.read_temperature())
    humi=0.01*int(bme.read_humidity())/10.24)

```

```

    pres=0.01*int(bme.read_pressure())/256)
    return temp,humi,pres

# постоянно отправлять данные
while True:
    oled.fill(0)
    oled.text('BME280 climate', 0, 0)
    temp,humi,pres=collectData()

    print('Temp: %7.1f' %temp)
    print('Humi: %7.1f' %humi)
    print('Pres: %7.1f' %pres)
    print()

    temp_string=str('Temp:%7.2f' %temp)
    humi_string =str('Humi:%7.2f' %humi)
    pres_string=str('Pres:%7.2f' %pres)

    oled.text(temp_string, 0, 10)
    oled.text(' C', 90 ,10)
    oled.text(humi_string, 0, 20)
    oled.text(' %', 90 ,20)
    oled.text(pres_string, 0, 30)
    oled.text(' hPa', 90 ,30)
    oled.show()

    #check data on terminal
    print("Data for ThingSpeak:")
    print(temp,humi,pres)
    print("-----")
    #send date to ThingSpeak
    payload="field1="+str(temp)+"&field2="+str(humi)+"&field3="+str(pres)
    client.connect()
    client.publish(topic, payload)
    client.disconnect()
    sleep(PUB_TIME_SEC)

```

После входа в поля Thingspeak настройка базы данных завершена, и все собранные значения могут отображаться графически в различных представлениях. На рис.16.1 показан пример визуализации данных датчика. В следующем разделе показано, как можно настроить графику.

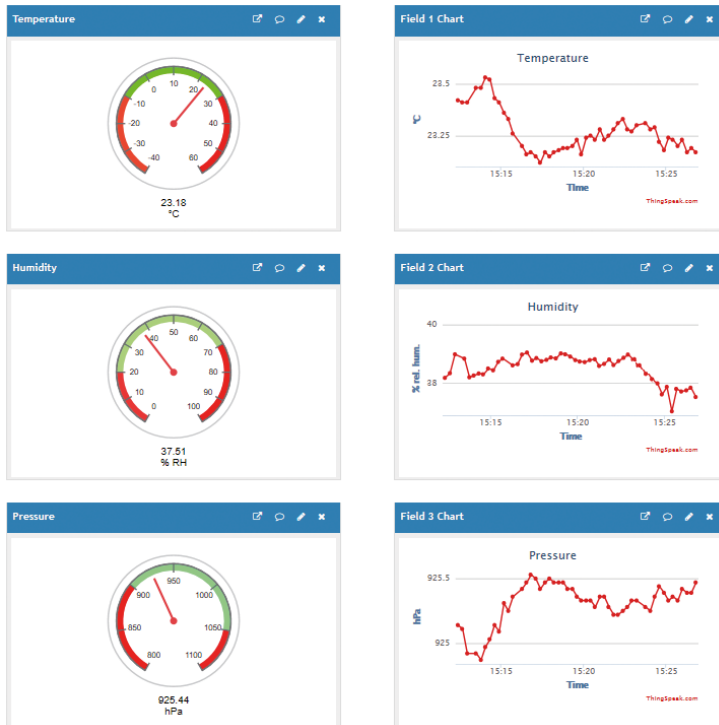


Рис.16.1: Отображение климатических данных в ThingSpeak.

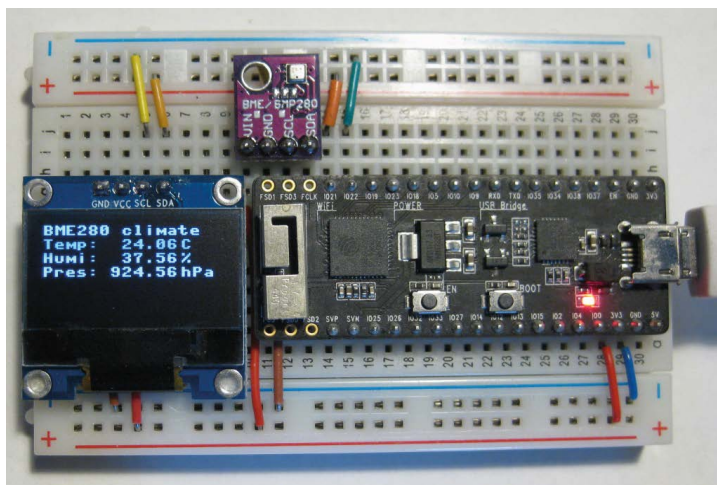


Рис.16.2: Локальное отображение климатических данных на OLED-дисплее.



Рисунок 16.3: Компактная метеостанция WLAN с дисплеем.

16.2 Графическое представление данных в ThingSpeak

Все данные и измеренные значения могут отображаться графически в различных представлениях. На следующих иллюстрациях показаны некоторые рекомендации по отображению данных датчика:

Рис.16.4: Отображение данных в ThingSpeak.

Графика может быть разработана индивидуально. Различные цвета и цветовые комбинации открывают широкие возможности. Оси, количество точек измерения, промежутки времени измерения и т. д. могут быть настроены индивидуально, как и типы линий, количество средних значений и т. д. Кроме того, могут отображаться различные виртуальные инструменты. С их помощью определенные заданные значения или опасные зоны могут быть отмечены цветом. Настройки можно вызвать с помощью символа карандаша в правом верхнем углу поля:

Field 1 Chart Options	
Title:	Temperature
X-Axis:	Time
Y-Axis:	°C
Color:	#dd0020
Background:	#eeffdd
Type:	step
Dynamic?:	false
Days:	1
Results:	20
Timescale:	
Average:	10
Median:	
Sum:	
Rounding:	
Data Min:	
Data Max:	
Y-Axis Min:	10
Y-Axis Max:	30

Рис.16.5: Варианты анализа данных.

Таким образом, можно создавать очень четкие и информативные дисплеи данных. В частности,

- названия графиков,
- метки осей,
- цвет линий и фона,
- типы графиков, такие как
 - линейные графики,
 - гистограммы,
 - сплайн-графики,
- количество точек измерения,
- шкалы времени,
- агентства,

и другие могут быть установлены. Кроме того, доступны виртуальные аналоговые приборы или яркие предупреждающие дисплеи. Так что практически нет ограничений для индивидуального оформления собственных данныхотображать.

16.3 Данные для смартфона с приложением ThingView

С «ThingView» доступно приложение для оценки данных на мобильных устройствах, таких как смартфоны или планшетные компьютеры. С его помощью можно быстро и легко проверить и контролировать все измеренные значения по всему миру. Приложение можно бесплатно загрузить из известных магазинов приложений (например, Play Store). Независимо от того, хотите ли вы прочитать значения дома из-за границы или проверить текущую погоду на работе, приложение предлагает все возможности. После ввода идентификатора канала и соответствующего API-ключа данные доступны на любом мобильном устройстве.

Приложение берет на себя такие настройки окна, как цвет, шкала времени или тип диаграммы и количество результатов. На следующем рисунке показано приложение в действии:

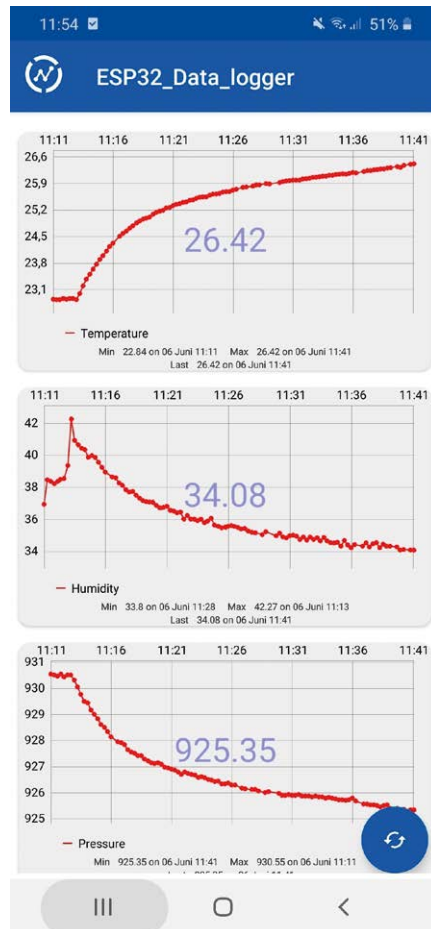


Рис.16.6: Данные в приложении ThingView.

16.4 Защита от нежелательных посетителей: оптическое наблюдение за помещением

Опдатчики можно использовать не только для измерения освещенности помещения. Их также можно использовать для настройки системы ThingSpeak Iot для мониторинга задач. Таким образом, комнаты можно контролировать на наличие нежелательных действий и незваных «гостей». Акустическая сигнализация часто не очень полезна, так как только предупреждает нежелательных посетителей. Если, с другой стороны, движения в помещении регистрируются, то легко проверить, не проникли ли в помещения без разрешения в случае подозрительной деятельности.

Также возможно сделать это без специального источника света. Часто бывает достаточно определить условия пассивного освещения в помещении. Например, неожиданное увеличение яркости в 01:30 в подвальном помещении указывает на незваного гостя, который непреднамеренно освещает модуль датчика освещенности фонариком.

Днем естественное освещение можно использовать для наблюдения за помещением. Дневной свет меняется сравнительно медленно. Рассветы и закаты всегда занимают больше времени. Если наблюдается так называемый всплеск, вероятно, человек или животное бросили тень на датчик освещенности.

В качестве аппаратной основы можно использовать структуру, показанную на рис.9.17. Соответствующая программа `main.py` выглядит так:

```
# BPW40_intrusion_detection.py
from time import sleep
from umqtt.simple import MQTTClient
from machine import Pin, ADC

# Данные ThingSpeak
SERVER="mqtt.thingspeak.com"
THINGSPEAK_CHANNEL_ID = b'123456'
THINGSPEAK_CHANNEL_WRITE_API_KEY = b'1234567890123456'
PUB_TIME_SEC=15 # final->300
# Объект клиента MQTT и строка темы MQTT
client=MQTTClient("umqtt_client", SERVER)
topic="channels/"+CHANNEL_ID+"/publish/"+WRITE_API_KEY

fotoPin = ADC(Pin(34))
fotoPin.atten(ADC.ATTN_11DB) # input range 0 ... 3,3 V

# Функция чтения всех данных:
def collectData():
    data0=fotoPin.read()/40.96
    return data0

# постоянно отправлять данные
while True:
    data0=collectData()
    #check data on terminal
    print(data0)
    payload="field1="+str(data0)
    client.connect()
    client.publish(topic, payload)
    client.disconnect()
    sleep(PUB_TIME_SEC)
```

В функции

```
def collectData():
```

текущие значения яркости записываются и снова нормализуются до 100%. Затем в основном цикле они отправляются на страницу ThingSpeak.

Там их можно отобразить в виде графика, как на рис.16.7. В примере видно несколько «всплесков», которые явно указывают на то, что человек находился в контролируемой зоне в рассматриваемый период.

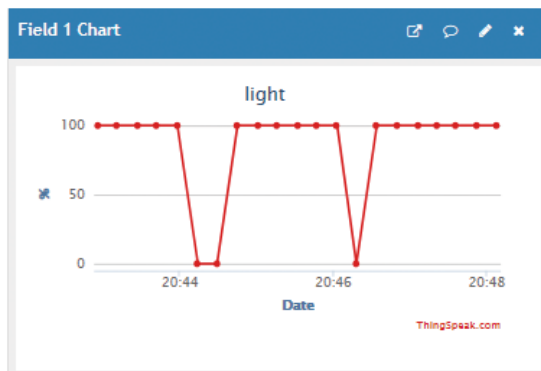


Рис.16.7: Всплески, производимые оптическим комнатным монитором, показывают, что в обследуемой зоне находился человек.

Глава 17 • Методы микромощности и спящие режимы

Если электронные устройства не работают в стационарном месте, а являются мобильными, они не могут быть снабжены блоком питания или подключены к USB-порту. В этом случае альтернативой является работа с перезаряжаемыми или сухими батареями. Климатические, экологические или другие измерительные станции в удаленных местах также часто зависят от работы от батарей. В этих случаях энергопотребление системы имеет решающее значение. Низкое энергопотребление особенно важно, когда устройства работают непрерывно. Часто такая измерительная система должна записывать определенное значение датчика, например, только один раз в минуту. В промежутке никаких действий не требуется. Если контроллер остается включенным в обычном режиме, это сопровождается значительным потреблением энергии. Гораздо дешевле перевести чип в «спящий режим», в котором он потребляет очень небольшое количество энергии.

В следующих разделах представлены соответствующие режимы работы и описаны некоторые примеры.

17.1 Энергосбережение защищает окружающую среду: технологии с низким энергопотреблением

Емкость современных аккумуляторных элементов или батарей находится в пределах нескольких ампер-часов (Ач). Таким образом, при продолжительности работы в несколько недель или месяцев измерительная система может потреблять в среднем всего несколько микроампер (мАч). С контроллером ESP32 эта цель, безусловно, достижима. Для этого контроллер надо перевести в режим «глубокого сна».

При нормальной работе с пустым основным циклом, содержащим только классическую команду `sleep(1)`, потребляемая мощность типичной платы ESP составляет около 50 мА. С помощью спящего режима можно значительно снизить энергопотребление. Следующая программа сначала вызывает три коротких мигания светодиода на порте 02, затем контроллер переходит в спящий режим:

```
# deepSleep_demo.py
import machine
from machine import Pin
from time import sleep

led = Pin (22, Pin.OUT)

# мигание LED
for n in range(3):
    led.value(1)
    sleep(.1)
    led.value(0)
    sleep(.1)

sleep(5) # awake time
```

```
print(,going to sleep...')  
  
machine.deepsleep(10000) # в миллисекундах!
```

Сразу после запуска программы ток потребления платы еще можно измерить на уровне почти 50 мА. Однако примерно через одну секунду потребление тока снижается до менее 5 мА. Сам контроллер потребляет намного меньше тока, но на платах ESP также активны преобразователь USB-to-serial и светодиод включения питания. Эти компоненты нельзя отключить без аппаратного вмешательства. Тем не менее снижение мощности в 10 раз весьма существенно и весьма полезно. Используя спящий режим, срок службы батареи или время работы от заряда батареи может быть увеличено на этот коэффициент. Этого вполне достаточно для многих приложений.

При мощности источника питания, например, 3000 мАч время работы можно достигать 500 часов или более 20 дней. С другой стороны, при постоянно активной работе источник питания будет исчерпан всего через 48 часов. Эксплуатация солнечной батареи с буферным аккумулятором теперь также намного проще. В климатической станции, работающей от солнечной батареи (см. раздел 17.3), спящий режим увеличивает время работы буферной батареи до приемлемых значений.

17.2 Отключение ненужных потребителей

Использование спящего режима уже значительно снижает потребление контроллером тока. Однако при использовании полных плат контроллеров это заметно лишь в ограниченной степени. Помимо самого чипа ESP32, они также содержат другие потребители, такие как светодиоды включения питания или преобразователи USB-to-serial. Поэтому для дальнейшего снижения энергопотребления устройства на базе ESP32, работающего исключительно от батареи, необходимы дополнительные меры. Светодиоды питания или другие индикаторы рабочего состояния должны быть отключены. Преобразователи интерфейсов и другие компоненты должны иметь возможность отключения. При соответствующих усилиях потребление тока может быть снижено до уровня менее 1 мА. Однако для этого все ненужные потребители должны быть отключены от источника питания с помощью соответствующих прерываний линии. Однако это вряд ли осуществимо простыми средствами на классических макетных платах. Необходимые аппаратные вмешательства предполагают высокий уровень опыта, поэтому эта процедура предназначена для пользователей с соответствующими специальными знаниями. Поэтому здесь эта тема больше обсуждаться не будет. В этих случаях обычно лучше все же перейти на специальные платы.

Кроме того, следует стремиться к прямой работе с напряжением 3,3 В. Снижение напряжения, скажем, с 5 В до 3,3 В всегда связано с потерями, и поэтому его следует избегать. Потери мощности электронных компонентов обычно увеличиваются пропорционально квадрату рабочего напряжения. По этой причине все внешние компоненты также должны работать при напряжении 3,3 В. Детектор пониженного напряжения (BOD) ESP32, который обнаруживает пониженное напряжение, а затем запускает сброс ЦП, должен быть настроен соответствующим образом.

Цифровых датчиков следует избегать, так как они также имеют довольно высокое потребление тока покоя. Предпочтение следует отдавать классическим аналоговым преобразователям (фототранзисторы, термодатчики и т.п.). Если, тем не менее, нельзя избежать использования цифровых датчиков, их следует включать и выключать через порты ввода-вывода, чтобы они работали только тогда, когда они фактически выполняют свои измерительные задачи.

17.3 Метеостанция с батарейным или солнечным питанием

В качестве применения режима глубокого сна метеостанция из главы 16 может быть переведена в режим работы с низким энергопотреблением. Для этого достаточно строки

```
sleep(PUB_TIME_SEC)
```

следует заменить на

```
machine.deepsleep(PUB_TIME_SEC)
```

Кроме того, перед командой "deepsleep" рекомендуется использовать команду sleep (1). Это сделает программу немного более стабильной. Для дальнейшего снижения энергопотребления можно также отключить дисплей и удалить соответствующие строки кода. Это позволяет станции работать в непрерывном режиме 24 часа с небольшим солнечным элементом с буферным аккумулятором. На следующем рисунке показана установка с аккумулятором солнечной энергии.



Рис.17.1: Метеостанция с питанием от солнечной энергии.

Глава 18 • Шинные системы для эффективной связи

Шинные системы уже использовались в различных главах. Их снова и снова использовали для управления исполнительными механизмами и датчиками. Они позволяют подключать большое количество компонентов к контроллеру с помощью нескольких выводов процессора и минимума схем. В этих последних главах книги эти системы будут обсуждаться более подробно. Что касается программного обеспечения, MicroPython предоставляет удобные драйверы для управления подключенными компонентами. Две системы шин нашли широкое применение в приложениях микроконтроллеров:

- I²C (Inter IC or IIC)
- SPI (Serial Peripheral Interface)

У этих двух систем есть интересные возможности, когда дело доходит до реализации более крупных проектов. Со все более сложными задачами и значительными расширениями в какой-то момент даже у ESP32 закончатся пины. Шинные системы являются здесь отличным решением, так как многими компонентами можно управлять с помощью всего несколькими GPIO. Поскольку многие датчики имеют соответствующие интерфейсы, шины уже неоднократно использовались. В следующей таблице приведены некоторые примеры:

Модуль	Шина
SSD1306 OLED display	I ² C
RC522 RFID	SPI
BMP280 Барометр	I ² C
BME280 Мультиклимат. датчик	I ² C

Кроме того, к ESP32 через шинные системы можно подключить множество других компонентов, например,

- дисплеи и блоки индикации,
- EEPROM, флэш-память или SD-карты,
- часы реального времени.

Большинство этих компонентов также можно без проблем использовать в MicroPython, особенно если доступны соответствующие библиотеки.

18.1 Основы и применение шины I²C

ESP32 имеет 36 пинов цифрового ввода-вывода, но не все из них доступны на большинстве плат. Для небольших проектов это вполне приемлемо, но для более крупных приложений вы можете столкнуться с ограничениями.

Шинные системы предлагают здесь экономичный вариант расширения. Они позволяют управлять широким спектром функций с помощью всего нескольких контактов. Классическим примером является использование дисплея I²C. Для него требуется всего два контакта порта, в отличие от шести или более портов, занимаемых классическим ЖК-дисплеем HD44780. В этой главе шина I²C будет рассмотрена в первую очередь. Шина SPI будет рассмотрена в следующих разделах.

I²C означает Inter-Integrated-Circuit. Последовательная двухпроводная шина была разработана компанией Philips более 30 лет назад. Система быстро развивалась и стала квазиотраслевым стандартом для задач управления в среде микроконтроллеров. Низкая стоимость и простота реализации быстро обеспечили широкое распространение. Скорость передачи до 3,4 Мбит/с достаточна для многих приложений и не позволила шине потерять свою популярность по сей день.

Шина I²C обеспечивает эффективный обмен данными между несколькими ИС на одной плате. Была реализована структура последовательной шины, чтобы обойтись всего несколькими дорожками. Это привело к двунаправленной шинной системе с архитектурой ведущий/ведомый. Благодаря встроенному протоколу передачи и программной адресации требуются только две соединительные линии:

- тактовая линия SCL (Serial Clock)
- линия данных SDA (Serial Data)

С помощью этих двух линий микроконтроллер может управлять всей сетью компонентов всего с двумя выводами порта. Применение шины в основном встречается в бытовой электронике. Считывание показаний часов реального времени, управление дисплеями или электронная регулировка громкости являются типичными приложениями I²C.

На следующем рисунке показана базовая структура шины I²C.

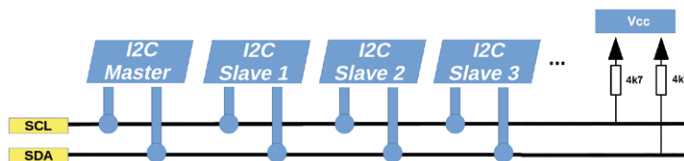


Рис.18.1: Базовая структура шины I²C.

Растущие требования к производительности привели к постоянному росту тактовой частоты с течением времени. Тем не менее, более медленные компоненты также могут работать на шине посредством растяжения тактовой частоты (см. ниже). Поскольку нет необходимости придерживаться фиксированного времени, одновременно могут работать как более медленные, так и очень быстрые участники шины.

Шина I²C постоянно совершенствовалась и теперь используется не только для соединения отдельных ИС. Простое управляющее программное обеспечение можно очень гибко адаптировать, чтобы шину можно было использовать и в более крупных системах. Структура программного обеспечения I²C также позволяет использовать более медленные языки программирования, такие как Python. Это еще одна причина, по которой система идеально подходит для широкого спектра приложений в секторе IoT.

Шина I²C основана на системе ведущий-ведомый. Передача данных всегда запускается «мастером». Затем «ведомое устройство», к которому обращаются по его адресу, отвечает на запрос. В режиме мультимастера на одной шине могут работать несколько мастеров. Затем два ведущих устройства связываются друг с другом, ненадолго перенастраивая один из них в качестве подчиненного. Этот особый контроль доступа к шине точно регулируется соответствующей спецификацией. Однако это относительно редко используемое приложение, которое мы не будем здесь рассматривать.

Компоненты и устройства, подключенные к шине, имеют входы с открытым коллектором. Вместе с подтягивающими резисторами это создает схему проводного И. Это позволяет шине работать с разными напряжениями. Подтягивающие устройства должны быть подключены только к соответствующему напряжению (например, $V_{dd} = 3,3 \text{ В}$ или $V_{dd} = 5 \text{ В}$).

Допустимый высокий уровень для сигналов шины составляет не менее 0,7 В вольт. Низкий уровень не должен превышать 0,3В вольт. При частых ошибках передачи следует проверить напряжения с помощью осциллографа. Если значения напряжения отклоняются от целевого диапазона, необходимо отрегулировать подтягивающие резисторы. В некоторых контроллерах даже встроены внутренние подтягивающие устройства; внешние резисторы в этом случае можно не использовать. Спецификации компонентов I2C обычно содержат соответствующую информацию.

Шинная система I2C основана на положительной логике. Следовательно, высокий уровень на линии данных соответствует логической единице, низкий уровень — логическому нулю. Часы шины всегда задаются мастером. Различные режимы шины всегда работают с максимально допустимой тактовой частотой шины. Более низкие тактовые частоты возможны только в том случае, если они поддерживаются мастером. Однако некоторые компоненты I2C, такие как аналого-цифровые преобразователи, требуют определенной минимальной тактовой частоты, чтобы работать без ошибок. Здесь необходимо проверить, поддерживается ли это мастером. Наиболее распространенные тактовые частоты:

- 100 kHz такты: стандартный режим
- 400 kHz такты: быстрый режим
- 1.0 MHz такты: быстрый режим +
- 3.4 MHz такты: высокоскоростной режим

Если ведомому устройству требуется больше времени, чем тактам ведущего, оно может удерживать тактовую линию на низком уровне между передачей отдельных байтов. Упомянутая выше растяжка часов может быть использована для настройки основных часов, если это необходимо.

В системе I2C данные действительны только в том случае, если их логический уровень не изменяется в течение верхней фазы тактового сигнала. Сигналы пуска и остановки являются исключением из этого правила. Стартовые сигналы характеризуются спадающими фронтами сигнала данных, в то время как тактовая линия остается на высоком уровне. Сигналы остановки, с другой стороны, сигнализируются нарастающим фронтом на линии данных, в то время как тактовый сигнал остается на высоком уровне.

Как обычно в технологии данных, отдельные блоки данных на шине I2C также состоят из восьми битов данных каждый, так называемый октет. Это можно интерпретировать либо как значение, либо как адрес. Кроме того, в качестве подтверждения используется бит ACK (подтверждение). Об этом сигнализирует ведомое устройство низким уровнем на линии данных.

Первый байт, отправленный мастером, представляет собой стандартный адрес I2C. Первые семь бит представляют фактический адрес. Восьмой и последний бит (бит R/W) сообщает ведомому, должен ли он получать данные от ведущего ((LOW) или передавать данные ведущему (HIGH)). Таким образом, в системе I2C доступно адресное пространство из 7 бит. При 16 адресах, зарезервированных для специальных целей, остается $112 \text{ из } 27 = 128$ возможных адресов, которые могут быть адресованы одновременно на одной шине.

Каждый компонент с поддержкой I²C имеет адрес, определенный производителем, из которых три бита, так называемый подадрес, часто можно определить индивидуально с помощью трех управляющих контактов. Это означает, что до восьми ИС одного типа могут работать на одна шина I²C. Это более подробно описано ниже с использованием датчика влажности и температуры воздуха типа VME280.

Для многих приложений достаточно более 100 участников на шинной системе. Доступные адреса все еще могут стать дефицитными в обширных профессиональных системах. По этой причине несколько лет назад адресация была расширена до 10 бит. Используя 4 из 16 зарезервированных адресов, новое адресное пространство обратно совместимо со старым 7-битным стандартом. Оба типа адресации могут использоваться одновременно, так что в настоящее время на одной шине можно адресовать до 1136 компонентов.

Передача данных I²C всегда начинается со стартового сигнала от ведущего устройства. Затем следует нужный адрес. Это подтверждается битом ACK адресуемого ведомого устройства. В зависимости от бита R/W данные могут быть записаны или прочитаны побайтно. При записи ACK отправляется ведомым устройством; при чтении он отправляется мастером. Передача прекращается стоп-сигналом.

В протоколе I²C передача данных начинается со старшего бита («MSB-first»). Для высокоскоростного режима сначала отправляется соответствующий мастер-код в быстром или стандартном режиме. Это заставляет всю систему переключаться на новую, более высокую частоту.

Рис.18.2: Синхронизация шины I²C.

Шина I²C позволяет управлять сетью электронных компонентов всего с двумя пинами GPIO. В профессиональных приложениях это дает значительные преимущества по стоимости. Поскольку риск сбоя увеличивается с увеличением количества необходимых соединений, затраты на тестирование и проверку также возрастают. Таким образом, меньше следов на плате минимизирует затраты на производство, разработку и тестирование.

Интерфейс I²C лучше всего подходит для периферийных устройств с более низкой скоростью передачи данных. Данные управления и конфигурации обычно не требуют высоких тактовых частот. Поэтому шина I²C часто используется для электронных регуляторов громкости, АЦП или ЦАП с низкой частотой дискретизации, небольших энергонезависимых запоминающих устройств, часов реального времени или двунаправленных переключателей. Как показывает пример VME280, электронные датчики климатических данных, таких как температура, давление воздуха или влажность, со встроенными аналого-цифровыми преобразователями особенно подходят для шины I²C.

В следующей таблице показаны некоторые другие часто используемые микросхемы с поддержкой I²C. Для них доступны соответствующие библиотеки MicroPython, поэтому их использование не должно вызывать никаких проблем.

ИС / Компонент	Функция
DSS1306	OLED дисплей 0,96 дюйма
AT24C256	EEPROM
BMP180/280	Датчик барометр
BME280	Датчик климата
DS1307	Часы реал. времени с буфером батареи
I2C16x4	4 x 16 segment LCD display
LM75	Датчик температуры
MAX127	АЦП
PCF8574	Расширитель портов
PCF8583	Часы реального времени

В таблице показано лишь небольшое количество доступных компонентов I²C. В частности, были выбраны компоненты, которые предпочтительно используются в секторе умного дома.

Еще одним преимуществом системы I²C является так называемая возможность горячей замены. Это означает, что компоненты могут быть добавлены или удалены из шины во время работы. В MicroPython эта функция может поддерживаться конструкцией try/except. Это означает, что можно эффективно избежать нежелательных прерываний программы, даже если элементы будут удалены или добавлены.

Таблица ясно показывает, что получение значений датчиков является одним из наиболее важных применений шины I²C. Почти все физические переменные могут быть зарегистрированы с помощью датчиков I²C. Некоторые компоненты стали особенно популярными в приложениях Интернета вещей (IoT). Наиболее важные из них будут представлены ниже в качестве примеров применения шинной системы.

Дистанционное управление температурой, например, очень ненадежно с простыми аналогами датчиков. Тепловые напряжения, сопротивление линии или электромагнитные помехи приводят к значительным ошибкам измерения. Кроме того, часто между датчиком и цифровой системой управления имеются значительные расстояния, особенно в системах умного дома. Это тоже может привести к ошибкам измерения. Цифровые датчики гораздо менее подвержены этим типам ошибок. При них измеренное значение преобразуется в цифровой сигнал непосредственно в точке измерения, что делает систему менее чувствительной к вышеупомянутым влияниям. По этим причинам также можно преодолеть большие расстояния между датчиком и центром управления умным домом.

Если к шине подключается все больше компонентов, целесообразно проверить всю установку перед вводом в эксплуатацию. В следующей таблице показаны некоторые известные датчики и связанные с ними адреса I²C:

Компонент	Двоичный адрес	Hex Address
BME280	0b1110110	0x76
BMP185	0b1110111	0x77
I ² C display	0b0011100	0x3C

Благодаря их индивидуально разным адресам все датчики, присутствующие на шине, могут быть адресованы одновременно без каких-либо проблем. Работу шины можно проверить с помощью так называемого сканера I²C. В MicroPython сканер I²C может выглядеть так:

```
# I2C_scanner.py

from machine import Pin, I2C

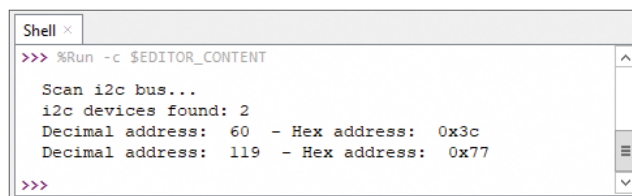
#i2c = I2C(scl=Pin(5), sda=Pin(4))
i2c = I2C(-1, scl=Pin(22), sda=Pin(21))

print('Scan i2c bus...')
devices = i2c.scan()

if len(devices) == 0:
    print("No i2c device !")
else:
    print('i2c devices found:',len(devices))

    for device in devices:
        print("Decimal address: ",device," - Hex address: ",hex(device))
```

Если все компоненты подключены правильно, в консоли появятся соответствующие адреса шин:



```
Shell x
>>> %Run -c $EDITOR_CONTENT

Scan i2c bus...
i2c devices found: 2
Decimal address: 60 - Hex address: 0x3c
Decimal address: 119 - Hex address: 0x77
>>>
```

Рис.18.3: Сканер I²C находит на шине два компонента.

Это гарантирует, что шина работает правильно и что все датчики могут передавать данные. Теперь ничто не мешает использовать систему I²C для приложений IoT или умного дома.

18.2 Шина SPI

Помимо системы I²C, шина SPI (последовательный периферийный интерфейс) является второй важной шинной системой, зарекомендовавшей себя в секторе микроконтроллеров. Тактовые частоты компонентов SPI могут достигать нескольких мегагерц. Это делает шину SPI потенциально намного быстрее, чем системы I²C.

В дополнение к термину SPI, который первоначально был введен Motorola, также часто используется термин «Microwire», придуманный National Semiconductor. Оба термина используют одни и те же функциональные принципы.

Многие важные приложения могут быть реализованы с помощью устройств SPI, в том числе:

- RFID-транспондер
- EEPROM
- Контроллер ЖК-дисплея
- ЦАП и АЦП
- Все типы датчиков
- Элементы управления LED матрицей
- и т. д.

Кроме того, аудиокомпоненты или элементы настройки или системы управления на основе SPI доступны для Hi-Fi и телевизоров или бытовой техники.

Шина SPI использует принцип синхронной передачи данных. Поэтому для передачи данных и синхронизации требуются отдельные линии, которые не требуют синхронной генерации тактовых импульсов с обеих сторон. Передаваемые тактовые сигналы приемнику с высокой точностью, в это время биты на линии данных должны быть просканированы. Для этой цели можно использовать нарастающий или спадающий фронт тактового сигнала. Если приемник обнаруживает фронт на тактовой линии, он немедленно сканирует сигнал данных, чтобы прочитать следующий бит (см. рисунок). Поскольку часы передаются вместе с данными, нет необходимости указывать тактовую частоту, так называемую «скорость передачи».

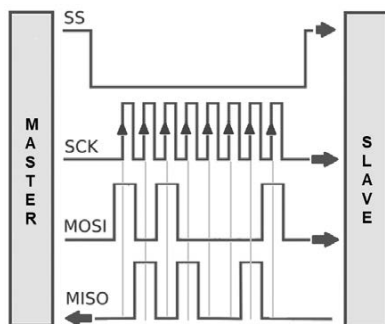


Рис.18.4: Передача данных по шине SPI.

Приемная сторона шины SPI может состоять из простого сдвигового регистра. Это гораздо более простое и дешевое решение, чем высокоинтегрированные компоненты UART (универсальный асинхронный приемник/передатчик) для асинхронных последовательных интерфейсов. Поэтому система SPI часто используется для простых и экономичных решений в непрофессиональном секторе.

Однако синхронная передача данных имеет тот недостаток, что требуется несколько сигнальных линий. Всего необходимо до четырех строк:

- Тактовая линия, SCLK или SCK (последовательные такты)
- MOSI или вход последовательных данных (SDI)
- MISO (выход Master In Slave) или вывод последовательных данных (SDO)
- Сигнал выбора чипа (CS) или SS для выбора ведомого

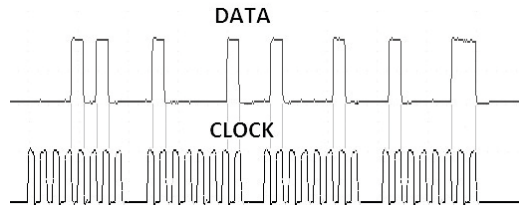


Рис.18.5: Сигнал шины SPI.

Через линию CS активируется адресованное в данный момент периферийное устройство. Сигналы в системе SPI малоактивны, т.е. линия CS остается высокопотенциальной до тех пор, пока не будет адресована микросхема. Линии данных имеют высокий импеданс в состоянии ожидания и, таким образом, электрически развязаны с шиной данных. На рис. 18.5 показано, как на практике выглядят телеграмма SPI и связанный с ней тактовый сигнал.

Если отдельные компоненты на шине SPI должны обмениваться данными в двух направлениях, требуются все четыре сигнальные линии. Если требуется только ограниченный обмен данными, одну строку можно опустить. Если, например, периферийный элемент не нужно конфигурировать, строку ввода данных можно опустить. Как только соответствующий чип выбран по линии CS, он начинает отправлять данные. В АЦП, например, строка MISO является лишней и может быть опущена. Точно так же контроллерам ЖК-дисплеев нужно только получать данные. В этом случае можно сэкономить на соединении MOSI.

Под MicroPython обычно доступны подходящие библиотеки для самых распространенных компонентов SPI. Если установлена правильная библиотека, можно без проблем обмениваться данными с соответствующим устройством. Далее будут представлены некоторые периферийные компоненты. Различные компоненты можно разделить на следующие основные категории:

- Память (EEPROM и FLASH)
- Преобразователь сигналов (АЦП и ЦАП)
- Часы реального времени (RTC)
- Датчики (температура, давление, влажность, освещенность и т.д.)
- ЖК-контроллер, цифровой потенциометр, USB-контроллер, микшер сигналов и т. д.

Большинство компонентов можно отнести к первым трем категориям: память, преобразователи и часы реального времени. С другой стороны, компоненты из последних двух групп зарезервированы для специальных приложений.

В области АЦП доступны различные тактовые частоты, номера каналов и разрешения. Разрешение битов варьируется от 8, 10 и 12 до 24 бит. Доступные тактовые частоты начинаются с 30 тысяч отсчетов (КС) в секунду и достигают более 600 тысяч отсчетов в секунду для высокопроизводительных ИС.

Если скорости или разрешения внутренних АЦП ESP32 недостаточно для конкретного приложения, подходящим решением может стать преобразователь SPI.

EEPROM преимущественно доступны в виде микросхем памяти. Емкость памяти EEPROM варьируется от нескольких бит до нескольких килобит. Тактовые частоты до 3 МГц доступны для скоростей хранения. Особый случай карт SD и micro-SD рассматривается в разделе 18.4.

В сенсорном секторе сравнительно мало SPI-устройств. Поскольку здесь часто требуется только низкая скорость передачи данных, шина I²C часто предпочтительнее. Такие измерения, как определение температуры, влажности, концентрации газа, скорости ветра, интенсивности света и т. д., не требуют чрезвычайно высоких скоростей передачи данных и также могут быть обнаружены с помощью медленных интерфейсов, так что можно обойтись без высоких скоростей передачи данных шины SPI.

18.3 Члены семейства SPI

В следующей таблице показаны некоторые из наиболее важных компонентов SPI:

Модуль	Функция
74HC595	Регистр сдвига
AD8403	Цифр. потенциометр
MAX7219	Драйвер LED дисплея
MCP4912	10-битный ЦАП
NRF24L01	Радиомодуль
RC522	RFID-транспондер

Таблица содержит лишь небольшой отрывок из обширного семейства SPI. В основном были выбраны устройства, для которых доступен драйвер MicroPython. Драйвер дисплея для светодиодных матриц уже использовался в Разделе 11. Транспондер RFID RC522 был представлен в главе 13.

Кроме того, SPI-подключение дисплеев является интересной альтернативой. Использование OLED-дисплеев с шиной I²C в главе 10 содержит соответствующие примеры.

18.4 Управление картами SD и µSD через SPI

Карты Secure Digital (SD) или Micro Secure Digital (µSD) также могут быть адресованы через интерфейс SPI. Карты памяти используются в цифровых камерах, смартфонах и т. д. в качестве запоминающих устройств. Эти карты также можно использовать с ESP32. Некоторые платы, такие как плата NodeESP, имеют внутренний слот для SD-карты. Хотя он несколько неудобно расположен на нижней стороне платы, его все же можно использовать для легкого и недорогого расширения памяти ESP32 до размера в несколько гигабайт.

SD-карты взаимодействуют с контроллером по протоколу SPI. Таким образом, все модули или библиотеки SD являются адресуемыми. Опять же, соединение картридера с контроллером осуществляется через уже известные пины. Это означает, что соответствующие выводы ввода-вывода больше нельзя использовать для других целей.

На рисунке показаны соответствующие соединения на SD-карте. Использование SD-карт особенно просто, если вы используете готовые SD-модули. Таким образом, вам не нужно беспокоиться о назначении контактов. После подключения модуля или его подключения к SD-карте все необходимые сигналы подключены правильно.

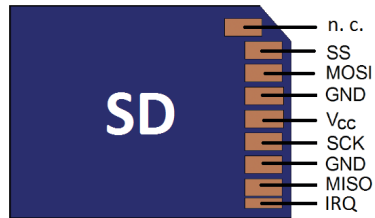


Рис.18.6: Интерфейс SPI на SD-карте.

Если вы хотите увеличить объем памяти ESP32, SD-карты — хороший и недорогой вариант. С их помощью доступную память можно расширить до гигабайтного диапазона. Из-за массового производства SD-карты дешевле и доступны с большей емкостью, чем внешние EEPROM.

Глава 19 • Создание схем с помощью компонентов и макетов

Если вы хотите работать с MicroPython на ESP32, вам практически всегда потребуются другие электронные компоненты. В простейшем случае это может быть только внешний светодиод с последовательным резистором. Однако по мере увеличения сложности проектов увеличивается и количество необходимых компонентов. Поэтому в этом разделе кратко описаны наиболее важные основные компоненты, такие как:

- Соединительные кабели (перемычки)
- Резисторы
- Светодиоды
- Конденсаторы

С большим опытом приходит возможность реализовывать более обширные проекты. Чтобы улучшить свои навыки, доступны различные методы проектирования электронных схем, тестирования и поиска неисправностей. Одним из вариантов является создание печатной платы, на которую вы припаиваете компоненты. Это позволяет создавать надежные и долговечные схемы и устройства.

Кроме того, для хобби широко используются специальные макетные платы. Они имеют регулярную структуру токопроводящих дорожек, что позволяет реализовывать более сложные варианты схем с элементами и соединительными проводами. Однако этот метод не очень подходит для экспериментальных целей, так как всегда необходимо перепаявать для модификаций или вариантов. Поэтому в этой области утвердились так называемые «макеты». Это беспаячные платы, которые позволяют создавать даже сложные схемы без пайки. Эти платы идеально подходят для обучения и экспериментов. На следующем рисунке показана «система игровой площадки», состоящая из двух плат, на которых можно построить более крупные схемы с различными датчиками, элементами управления, дисплеями и т. д.

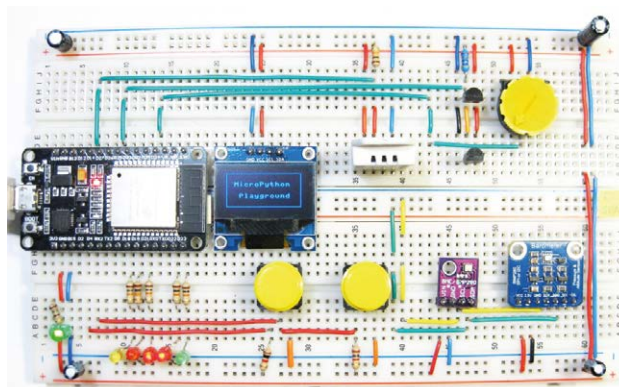


Рис.19.1: Игровая площадка, состоящая из двух больших макетных плат.

19.1 Макеты платы

Для соединения плат ESP32 с другими электронными компонентами без пайки можно использовать различные беспаячные макетные платы разных размеров.

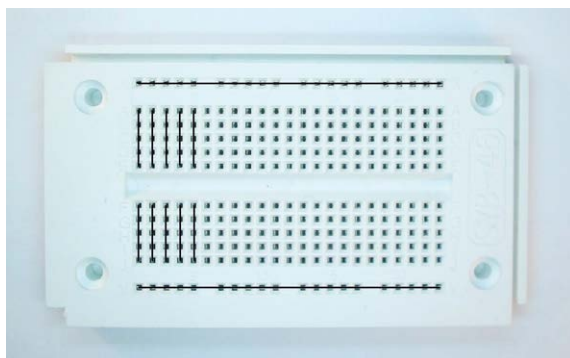


Рис.19.2: Типичный макетная беспаячная плата.

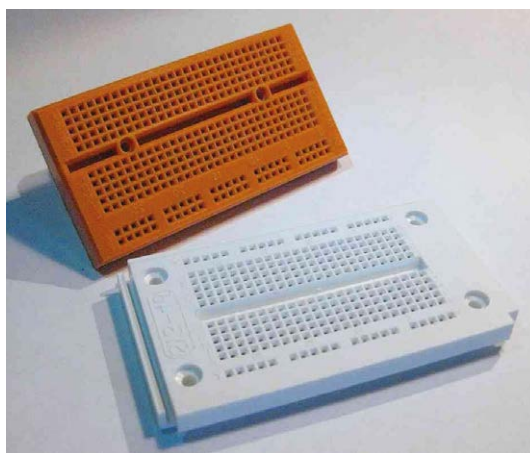


Рис.19.3: Варианты макетной платы

Макетные платы доступны в самых разных формах, цветах, дизайнах и размерах. Многие варианты имеют разъемы по краям, чтобы можно было соединить несколько плат.

Такие «вставные платы», «вставные платы для экспериментаторов» или просто «макеты» сегодня широко используются в электронной промышленности, а также в домашних лабораториях для разработки схем. При правильном использовании они имеют долгий срок службы.

Тонкие соединительные провода могут легко гнуться при подключении, поэтому ввод проводов компонентов в отверстия вставной платы всегда должен выполняться строго вертикально. В частности, если макетная плата еще новая, для вставки проводов может потребоваться некоторое усилие. Более прочный пинцет или плоскогубцы могут помочь при вставке соединений. Идеально, если кончик инструмента покрыт резиновой трубкой или клейкой лентой, так как это защищает компоненты и обеспечивает лучший захват инструмента. При необходимости можно немного расширить отдельные пружины внутри платы, вставив штифт. Однако не стоит переусердствовать, так как это может привести к проблемам с контактом в дальнейшем.

19.2 Перемычки и соединительные кабели

Для соединения на макетной плате можно использовать отрезки проводов. Однако они часто не обеспечивают хорошего электрического контакта. Кроме того, оголенные концы проводов легко сгибаются и в конечном итоге обламываются. Так называемые «прыгуны» здесь намного лучше. Это гибкие многопроволочные провода с контактными штифтами на обоих концах. Это позволяет выполнять электрические соединения быстро и безопасно.

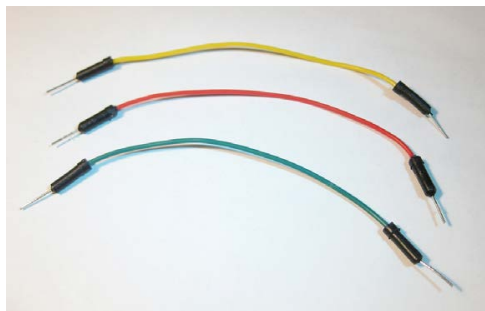


Рис.19.4: Соединительный проводник (перемычка).

Альтернативой перемычкам являются так называемые кабели Dupont. Их можно использовать для подключения таких компонентов, как датчик DHT22 или модуль OLED, к макетной плате.

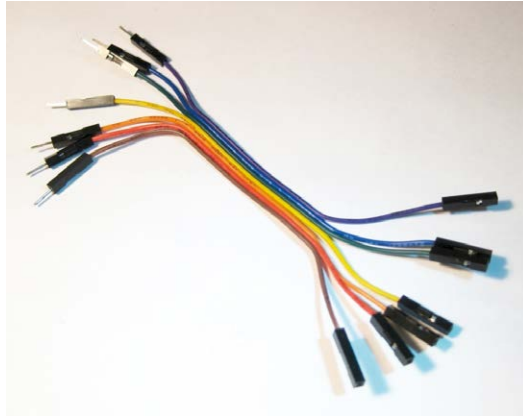


Рис.19.5: Кабель Dupont.

Штыревые разъемы, изображенные на рис.19.6, также являются подходящими соединительными устройствами.

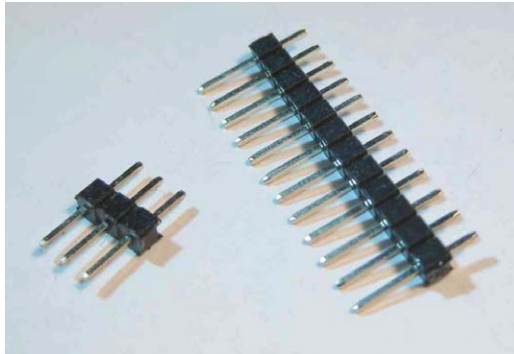


Рис.19.6: 3-контактный и 12-контактный разъемы.

Перемычки Dupont также можно подключить к макетной плате:

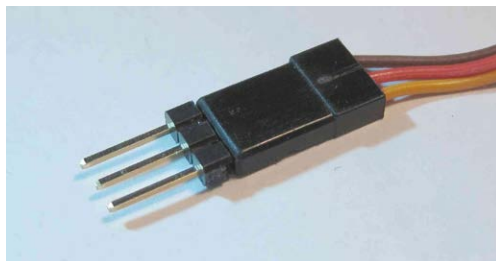


Рис.19.7: Кабель Dupont с подключенным 3-контактным разъемом.

19.3 Резисторы

Резисторы являются одними из основных компонентов в электронике. Они неполяризованные, поэтому направление установки не имеет значения. Номинал резистора кодируется напечатанными цветными кольцами. Четыре цветных кольца требуются для резисторов с допуском 5%.

Три из них указывают значение сопротивления, четвертый имеет золотой цвет для допуска и указывает значение допуска. Для металлопленочных резисторов с допуском 1 % требуется пять цветных колец, здесь коричневое пятое кольцо указывает допуск. Кольцо для значения допуска можно узнать по тому, что оно немного шире остальных.

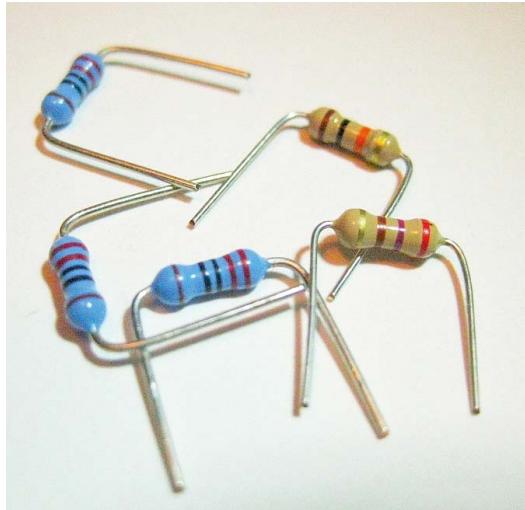


Рис.19.8: Выводные резисторы.

Резисторы номиналом 150 или 220 Ом часто используются в качестве последовательных резисторов для светодиодов или дисплеев на основе светодиодов. Также часто требуются значения 1 кОм, 10 кОм и 100 кОм (где кОм = килоом; кОм; 1000 Ом). При необходимости вы можете создать дополнительные значения самостоятельно, подключив их параллельно или последовательно (например, 500 Ом, подключив два резистора по 1 кОм параллельно, или 20 кОм, подключив два резистора по 10 кОм последовательно и т. д.). Для начала рекомендуются следующие значения:

Quantity	Value	Colour rings for 5% tolerance	Colour rings for 1% tolerance
10	220 ohms	Red - Red - Brown	Red - Red - Black - Black
10	1 kohms	Brown - Black - Red	Brown - Black - Black - Brown
5	10 kohms	Brown - Black - Orange	Brown - Black - Black - Red
5	100 kohms	Brown - Black - Yellow	Brown - Black - Black - Orange

В специализированных торговых и интернет-магазинах часто предлагаются ассортиментные коробки с наиболее важными значениями номиналов. В целом гораздо дешевле приобрести такую коробку, чем заказывать резисторы по отдельности.

19.4 Светодиоды

Светодиоды используются, например, для индикации состояния порта (HIGH/LOW). Они загораются только в том случае, если они правильно подключены. Если, вопреки ожиданиям, светодиод не загорается, сначала проверьте полярность. Катод можно определить по более короткому выводу.

При подключении стандартных LED к источнику напряжения всегда требуется последовательный резистор. Поскольку ESP32 работает с уровнями сигналов 3,3 В, для этой цели хорошо подходят резисторы на 220 или 150 Ом. Поскольку на LED падает от 1,6 В до 2,5 В — в зависимости от цвета — для падения последовательного резистора остается напряжение от 0,8 В до 1,6 В. Таким образом, ток LED составляет от 4 мА до 7 мА. Таким образом, LED светят с достаточной яркостью, а LED и контроллер работают в безопасном диапазоне.



Рис.19.9: Светодиоды.

19.5 Конденсаторы и электролитические конденсаторы

В дополнение к резисторам, конденсаторы также являются одними из основных компонентов электроники. Простые конденсаторы не полярны. Однако следует соблюдать осторожность при использовании электролитических конденсаторов. Всегда следите за соблюдением полярности, так как при переплюсовке они могут взорваться. Вот почему электролитические конденсаторы всегда имеют соответствующую маркировку.

В микроконтроллерной технике точные номиналы электролитических конденсаторов менее важны. Они часто нужны в качестве так называемых блокировочных конденсаторов для подавления помех, поэтому точное значение не критично. Например, вместо 10 мкФ практически всегда можно использовать значение 33 мкФ, если оно доступно. Для начала достаточно нескольких значений, которые сведены в следующую таблицу.

Число	Значение	Маркировка
Около 10	10 мкФ, 16V	10μ
Около 5	100 мкФ, 16V	100μ

На следующем рисунке показаны электролитические конденсаторы в радиальных корпусах:

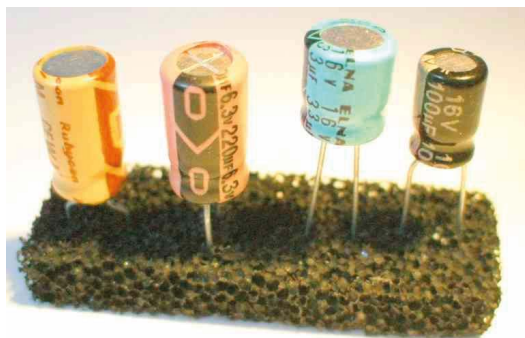


Рис.19.10: Электролитические конденсаторы.

Если в цифровых схемах время от времени возникают ошибки, особенно в более сложных схемах, рекомендуется добавить несколько электролитических конденсаторов на шины питания — естественно, с соблюдением полярности.

При наличии осциллографа напряжение питания можно проверить более точно. Если становятся видны пики на шине питания, т. е. кратковременные провалы напряжения, следует использовать электролитический конденсатор 100-200 мкФ локально на плате, чтобы поддерживать чистое напряжение питания.

MicroPython для микроконтроллеров

Проекты с Thonny-IDE,
uPyCraft-IDE и ESP32

Язык программирования «Python» в последние годы пережил огромный подъем. Не в последнюю очередь его популярности способствовали различные одноплатные системы, такие как RaspberryPi. Но Python также нашел широкое применение в других областях, таких как искусственный интеллект (ИИ) или машинное обучение (МО). Таким образом, очевидно, что Python или вариант «MicroPython» также следует использовать в SoC (системы на кристалле).

Мощные контроллеры, такие как ESP32 от Espressif Systems, предлагают отличную производительность, а также функции Wi-Fi и Bluetooth по доступной цене. Благодаря этим функциям сцена Maker была взята штурмом. По сравнению с другими контроллерами, ESP32 имеет значительно больший объем flash и SRAM памяти, а также гораздо более высокую скорость процессора. Благодаря этим характеристикам чип подходит не только для классических Applications, но и для программирования на MicroPython.

Эта книга знакомит с применением современных однокристальных систем. Помимо технической подготовки, основное внимание уделяется самому MicroPython. После знакомства с языком полученные навыки программирования сразу же применяются на практике. Индивидуальные проекты подходят как для использования в лаборатории, так и для повседневного использования. Таким образом, в дополнение к фактическому эффекту обучения, основное внимание уделяется радости от создания полных и полезных устройств. Используя лабораторные макеты, можно с минимальными усилиями создавать схемы всех видов, превращая тестирование и отладку стопроцентно самодельных проектов в поучительное удовольствие.

Различные приложения, такие как метеостанции, цифровые вольтметры, ультразвуковые дальномеры, считыватели карт RFID или генераторы функций, делают представленные проекты идеально подходящими для практических курсов или предметных и учебных работ в области естественных наук или в классах.



Гюнтер Спаннер

Автор работает в области разработки электроники и управления технологиями для различных крупных корпораций более 25 лет.

В дополнение к своей работе в качестве лектора, он успешно публикует технические статьи и книги по темам электроники, микроконтроллеров и сенсорных технологий, а также курсы и учебные пакеты. Кроме того, он известен широкой аудитории благодаря известным лекциям специалистам и вебинарам.

ISBN 978-3-89576-436-3



9 783895 764363